An Introduction to Computation



Computer Science has a long history. History is empowering and fun. We build on our past. So let's learn where computing comes from, why we do it, and what we can to with it.

CONTENTS

What is Computation? • A Brief History • Other Histories • Beyond History • Big Ideas • Theories and Practices • Recall Practice • Summary

What is Computation?

Information is that which informs.

Computation is the *mechanical* generation of new information from old information.

Computations are expressed within a **programming language** and carried out with **automata**.

The study of computation as a discipline is therefore the study of languages and automata.

The human understanding of, and application of, computing has evolved over millennia. It has arguably enhanced human capabilities. There is a history here that is empowering to know. History is an essential component of the study of any discipline. However:

> *[Computing is] complete pop culture. The big problem ... [is that] the electronic media we have is so much better suited for transmitting pop-culture content than it is for high-culture content. I consider*

jazz to be a developed part of high culture. [Jazz aficionados take deep pleasure in knowing the history of jazz, but] pop culture holds a disdain for history. Pop culture is all about identity and feeling like you're participating. It has nothing to do with cooperation, the past, or the future—it's living in the present. I think the same is true of most people who write code for money. They have no idea where [their culture came from]. —Alan Kay

Woah. He's right. Let's do something about this.

A Brief History

Here are some important events, people, works, and machines to be aware of in order to have a proper context for the study of computation.

In ancient times

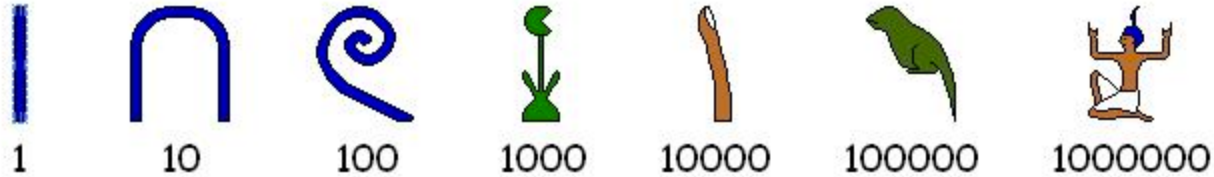
Since antiquity, humans have been marking, matching, comparing, tallying, measuring, and reckoning.



A big step

Early inventories featured pictorial markings like 🌽🌽🌽🌽🌽🍓🍓🍓. Then someone figured out they could save space by writing something like 5🌽 3🍓. An incredibly powerful idea: symbols representing quantities!

Numerals!



Early numeral systems like the Egyptian system were additive, but today positional systems are universal.

> ***Exercise**: Browse Wikipedia's List of Numeral Systems.*

Recipes for calculation

Recipes, or lists of instructions, for manipulating quantities and measurements were sometimes **recorded**. Notable examples include:

- Babylonian Tablets 1800-1600 BC
- Rhind Mathematical Papyrus (~1550 BC)
- Euclid's greatest common divisor method (~300 BC)
- Eratosthenes method for enumerating primes (~240 BC)
- Muḥammad ibn Mūsā al-Khwārizmī's Al-Jabr (813-833), which included systematic solutions for linear and quadratic equations





Calculating machines

While humans could follow recipes and carry out reckoning with their fingers and toes, or use tally marks on bones, gains in efficiency and accuracy naturally arise with the creation of mechanical devices for calculation. Here are just a few examples:

- Abacus (earliest known from Sumeria 2700–2300 BC)
- Astrolabe (~200 BC)
- Antikythera mechanism (~200 BC)
- Slide Rule (1620s)
- Pascal's calculators (1640s)

- Jacquard Loom (1804)
- Babbage's Difference Engine (1820s) and Analytical Engine (1837)
- Hollerith's Tabulating Machine (1880s)






The Analytical Engine was fascinating historically. Although never built, it was perhaps the world's first **programmable**, **general-purpose** computer. Several programs are known to have been written for the device, the most famous of which—a program for the computation of Bernoulli numbers—was presented by Babbage's collaborator Ada Lovelace in her translator's notes to Luigi Menabrea's article in *Taylor's Scientific Memoirs* in 1843. (Also see this shorter summary). Lovelace is often considered the world's first programmer. She is also widely admired as being perhaps the first to recognize the wide application of computing beyond arithmetic, writing that the Analytic Engine

> *might act upon other things besides number, were objects found whose mutual fundamental relations could be expressed by those of the abstract science of operations, and which should be also susceptible of adaptations to the action of the operating notation and mechanism of the engine.... Supposing, for instance, that the fundamental relations of pitched sounds in the science of harmony and of musical composition were susceptible of such expression and adaptations, the engine might compose elaborate and scientific pieces of music of any degree of complexity or extent.*

Philosophers, mathematicians, and logicians take interest

The 19th century saw wild advances in formal logic and the foundations of mathematics. Logic broke free of Aristotlean syllogisms. Cantor introduced us to the paradise of multiple infinities. Three philosophies about math even was emerged:

Logicism	Intuitionism	Formalism

Math is just logic

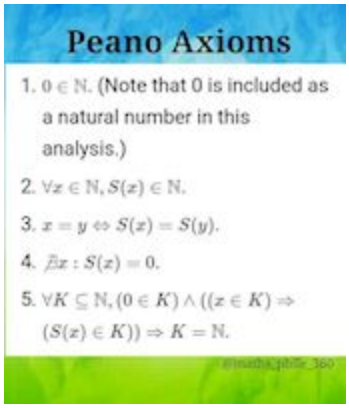
Math is only what we invent and can construct or demonstrate

Math is done by manipulating symbols according to rules

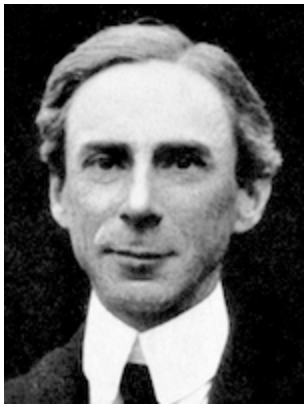
Major works of this era include:

- Boole's Boolean Algebra (1847)
- Frege's Begriffsschrift (1879)
- Peano's Axioms (1889)
- Hilbert's Program (early 1920s)
- Russell and Whitehead's *Principia Mathematica* (1910–1913)









People learned that formalizing mathematics, or even giving it a logic foundation, was not so direct. Any attempt at providing a consistent, formal, axiomatic basis for mathematics (that included arithmetic) could never capture all truths. A shocking result at the time! Gödel proved this in 1931.

Computation gets formalized

Though Gödel showed there could be no logically consistent and complete formalization of all of mathematics, the question of whether there existed an effective, finite, **decision procedure** for all statements of mathematics remained open. (Yay, we're back to computation!) This question is known as the Entscheidungsproblem. Hilbert thought such a procedure existed. Others did not. To prove him wrong, one needed to formalize the very idea of algorithm. The three main approaches of the 1930s were:

- Church's Lambda Calculus

- The μ-recursive Functions (mostly worked out by Gödel, Kleene, and others)
- Turing Machines





Church and Turing both showed the Entscheidungsproblem had no solution. Poor Hilbert.

> ***Story Time***
>
> *We'll go over the story of Church, Gödel, and Turing in class only. You can't find everything on the notes!*

Turing

Turing's paper is one of the most impactful and significant of all time.

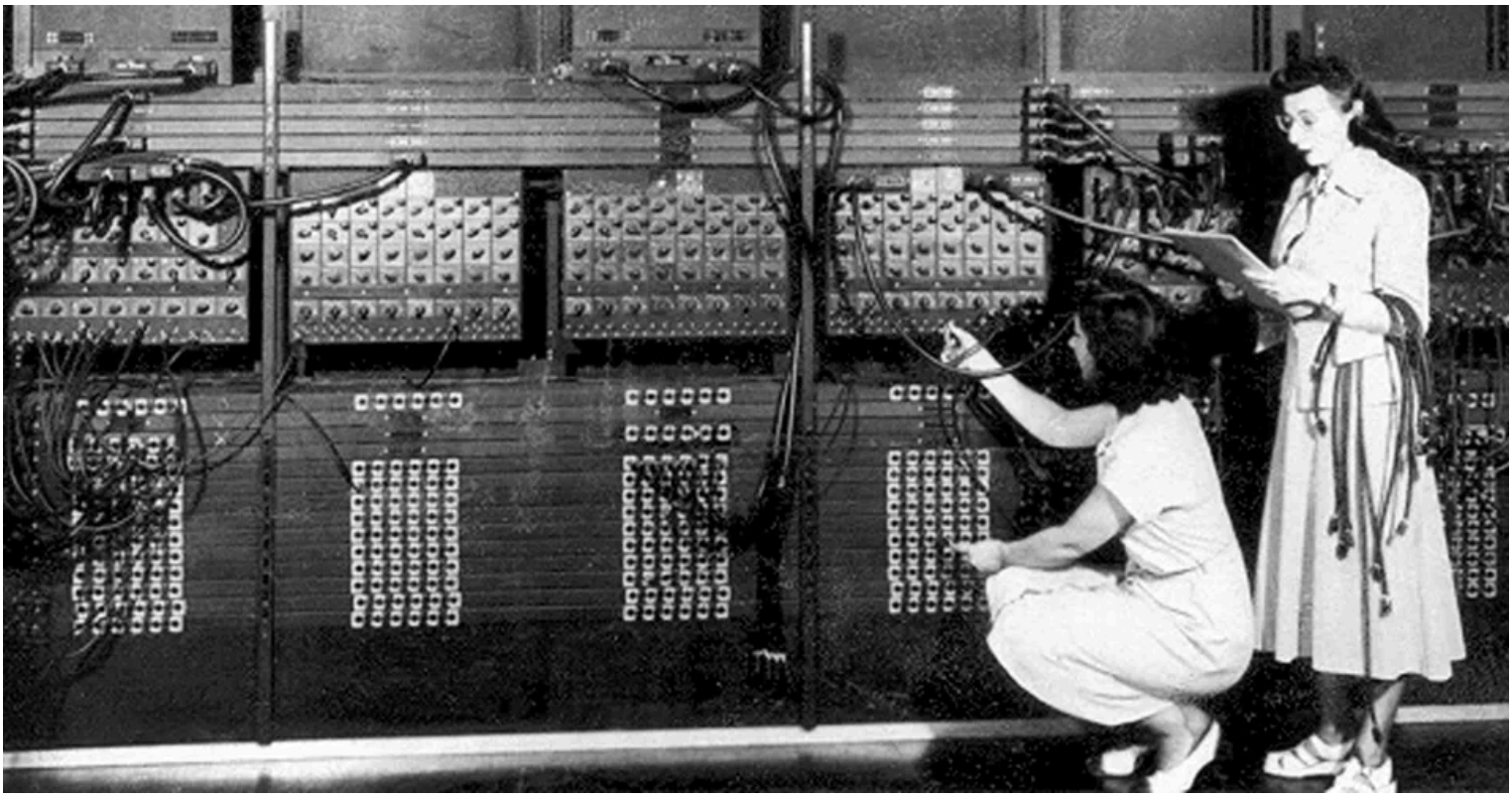
- He gave a **formalization of effective computation** (algorithms) in a way people could all understand
- He gave evidence as to why his formalization, now called the Turing Machine, **coincides with our intuitive notion of "computable"** (the Church-Turing Thesis).
- He introduced **computational universality**: programs were just data (ok so far, not surprising) and, wait for it, a single machine could emulate, or "run," any other machine (!!!!!!!!!!!!!!!!). Before this work, it was not known (but it may, we don't know, have been suspected by some) that a truly universal computer could exist—separate machines were thought to be necessary for different tasks.
- He showed that some numbers were not computable, some functions were not computable, and that some questions could not be mechanically decided—and hence, that the Entscheidungsproblem had no solution.

Details coming soon

We'll study Turing's paper, Turing machines, and the notion of universal computation in detail later in this course. Why? Turing is revered as the founder of computer science because of this work. The ideas that there are limits to computation, equivalent ways to express computation, and the existence of a truly universal computing machine make computer science a discipline.

Electronic computing machines

Turing's Universal Machine (1936) opened up a flood of new work in general purpose machines. Wikipedia's History of Computing Hardware page covers both ancient devices and a fair number of machines from the 1930s–1960s. It is worth a look to see how we eventually got to the electronic devices that are ubiquitous today.



Programming Languages

von Neumann once angrily asked someone writing an assembler "Why would you ever want more than machine language?" Today that question sounds silly. Even assembly language is too low-level.

One of the biggest proponents of high-level languages was Grace Hopper, who also happened to write the first compiler. Here are some notable high-level languages:

- Fortran, COBOL, LISP
- Algol, PL/1, Simula
- APL, J, K, Q
- CLU, Alphard, Euclid, Mesa, Cedar, Turing, Ada
- BCPL, B, C, C++, D, Rust, Zig, Odin, Swift
- Java, C#, Scala, Kotlin
- Perl, PHP, Hack, Tcl, Python, Ruby, Groovy, Lua
- Simula, Smalltalk, Object Pascal, Eiffel
- Concurrent Pascal, Occam, Linda, SR, Erlang, Elixir, Parasail, Chapel, Go
- Self, JavaScript, ActionScript, CoffeeScript, TypeScript
- Standard ML, CAML, Ocaml, F#, Elm, Reason
- Miranda, Haskell, PureScript, Idris
- Prolog, Mercury, Oz, Curry
- Agda, Coq, Lean
- Snobol, Icon
- Id, Val, Sisal
- SQL, Cypher, SPARQL
- Postscript
- MATLAB, R, Julia, Mojo
- Intercal, Unlambda, Brainfuck, Malbolge, Befunge, ZT, Java2K, LOLCODE
- GolfScript, Pyth, CJam, Jelly, 05AB1E, Stax



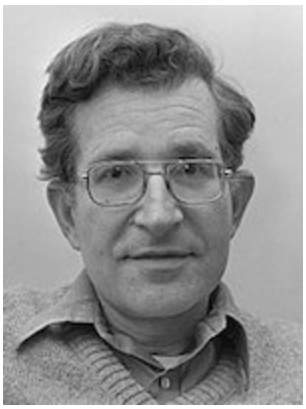



***Exercise**: The languages above are informally grouped. Try to determine, for each line, which characteristic is shared by the languages on that line. You'll likely need to do a lot of research, so don't spend too much time on this. Just focus on the lines that may interest you the most.*

Language Implementation

Languages have to not only be expressive, but they have to be efficiently implementable. This is why we study compilers and interpreters. Two major parts of this field are:

- Linguistic Theories (from Chomsky and others) that enable parsing to be efficient
- Optimizations (Frances Allen was a major figure in this area)




Human-centric computing

People create programs for people. Areas with people-focus include:

- Graphics and user interfaces
- Computing in education
- Libraries for creative computing



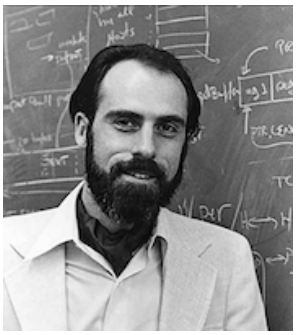



***Exercise**: Read about Douglas Engelbart's Mother of All Demos (1968).*

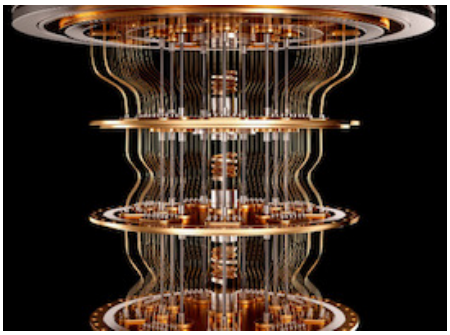
Modern Trends

Trends in the modern computing era:

- Personal computers (1970s)
- Internet (1980s)
- Web (1990s)
- Mobile (2000s)
- Cloud (2010s)
- Generative AI (2020s)
- Quantum Computing (?)



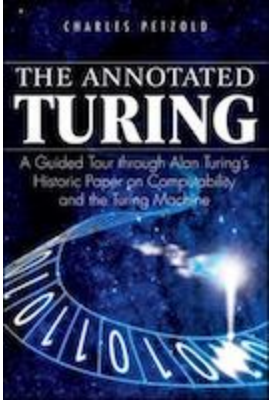




Other Histories

Computing history is so much richer than the brief outline above. The following are highly recommended:

- Gabriel Robins' remarkable A Brief History of Computing
- Wikipedia's Computing History article
- Wikipedia's Computing Timeline article
- Charles Petzold's fabulous book The Annotated Turing



Beyond History

Computing is so much more than its technical core, its theoretical concepts, or even its history. Computing is a human activity. Computations, programming languages, and machines, are designed by humans for humans.

Grady Booch is building a remarkable [transmedia documentary on the intersection of computing and the human experience](#).



The web version introduces itself as:

> *The story of computing is the story of humanity: this is a story of ambition, invention, creativity, vision, avarice, power, and serendipity, powered by a refusal to accept the limits of our bodies and our minds. Computing: The Human Experience is a transmedia project that explores the science of computing, examines the connections among computing, individuals, society, and nations, and by considering the history of computing contemplates the forces that will shape its future.*

“powered by a refusal to accept the limits of our bodies and our minds”

It is worth immersing yourself in this content for hours.

> ***Exercise**: Spend an hour exploring the site.*

Big Ideas

During humans’ creation of a science, art, and craft of computing, several major themes have emerged. They form the core of the field that we now call **computer science**. Among these themes, or big ideas, are:

- ✅ Symbols and symbol manipulation **amplify** our thought processes.
- ✅ The functions that can be computed comprise an **infinitesimal fraction** of the functions we know that exist.
- ✅ Theoretical computing devices (such as Turing Machines) are important for showing the **limits of computation**, but are not suitable for real computation.
- ✅ **Universal computation** is far, far from obvious, but we take it for granted today.
- ✅ Modern computers speed up time to such an extent that what we know and what we can know is **qualitatively different** than what we could know before computing.
- ✅ **Programming languages** enable the expression of computation at such a high level that creative and impactful computing become accessible to the masses.

Theories and Practices

In studying any discipline, humans create **theories**. A theory is an organized body of knowlege with explanatory and predictive powers. There are four major theories of computation:

- **Language Theory**, dealing with how computations are expressed
- **Automata Theory**, dealing with how computations are carried out

- **Computability Theory**, dealing with what is and is not computable
- **Complexity Theory**, dealing with how hard (in terms of resources like time and space) computations are

The practice of computing involves gaining proficiency in crafting computations (programs) in dozens of different programming languages and in many different programming styles, applying concepts, patterns, and idioms. It also includes the design and implementation of new programming languages and even new machines.

Designing one's own language and implementing compilers and interpreters are essential activities in building a deeper understanding of computation.

Recall Practice

Here are some questions useful for your spaced repetition learning. Many of the answers are not found on this page. Some will have popped up in lecture. Others will require you to do your own research.

What is computation? HIDE ANSWER

The formal, mechanical, generation of new information from old information.

What is a programming language? HIDE ANSWER

A language for expressing computations.

What is an automaton? HIDE ANSWER

A machine for carrying out computations.

According to Alan Kay, why is computing history not widely known? HIDE ANSWER

Because pop culture, which computing arguably is, holds a disdain for history.

What is the significance of numerals? HIDE ANSWER

They allow large quantities to be expressed with fewer symbols.

Show how to represent the quantity twenty-one thousand two hundred thirty-seven using Egyptian Numerals: HIDE ANSWER



What are some ancient media on which early recipes for calculation were recorded?
HIDE ANSWER

Babylonian Tablets and the Rhind Mathematical Papyrus.

What are some early famous nontrivial algorithms? HIDE ANSWER

Euclid's greatest common divisor method and Eratosthenes method for enumerating primes

What were some of the accomplishments of Al-Khwārizmī? HIDE ANSWER

He wrote *Al-Jabr*, which included systematic solutions for linear and quadratic equations.

Where and when were abaci first known to be used? HIDE ANSWER

Sumeria 2700–2300 BC.

What could the Antikythera mechanism do? HIDE ANSWER

It was an analog computer that could predict astronomical positions and eclipses.

Babbage is known for designing to machines. Which ones? HIDE ANSWER

The Difference Engine and the Analytical Engine.

Was the analytical engine ever built? HIDE ANSWER

No.

What was Ada Lovelace's published program in her famous *Notes*? HIDE ANSWER

A program for the computation of Bernoulli numbers.

What was Lovelace most known for besides the first published program? HIDE ANSWER

She was arguably the first to recognize and to write about the wide application of computing beyond arithmetic.

What did George Boole introduce to the world? HIDE ANSWER

Boolean Algebra.

What was Hilbert's Program? HIDE ANSWER

A call to people of his time to formalize all of mathematics.

What did the logicists think math was? What did the formalists think it was? The intuitionists? HIDE ANSWER

Logicists: a branch of logic. Formalists: a game of manipulating symbols according to rules. Intuitionists: an invention for constructing objects and facts about them.

What was the Entscheidungsproblem? HIDE ANSWER

Hilbert's question of whether there was an effective, finite, decision procedure for all statements of mathematics.

How did Church formalize the notion of effective computability? HIDE ANSWER

With the Lambda Calculus.

How did Turing formalize the notion of effective computability? HIDE ANSWER

With the Turing Machine.

What was Gödel's initial reaction to Church's Lambda Calculus, and what did this reaction motivate Gödel to do? HIDE ANSWER

He was skeptical that the Lambda Calculus was a model for effective computability, so he created his own mode, the μ-recursive functions.

After Church demonstrated the equivalence of the Lambda Calculus and the μ-recursive functions, what was Gödel's reaction? HIDE ANSWER

He thought his own model, the μ-recursive functions, must be insufficient.

When did Gödel finally accept the Lambda Calculus a model for effective computability? HIDE ANSWER

After Turing showed that the Lambda Calculus and Turing Machines were equivalent. All agreed that Turing machines were a sufficient model, thus the other two approaches must also be.

Name four stunning achievements of Turing's famous paper. HIDE ANSWER

He gave a formalization of effective computation, showed that his formalization coincided with our intuitive notion of "computable", introduced computational universality, and showed that some numbers (and by extension some functions) were not computable.

Why was the ENIAC famous? HIDE ANSWER

It was the first general-purpose electronic computer.

Who wrote the first compiler and what was it called? HIDE ANSWER

Grace Hopper. The A-0 System.

Why are programming languages important? HIDE ANSWER

They enable the expression of computation at such a high level that creative and impactful computing become accessible to the masses.

Why is LISP so loved? HIDE ANSWER

Simple syntax and semantics, homoiconicity, macros.

Why is Ruby so loved? HIDE ANSWER

It is extremely expressive, allows for rapid development, and has excellent metaprogramming facilities.

Why is CLU so significant? HIDE ANSWER

It introduced data abstraction, iterators, and exception handling.

What else is Barbara Liskov known for? HIDE ANSWER

She introduced the Liskov Substitution Principle.

What was Noam Chomsky's biggest contribution to computer science? HIDE ANSWER

He introduced the Chomsky Hierarchy, which served as a foundation for modern parsing theory.

What is Frances Allen known for? HIDE ANSWER

She was a (if not the) major figure in the field of compiler optimization. It is said that among all the optimizations performed in modern compilers, she probably came up with or at least described 80% or more of the most important ones.

What year did Allen win the Turing Award? HIDE ANSWER

2006.

What even is Generative AI? HIDE ANSWER

AI that can create new things, like images, music, code, or text.

Where can I find more histories of computing? HIDE ANSWER

Gabriel Robins' A Brief History of Computing, Wikipedia's Computing History article, Wikipedia's Computing Timeline article, and Charles Petzold's The Annotated Turing.

What can I find at *Computing: the Human Experience*? (You should visit and explore the site to answer) HIDE ANSWER

Histories, timelines, stories, articles, films, interactive exhibits, and educational resources on computing and its impact on humanity (considering individuals, cultures, nations, and societies).

What are the four major theories of computation? HIDE ANSWER

Language Theory, Automata Theory, Computability Theory, and Complexity Theory.

Learning about computing requires a studying languages and automata. Why do we study these two subjects? HIDE ANSWER

Languages are for expressing computations, and automata are for carrying out computations.

Summary

We've covered:

- ✅ Earliest notions of computing
- ✅ Numbers as abstractions of quantities
- ✅ Numerals as representations of numbers
- ✅ Earliest known recordings of recipes for calculations
- ✅ Early computing machines
- ✅ Influence of the analytical engine
- ✅ Ada Lovelace's insights
- ✅ Math in the late 1800's
- ✅ The formalization of computing
- ✅ Undecidability
- ✅ Human-centric computing
- ✅ The modern era in computing
- ✅ Several big ideas of computing
- ✅ Computing Theories

Language Design



So, you want to design your own language? Of course you do. Or perhaps you are taking a class and are being forced to create a programming language under penalty of a bad grade. What kinds of things do you need to know?

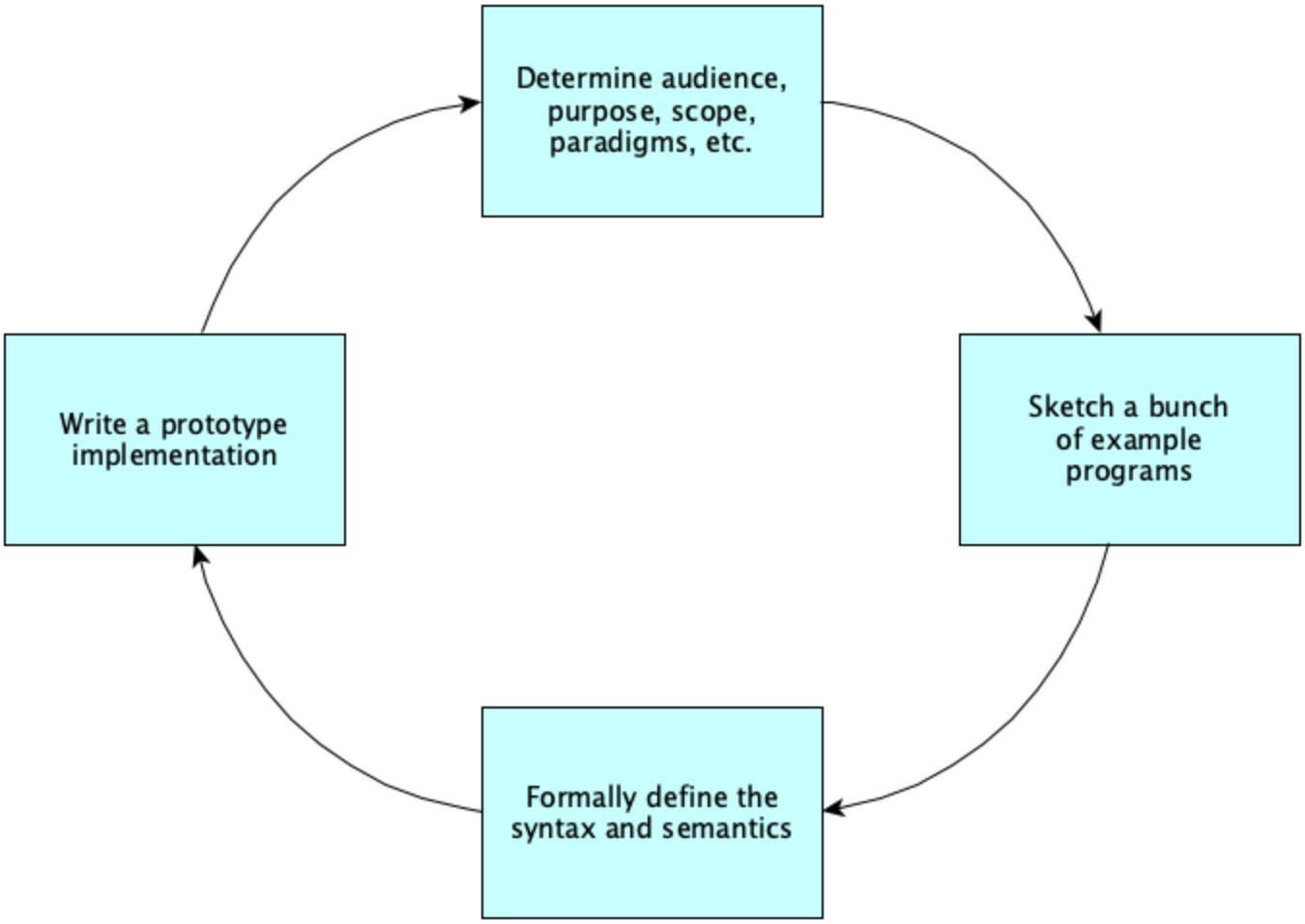
CONTENTS

Designing a Language • Prerequisites • Getting Ready • Getting Started • Choosing a Starter Set of Features • From Features To Abstract Syntax • From Abstract Syntax to Concrete Syntax • Defining Your Language • Prototyping • Examples • More Examples • Recall Practice • Summary

Designing a Language

Of course you want to design (and implement!) your own programming language! It's fun. It's creative. It's empowering.

How do we do it? In a nutshell, the process is iterative, cycling between four phases:



Doing the phases over and over is important; for example, while writing the compiler, you may be like “woah this is impossible” and then realize “oh shoot this part of the language wasn’t designed right!”

Prerequisites

It helps to be experienced. If you’re not, that’s okay, actually—you might get lucky!

But don’t mistake creativity for luck that shows up without pre-existing knowledge. The most creative people are those with a *lot* of knowledge and experience. So you should still study and practice!

Your success as a language designer will be massively aided by knowledge in three main areas:

1. **Programming Paradigms**. You should know a number of different ways that computations can be structured. These include: imperative | declarative | structured | object-oriented

functional | applicative | concatenative | logic | protocol-oriented | aspect-oriented | array | event-driven | dataflow | agent-based etc.

See Wikipedia's list of programming paradigms.

2. **Programming Language Concepts**. You should have a good sense of most, if not all, of the following things: Sequencing, conditional execution, iteration, recursion, functional decomposition, modularity, synchronization, metaprogramming, binding, scope, extent, volatility, subtyping, pattern matching, type inference, closures, prototypes, introspection, instrumentation, annotations, decorators, memoization, traits, streams, monads, actors, mailboxes, comprehensions, continuations, wildcards, promises, regular expressions, proxies, transactional memory, inheritance, polymorphism, parameter modes, type classes, generics, reflection, concurrency, parallelism, distribution, persistence, transactions, garbage collection, and many more terms.

   I'm working on a glossary of such terms that may be helpful to review.

3. **Existing ProgrammingLanguages**. It's nice to have a feel for a variety of languages. Here are a few that are good to be familiar with (and the reasons why they are good to study):

   - **Python**, for basic imperative programming and scripting
   - **Smalltalk**, for OOP done beautifully
   - **JavaScript**, for event-driven features, async, and promises
   - **Io**, for ideas on building up everything from a minimal semantics
   - **Julia**, for multiple dispatch
   - **Clojure**, **Racket**, **Scheme**, and **Common Lisp**, for Lispiness (macros, etc.)
   - **Standard ML**, **OCaml**, **F#**, and **Elm**, for Hindley-Milner typing and more
   - **Haskell**, for typeclasses and functional purity
   - **eToys**, **Scratch**, **Snap!**, and **GP** for the blocks approach
   - **Java** and **C#**, for being enterprisey
   - **Kotlin**, **Ceylon**, and **Scala**, as examples of evolving Java
   - **Erlang** and **Elixir**, for expressing concurrent processes in a distributed fashion
   - **Fortran**, **Chapel**, and **Parasail**, for expressing muticore and parallel programming
   - **PureScript**, **TypeScript**, and **ClojureScript**, as examples of evolving JavaScript
   - **C**, because it is the quintessential systems language
   - **C++**, **Rust**, **Zig**, and **Odin** for making systems programming less painful and more secure
   - **Go** and **Swift**, for more examples of modern, but generally mainline, ideas

- **J** and **K**, for array programming
- **Idris**, for dependent typing
- **Prolog** and **Mercury**, for logic programming
- **Forth** and **Factor**, for concatenative programming
- **Quipper**, because it's a language for quantum computing
- **Brainfuck**, **Malbolge**, **LOLCODE**, **Whitespace**, and other classic esoterics
- **GolfScript**, **CJam**, **Pyth**, **Jelly**, and other golfing languages, because this gives you a good sense of putting power into small syntactic constructs
- **Piet** and **Hexagony**, for, well, check them out....

Also check out this mini-encyclopedia of 70 languages.

Here are some excellent cross-language comparisons that help you to hone your understanding of how different syntaxes can express the same ideas:

- Syntax Across Languages
- Wikipedia Programming Language Comparisons
- Hyperpolyglot

These are really good too:

- Showcase of Languages
- Languages Through the Years

Getting Ready

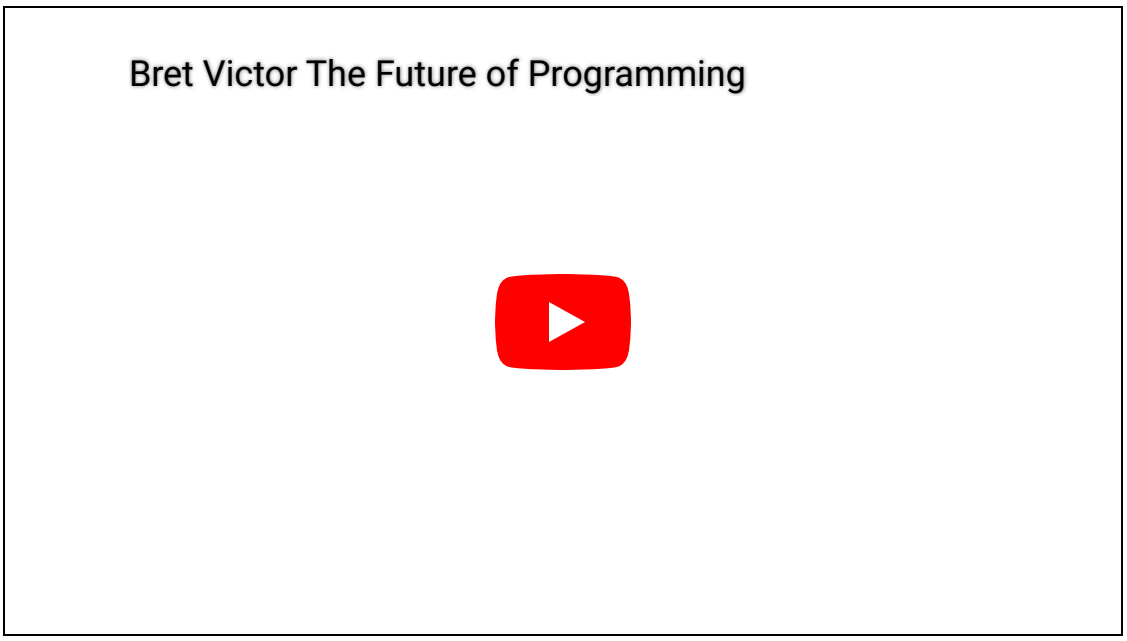
Remember, many people have designed languages before you. They made mistakes. They came up with brilliant ideas. Many were wildly successful. Some never made it big. Some people have brought in years of research on how people think and learn to come up with principles for language (and environment) design.

You should learn form their experiences.

Study classic papers. Read web essays. Visit online courses. Here is a small sampling of things to study and places to look for more information:

- Hints on Programming Language Design by Tony Hoare
- Learnable Programming by Bret Victor
- Five Questions about Language Design by Paul Graham
- For early classic papers, check out the massive bibliography of this paper
- Also look at the papers from Kathleen Fisher's Programming Language Design class

Think about the future:



And understand that traditional, mainstream programming languages, are not at all the epitome of computational expression. Languages can be much more:



Getting Started

Ready to strike out on your own? Here are some things to think about, in the form of a, you guessed it, a checklist:

- ✅ Do you have a specific audience in mind? Artists? Graphic designers? AI researchers? Numeric and Scientific nerds? Natural language types? Game developers? Machine learning people? Animators? High performance folks? System programmers? Or do you want a general purpose language? Or do you just want to do what *you* want?
- ✅ Understand the audience that the language is designed for, and what kinds of things they want to create with it (or problems they want to solve with it).
- ✅ Determine if your language is to be (1) a reasonable, usable language or (2) an esoteric/joke/golfing language.
- ✅ Determine if it is to be pragmatic, idealistic, researchy, or evil.
- ✅ Determine whether you want your language firmly in one camp—OO, functional, logic, concatenative, plain-imperative—or be a multiparadigm symphony. Or a multiparadigm cacophony.
- ✅ Determine whether it is to be built on a single characteristic building block (a *crystallization of style*) or one with a huge variety of syntactic forms for multiple semantic aspects (an *agglutination of features*).
- ✅ Determine your concurrency model. Do you want all your programs to be single-threaded and sequential? Or are you looking for something event-driven and async? Or multithreaded? Or distributed?

Choosing a Starter Set of Features

Come up with a list of capabilities, or features. Make sure they enable programmers to express their creations by following the suggestions and principles in the Learnable Programming essay, including:

- Make meaning transparent
- Explain in context
- Make flow and time tangible and visible
- Show the data, and avoid hidden state
- Use metaphor
- Allow easy decomposition and recomposition
- Allow the programmer to write readable code—you *do* support mentioning parameter names in calls, right?

***Exercise**: Skim the Learnable Programming essay. Or, better, read the whole thing ~~if~~ when you have time.*

What kind of questions might you have here? Here are some totally random ideas:

- If your language supports functions:
  - Must you pass the exact number of arguments the function expects?
  - Can you specify parameter names in the call?
  - Do you have rest parameters? optional parameters? keyword parameters? default parameters?
  - Are the parameters typed?
  - Can parameters be modified? Are they passed by value, value-result, reference, name, etc?
  - Are arguments evaluated L-R, R-L, in parallel, or in arbitrary order?
  - How does a function return a value? Can it return zero or more values or just one?
  - Are functions first-class? If so, do you use deep binding or shallow binding?
  - Is recursion allowed?
  - Can you test functions for equality?
  - Can functions be anonymous?
- What is your type system like?
  - Static vs. dynamic, strong vs. weak, manifest vs. implicit?
  - Do you distinguish primitive types from reference types?
  - Do you have all those different numeric types based on bit size? What about decimals and ratios?
  - Do you have a separate character type? A separate boolean type?
  - Can you get the type of an expression at runtime?
  - Are types objects?
  - Do you have supertypes and subtypes? Multiple-inheritance? If so, how do you handle name collisions?
  - Are types the same as classes? Can you add new types?
  - Do you have both sum and product types?
  - Do functions have a type?
  - Are there pointer types?
  - Are there parameterized types? If so, are they (mostly) covariant, contravariant, or invariant?
  - Any dependent types?
- What do your expressions look like?
  - And does your runtime follow eager or lazy evaluation? Or both?
  - Do you have only prefix, only postfix, only infix, or a wild mix of operators?
  - Can you overload operators? If so, how?

    - Can you change the precedence of operators? Even the built-in ones? At runtime?
    - How much type inference do you have?
    - Can you mark variables as mutable or immutable?
    - Are your variables bound or assigned? Can they be assigned more than once?
    - Do you have destructuring and/or pattern matching?
    - How do you determine scope? Do you implicitly or explicitly import into inner scopes?
    - Are there keywords that define access like `public`, `private`, and `protected`, or are there conventions, like in Go, where capitalized entities are implicitly exportable and lower-cased entities are private?
    - Is there a `let`-expression for super-local scopes?
    - Is shadowing allowed?
    - Do you have anything like JavaScript's `this` expression?
- How do you express control flow?
    - Do you like expression-orientation or do you have real statements?
    - How do you express sequential flow vs. nondeterministic flow vs. parallel flow?
    - If you have nondeterminism, how do you ensure fairness?
    - Must guards in a multiway selection be executed in any particular order?
    - Do you have short-circuit operators? Iterator objects? Generators?
    - Do you have all the loops: (1) forever, (2) n times, (3) while/until a condition, (4) through a numeric range (with an optional step size), or (5) through a collection?
    - Do you have `break` and `continue`? Anything like Ruby's `retry` and `redo`?
    - Do you have exceptions? Or do you need Go-style constructs? Or do you have nullables everywhere?
    - Do you have a timer for sleeping, delaying execution, or running on intervals?
    - Can I haz `goto`?
- How do you support concurrency?
    - Threads, or events?
    - Shared memory only? Message passing only? Or both? If shared memory, is it mutable? If so, do you have locks, higher-level synchronization devices, or some kind of transactional memory?
    - Do you support different levels of granularity of concurrency?
    - Are tasks spawned implicitly when their enclosing block begins, or do they require explicit invocation?
    - Do tasks die when the enclosing block or spawning task terminates? Or does the spawner wait for all internal tasks to terminate?
    - Do taks know who started them?
    - If an asynchronous task is launched, how is a result obtained? Callback, promise, or some other mechanism?

    - Is message passing done via named channels, named processes, or bulletin board?
    - Is message passing always synchronous, asynchronous, or both?
    - Is there timed or conditional synchronization mechanisms?
    - Can tasks detect their own state?
- How meta are you?
    - Can a program get a list of its global variables, loaded classes, top-level functions, etc.? Can a class get a list of its fields, methods, constructors, superclass(es), subclasses, etc.? Can a function get its parameters, locals, return types, etc.?
    - Can a function find out who called it?
    - Can a variable be read or set via its name (a string) only? Can a function or method be invoked by its name (a string) only? Can new variables or types or functions be created at run time, given only their name and some indication of how they should look?
    - Does the language have a macro system? If so, is it string-based or AST-based?
    - Is it possible to unquote within a macro?
    - Are macros hygienic? What syntactic devices are provided to explicitly capture in- scope variables, if any?

> ***What did I just read?***
>
> *Feeling like only about 20% of the questions above made any sense? Feeling like that vocabulary came out of nowhere? That's fine for now. Learning about programming languages can be a never-ending lifelong journey, but you can use the questions you don't quite understand now as a place to start some research.*
>
> *Oh, I have a glossary you might find helpful.*

From Features To Abstract Syntax

When you have a good idea of your language features, you'll want to figure out a good way to organize them, structurally. This is known as your language's **abstract syntax**. In an abstract syntax we don't worry much about punctuation and parentheses and such microscopic details. We are interested in the overall structure. Here's a look at abstract syntax trees in JavaScript:



When defining your language, you will want to specify exactly what the AST Nodes are, how they are related to each other, and what their properties are. For JavaScript, the AST Node types come from a specification called EsTree. I've summarized the main interfaces here:

```
Program sourceType:["script"|"module"] body:[Statement|ModuleDeclaration]
Statement
    Declaration
        FunctionDeclaration(Function) id:Identifier
        VariableDeclaration declarations:[VariableDeclarator] kind:("var"|"let"|"
        ClassDeclaration(Class) id:Identifier
    EmptyStatement
    DebuggerStatement
    ExpressionStatement expression:Expression
    BlockStatement body:[Statement]
    ReturnStatement argument:Expression?
    LabeledStatement label:Identifier body:Statement
    BreakStatement label:Identifier?
    ContinueStatement label:Identifier?
    IfStatement test:Expression consequent:Statement alternate:Statement?
    SwitchStatement discriminant:Expression cases:[SwitchCase]
    WhileStatement test:Expression body:Statement
    DoWhileStatement body:Statement test:Expression
    ForStatement init:(VariableDeclaration|Expression)? test:Expression? update:E
    ForInStatement left:(VariableDeclaration|Pattern) right:Expression body:State
        ForOfStatement await:boolean
    ThrowStatement argument:Expression
```

```
    TryStatement block:BlockStatement handler:CatchClause? finalizer:BlockStateme
    WithStatement object:Expression body:Statement
Function id:Identifier? params:[Pattern] body:BlockStatement generator:bool async
VariableDeclarator id:Pattern init:Expression?
SwitchCase test:Expression? consequent:[Statement]
CatchClause param:(Pattern?) body:BlockStatement
Expression
    ThisExpression
    Identifier(Pattern) name:string
    Literal value:(string|bool|number|Regexp|bigint)?
        RegExpLiteral regex:{pattern:string flags:string}
        BigIntLiteral bigint:string
    ArrayExpression elements:[(Expression|SpreadElement)?]
    ObjectExpression properties:[Property|SpreadElement]
    FunctionExpression(Function)
    ArrowFunctionExpression(Function) body:(BlockStatement|Expression) expression
    UnaryExpression operator:UnaryOperator prefix:bool argument:Expression
    UpdateExpression operator:UpdateOperator argument:expression prefix:bool
    BinaryExpression operator:BinaryOperator left:Expression right:Expression
    AssignmentExpression operator:AssignmentOperator left:Pattern right:Expressio
    LogicalExpression operator:LogicalOperator left:Expression right:Expression
    MemberExpression(ChainElement) object:(Expression|Super) property:Expression
    ChainExpression expression:ChainElement
    ConditionalExpression test:Expression consequent:Expression alternate:Express
    CallExpression(ChainElement) callee:(Expression|Super) arguments:[(Expression
    YieldExpression argument:Expression? delegate:bool
    TemplateLiteral quasis:[TemplateElement] expressions:[Expression]
    TaggedTemplateExpression tag:Expression quasi:TemplateLiteral
    NewExpression
    SequenceExpression expressions:[Expression]
    ClassExpression(Class)
    AwaitExpression argument:Expression
    ImportExpression source:Expression
    MetaProperty meta:Identifier property:Identifier
Class id:Identifier? superClass:Expression? body:ClassBody
ClassBody body:[MethodDefinition]
MethodDefinition key:Expression value:FunctionExpression kind:("constructor"|"met
SpreadElement argument:Expression
Property key:Expression value:Expression kind:("init"|"get"|"set") method:bool sh
```

```
    AssignmentProperty value:Pattern kind:"init" method:false
Pattern
    ObjectPattern properties:[AssignmentProperty|RestElement]
    ArrayPattern elements:[Pattern?]
    RestElement argument:Pattern
    AssignmentPattern left:Pattern right:Expression
Super
TemplateElement tail:boolean value:{cooked:string? raw:string}
ChainElement optional:boolean
enum UnaryOperator {"-"|"+"|"!"|"~"|"typeof"|"void"|"delete"}
enum UpdateOperator {"++"|"--"}
enum BinaryOperator {"=="|"!="|"==="|"!=="|"<"|"<="|">"|">="|"<<"|">>"|">>>"|"+"|
enum AssignmentOperator {"="|"+="|"-="|"*="|"/="|"%="|"**="|"<<="|">>="|">>>="|"|
enum LogicalOperator {"||"|"&&"|"??"}
ModuleDeclaration
    ImportDeclaration specifiers:[ImportSpecifier|ImportDefaultSpecifier|ImportNa
    ExportNamedDeclaration declaration:Declaration? specifiers:[ExportSpecifier]
    ExportDefaultDeclaration declaration:(Declaration|Expression)
    ExportAllDeclaration source:Literal exported:(Identifier?)
ModuleSpecifier local:Identifier
    ImportSpecifier imported:Identifier
    ImportDefaultSpecifier
    ImportNamespaceSpecifier
    ExportSpecifier exported:Identifier
```

I've built an interactive application for you to explore AST generation. Please try it out!.

***Exercise**: Please try it out.*

> ***Too much detail?***
>
> *You may notice that the EsTree specification has a lot more detail than the hand-drawn ASTs in the video above. This is okay. The specification is used to build actual compilers and interpreters, while for hand-drawn ASTs we just like to give the big picture and can take some "shortcuts."*

From Abstract Syntax to Concrete Syntax

Now it's time to think about what your language will really look like! Remember that design is creative and iterative, so you will want to begin, like all artists do, with sketches:

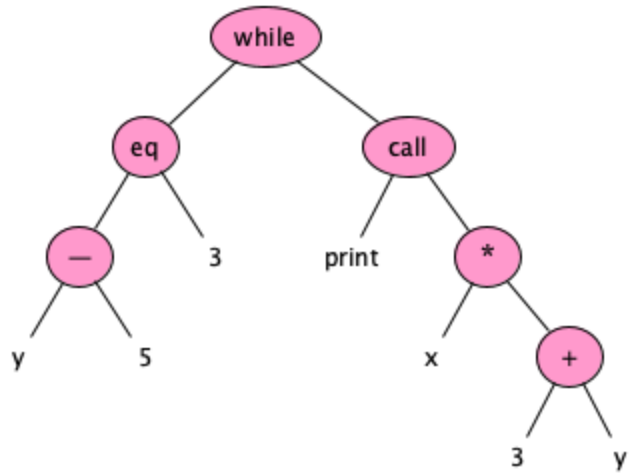
- Sketch programs or program fragments in your language.
- Sketch fragments that are *not* in your language.
- Come up with a nice, consistent, syntactic theme.

Do a lot of experimentation here! You will probably want to put creative effort into designing languages people like to use! What kind of syntax issues do they deal with? Dozens, actually, and we can't cover them all. But how about a taste of just a few. We'll peek at just a few issues that sometimes generate strong opinions.

Overall Phrase Structure

You will need to adopt a scheme for showing structure. The popular approaches are: Curly-brace (JavaScript, Java, C++, C#), Terminal-end (Ruby, Ada), Nested parentheses (Lisp, Clojure, Racket), Indentation (Python), Blocks (EToys, Scratch, Snap!), Pictures (Piet), Other (Haskell, Erlang, Prolog).

The important idea here is that a single abstract syntax can be realized with many different **concrete syntaxes**. A concrete syntax specifies exactly which strings of characters make up structurally valid programs. For example, the AST:



represents each of the following (and more!):

```
while y - 5 == 3:
    print(x * (3 + y))
```

```
while y - 5 == 3 {
  print(x * (3 + y))
}
```

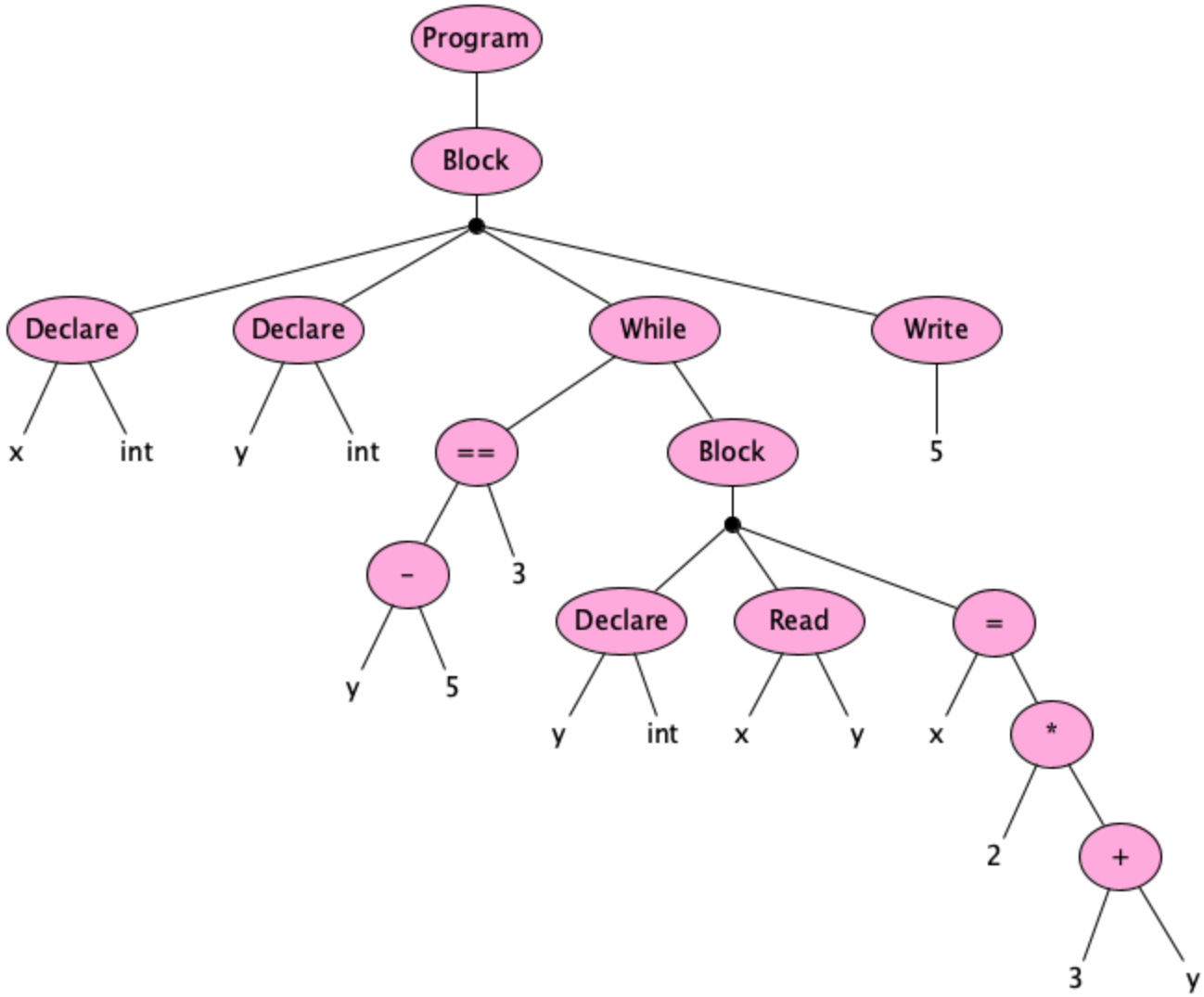
```
while y - 5 == 3 loop
  print(x * (3 + y))
end
```

```
(while (= (- y 5) 3)
    (print (* x (+ 3 y))))
```

> ***Exercise**: The last syntax is used in Clojure, Lisp, Scheme and others. Note that in some ways, this **IS** the abstract syntax. How so?*

> ***Exercise**: Here is another:* `y 5 - 3 == [x 3 y + * print] while`. *Do you know of, or can you find, any languages which have that kind of syntax?*

In addition to structure, your choice of keywords, operators, punctuation (or lack thereof) are part of your design. Here's an abstract syntax:



Let's try out a few things:

```
program:
    var x, y: integer
    while y - 5 == 3:
        var y: integer
        get(x)
        get(y)
        x = 2 * (3 + y)
    put(5)
```

```
int x, y;
while y - 5 = 3 {
    int y;
    STDIN -> x;
    STDIN -> y;
```

```
    x <- 2 * (3 + y);
}
STDOUT <- 5;
```

```
COMMENT THIS LOOKS LIKE OLD CODE
DECLARE INT X.
DECLARE INT Y.
WHILE DIFFERENCE OF Y AND 5 IS 3 LOOP:
    DECLARE INT Y.
    READ FROM STDIN INTO X.
    READ FROM STDIN INTO Y.
    MOVE PRODUCT OF 2 AND (SUM OF 3 AND Y) INTO X.
END LOOP.
WRITE 5 TO STDOUT.
```

```
(program
  (declare x int)
  (declare y int)
  (while (= (- y 5) 3)
    (define (y int))
    (read x y)
    (assign x (* 2 (+ 3 y)))
  )
  (write 5)
)
```

***Exercise**: Play around and sketch a few more.*

Delimiters

How to separate one construct from another is a really big issue in syntax design, believe it or not. We can identify two main classes of languages: those in which newlines are significant and those in which they are not.

"Insignificant" Newlines

In many languages, newlines are just like any other whitespace character (except for minor exceptions such as single-line comments and single-line string literals. Then, unless you have an S-Expression-based syntax as in LISP, Scheme, and Clojure, you'll need semicolons to terminate (or separate) statements. This means you *can* (but shouldn't) write code like:



```c
#define ZERO 0
    unsigned  gcd(   unsigned   int  // Euclid's algorithm
      x,unsigned   y) {   while ( /* hello */  x>   ZERO
   ){unsigned temp=x;x=y   %x;y  = temp ;}return

   y ;}
```

"Significant" Newlines

Where you place your newlines matters greatly in, let's see, Assembly languages, Python, Ruby, JavaScript, Elm, Haskell, Go, Swift, and yes, many others. The rules can get pretty technical.

Python scripts are defined as sequences of **logical lines**, delimited by the token NEWLINE. A statement may not cross logical lines, except in the case of **compound statements** in which each constituent **simple statement** ends with a NEWLINE. Logical lines are made up of one or more physical lines according to line joining rules. Lines are *implicitly* jointed within parentheses, brackets, or braces; lines can be *explicitly* joined by ending with a backslash. These rules are somewhat exclusive of comments and string literals.

Ruby looks at the end of each line and says "well if up to here it looks like we've completed a statement the we have." This means you have to be careful where you break lines:



```ruby
puts 5
  + 3
puts 5 +
  3
```

prints 5 then 8.

> ***Exercise**: Why?*

"Possibly Significant" Newlines

JavaScript requires most statements to be terminated by semicolons, but the *compiler will put one in for you if it looks like you might have missed one*. The rules by which this **automatic semicolon insertion** (ASI) is done have to be learned and they might be hard to remember.



If you are going to be a serious JavaScript programmer, you need to learning the rules of ASI **whether you choose to use semicolons or not**.

> ***Exercise**: Research the famous Rules of Automatic Semicolon Insertion. Which statements are supposed to be terminated by a semicolon? When is a semicolon inserted? Give four examples of how writing JavaScript in a "free-form" manner is impossible because of semicolon insertion.*

> ***Exercise**: Get your ASI Certification*

Some people feel very strongly whether to use or not to use semicolons:



douglascrockford commented — a year ago

That is insanely stupid code. I am not going to dumb down JSMin for this case.



douglascrockford commented — a year ago

TC39 is considering the use of ! as an infix operator. This code will break in the future. Fix it now. Learn to use semicolons properly. ! is not intended to be a statement separator. ; is.



fat commented — a year ago

i have learned to use them, that's why there isn't one present.



kitcambridge commented — a year ago

> i have learned to use them, that's why there isn't one present.

Zzzzing!

Function Calls

Most programming languages have functions. Seriously. But there are a lot of ways to work them into your design. Basic questions include: Must functions have exactly one argument, or zero or more arguments? Parens or no parens? Positional or keyword arguments? Argument labels? If no arguments, can we omit parentheses?

You can play around and see what you can come up with:

- `push(myStack, 55)`
- `push myStack 55`
- `[push myStack 55]`
- `(push myStack 55)`
- `push(on: myStack, theValue: 55)`
- `push(theValue: 55, on: myStack)`
- `push on:myStack theValue:55`
- `push({ on: myStack, theValue: 55 })`
- `push { on: myStack, theValue: 55 }`
- `push({ theValue: 55, on: myStack })`

You might want to consider an ultra-low precedence function application, like they have in Haskell and F#:

- `sum (filter even (map square a))`
- `sum $ filter even $ map square $ a`
- `sum <| filter even <| map square <| a`
- `a |> filter even |> map square |> sum`

The flip side of function calls is function definitions. You're likely familiar with default parameters, and rest parameters. Python has cool mechanisms for requiring arguments to be positional or keyword, based on the definition. Examples:

- `def sqrt(x, /)`
- `def line(*, x1, x2, y1, y2, width, style, color)`
- `def f(a, b, /, c, d, *, e, f)`

Syntactic Sugar

Syntactic sugar refers to forms in a language that make certain things easier to express, but can be considered surface translations of more basic forms.

This is best understood by example. There are zillions of examples out there. Here are a few. (**Disclaimer**: Some of these are just examples I made up and are not part of any real language.)

Construct	Desugared Form	Description
`x += n`	`x = x + n`	Compound assignment
`a + b`	`operator+(a, b)` *or* `"+"(a, b)` *or* `__add__(a, b)`	Common in languages that allow overloading
`a[i]`	`*(a + i)`	(C, C++ pointer arithmetic) And `i[a]` works too!
`p -> x`	`(*p).x`	(C, C++) Field of struct being pointed to
`f`	`f()`	Some languages let you leave off parentheses in calls with no arguments
`f x`	`f(x)` *or* `x.f()`	Some languages let you leave off parentheses in calls with one argument

`x op y`	`op(x, y)` *or* `x.op(y)`	Some languages let you leave off parentheses in calls with two arguments
`let x=E1 in E2`	`(x => E2)(E1)`	Let-expression (in functional languages)
`(E1 ; E2)`	`(() => E2)(E1)`	Expression sequencing (in eager functional languages)

`r = [s`
`  for x in a`
`  if e]`
	`r = []`
	`for x in a:`
	`  if e:`
	`  r.add(s)`
		List comprehension

`x orelse y`	`if x then x else y`	(Standard ML) short-circuit disjunction
`x andalso y`	`if x then y else x`	(Standard ML) short-circuit conjunction
`[x, y, z]`	`x :: y :: z :: nil`	Lists in Standard ML
`"a${x}b"`	`"a" + x + "b"`	String interpolation

***Exercise**: Find some more examples.*

When the sugared form is completely gratuitous or actually makes the code *less* readable, you sometimes hear the term **syntactic syrup** or **syntactic saccharin**.

Syntactic Salt

Here's the definition from The New Hacker's Dictionary:

> *The opposite of syntactic sugar, a feature designed to make it harder to write bad code. Specifically, syntactic salt is a hoop the programmer must jump through just to prove that he knows what's going on, rather than to express a program action. Some programmers consider required type declarations to be syntactic salt. A requirement to write "*`end if`*", "*`end while`*", "*`end do`*", etc. to terminate the last block controlled by a control construct (as opposed to just "*`end`*") would definitely be syntactic salt. Syntactic salt is like the real thing in that it tends to raise hackers' blood pressures in an unhealthy way.*

Candygrammars

Some people love verbose code, because explicit is better than implicit. But if you are language designer, be pragmatic: there is such a thing as code that is too verbose. What about trying to make the code like human language? Here's an example in Hypertalk (taken from Wikipedia):

```
on mouseDown
  answer file "Please select a text file to open."
  if it is empty then exit mouseDown
  put it into filePath
  if there is a file filePath then
    open file filePath
    read from file filePath until return
    put it into cd fld "some field"
    close file filePath
    set the textStyle of character 1 to 10 of card field "some field" to bold
  end if
end mouseDown
```

And here's an example from a language that some students developed, and regretted:

```
to get the truth value prime of whole number n:
    return no if n < 2
    for each d in 3 to n - 1 by 2:
        return no if d divides n
    end
    return yes
end
for each k in 1 to 100:
    write k if prime(k)
end
```

In practice this kind of verbosity is worse than it sounds. Here's what the New Hacker's Dictionary has to say about this:

> ***candygrammar /n./*** *A programming-language grammar that is mostly syntactic sugar; the term is also a play on "candygram." COBOL, Apple's Hypertalk language, and a lot of the so-called "4GL" database languages share this property. The usual intent of such designs is that they be as English-like as possible, on the theory that they will then be easier for unskilled people to program. This intention comes to grief on the reality that syntax isn't what makes programming hard; it's the mental effort and*

organization required to specify an algorithm precisely that costs. Thus the invariable result is that candygrammar languages are just as difficult to program in as terser ones, and far more painful for the experienced hacker.

[The overtones from the old Chevy Chase skit on Saturday Night Live should not be overlooked. This was a "Jaws" parody. Someone lurking outside an apartment door tries all kinds of bogus ways to get the occupant to open up, while ominous music plays in the background. The last attempt is a half-hearted "Candygram!" When the door is opened, a shark bursts in and chomps the poor occupant. There is a moral here for those attracted to candygrammars.]

Terseness

Some languages pride themselves on doing a whole lot with few characters:

An example from Ruby (do you see what this does?):



```
c = Hash.new 0
ARGF.each {|l| l.scan(/[A-Z']+/i).map {|w| c[w.downcase] += 1}}
c.keys.sort.each {|w| puts "#{w}, #{c[w]}"}
```

An example from APL (The 99 bottles of beer program taken from Rosetta Code):



```
bob  ←  { (⍕⍵), ' bottle', (1=⍵)↓'s of beer'}
bobw ←  {(bob ⍵) , ' on the wall'}
beer ←  { (bobw ⍵) , ', ', (bob ⍵) , '; take one down and pass it around, ', bobw
↑beer¨ ⌽(1-⎕IO)+⍳99
```

Here's APL again, with an expression to find all the prime numbers up to R:



```
(~R∈R∘.×R)/R←1↓⍳R
```

Some people love terse, concise code, because it says only what it needs and reduces the cognitive load, leaving you with less useless noisy syntax to learn. But if you are language designer, be pragmatic: there is such a thing as code that is too terse. Unless...that's your goal....

Golfing Languages

Golfing languages take terseness to the next level. A golfing language is a kind of esoteric programming language (a non-practical language created to experiment with weird ideas, be hard to program in, or be humorous) that allows programs to be written in an insanely small number of characters (or bytes).

Here are some CJam programs:

- `"Hello, world!"`
- `5{"Hello, world"oNo}*`
- `0X{_2$+}A*]N*`
- `l~@-@@-mh`
- `1{_B<}{_'**N+o)}w;`

Here are some Pyth programs (taken from the documentation):

- `"Hello, world!`
- `FNrZhTN`
- `FNUhTN`
- `VhTN`
- `K1FNr1hQ=K*KN;K`
- `.!Q`
- `WtQ=Q?%Q2h*Q3/Q2Q`
- `A(Z1;VhhTGA(H+GH`

> ***Exercise**: Find a bunch more examples of CJam and Pyth programs. Try them out. You can run them both at TIO. For Pyth, there's also a hosted interpreter with a Cheat Sheet.*

> ***Exercise**: Try out Stax.*

Defining Your Language

Traditionally, real world language definitions come in three main flavors:

- An official document, with a mix of formal notation and informal descriptions (Very common)
- An official document, with 100% of the definition specified in a formal notation (Very rare)
- A "reference implementation," namely a compiler or interpreter, so that the language "definition" is simply "whatever this program does" (Typical for some scripting languages). An advantage if this approach is that there are never any compiler bugs!

> ***Exercise**: Why exactly is a reference implementation compiler by definition bug-free? If being bug-free is so great, why don't all languages do this? What are the downsides?*

An official definition will have three parts:

Syntax (Structure)	**Semantics (Meaning)**	
	Statics	**Dynamics**
What are the structural entities (e.g., declarations, expressions, statements, modules) and how do they fit together, perhaps with punctuation?	What are the non-structural rules that define a legal program (e.g., type checks, argument-parameter matching rules, visibility rules, etc.)?	What does a program do? What effects do each of the forms of a well-structured, legal program have on the run-time environment?

Why are there three parts instead of two (i.e., just syntax and semantics)? Here's why. While everyone might agree that the following is *structurally* malformed:

```
#<include > stdio.h
main() int }
    printf["Hello, world!\n");]
{
```

the following program looks good in terms of "structure" but it's actually *meaningless* since it violates a **contextual rule** that says identifiers must be declared before use:

```
int main() {
    printf("%d\n", x);
}
```

We say the latter program has **static semantic errors** because they can be detected by a compiler before the program is ever run. This is in contrast to a dynamic semantic error, which can only be

detected at run time.

> **Example**: You've probably heard the distinction between "static" and "dynamic" before. Perhaps you know that "static typing" involves type checking is done *prior to* program execution and "dynamic typing" involves checking *during* program execution. Most languages do a little of both, but one or the other usually predominates. Sometimes you get a good deal of both: in TypeScript for example, you have a set of static types which is much larger and completely different than the eight dynamic types. Fun.

Defining the Syntax

Let's see how we would formally specify the syntax for the simple language Astro. Assume we've gone through the first two design phases, and we've sketched out a program that shows all the features:

```
// A simple program in Astro

radius = 55.2 * (-cos(2.8E-20) + 89) % 21;    // assignment statement
the_area = π * radius ** 2;                   // a built-in identifier
print hypot(2.28, 3 - radius) / the_area;     // print statement
```

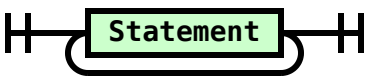
Next, we put our ideas into words. A first pass: "Programs are structured as a sequence of one or more statements, each of which is an assignment or print statement, with expressions formed with numbers, variables, function calls, and the usual arithmetic operators, which can have parentheses when needed. Comments look like the slash-slash comments of C-like languages."

Natural language isn't super precise, so as we sometimes do in design, we'll next turn thoughts into pictures.

Syntax Diagrams

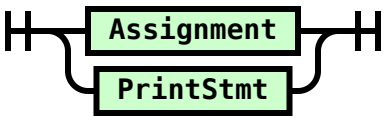
A program is a sequence of one or more statements:

Program



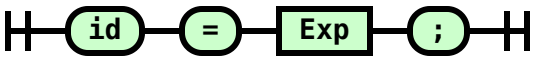
There will be two kinds of statements: assignments and print statements:

Statement

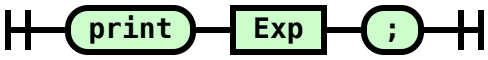


Assignments and print statements look like this:

Assignment



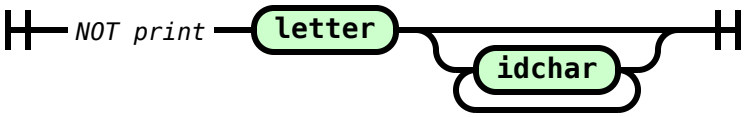
PrintStmt



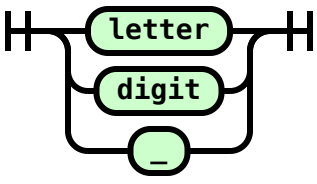
An **identifier** is the computer science word for a *name* you attach to an **entity** (a variable, constant, function, type, parameter, or similar thing). A **keyword** is a special word in your language, like `let` or `if` or `while` —these words generally do not name things—they are just used to structure the code. If a keyword is not allowed to be an identifier, we call it a **reserved word**. Astro has one reserved word: `print`.

Identifiers in Astro begin with a letter, and can have letters, digits, and underscores (examples: `x`, `last_attempt`, `p1`, `p2`, `overTheLimit`, `bot8675309_jEnNy`). We will call letters, digits, and underscores **identifier characters** (`idchar`s). But notice that `print` is not allowed to be an identifier:

id



idchar



How is the reserved word `print` specified? We have to be careful. It's not just the five letters p, r, i, n, and t. If it were then the program:

```
printy;
```

would be legal! It would be the five characters spelling print followed by a legal expression, namely the identifer y. We want the word `print` to not bleed into any following characters that might be part of an expression. In other words, `print` must not be immediately followed by an identifier character:

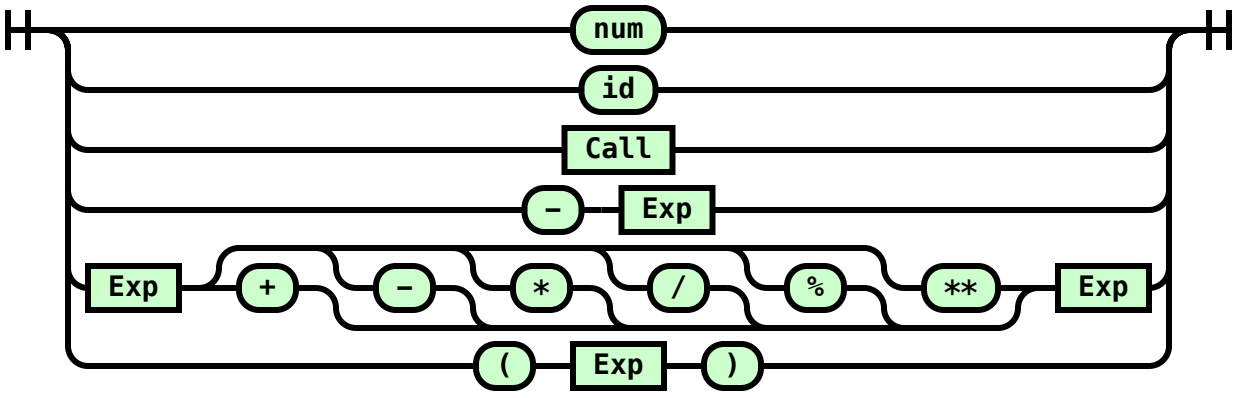
print



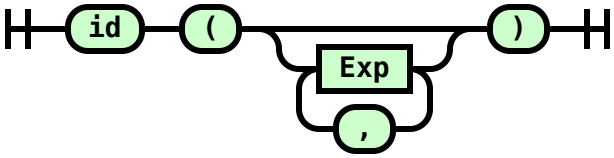
> ***Exercise**: Make sure you understand how this definition ensures that "printy" is an identifier and not* `print` *followed by the identifier* `y`.

A **statement** performs an action (perhaps with a side effect), but **expressions** represent the computation of a value. Expressions can be numbers, identifiers (representing variables), function calls, or can be made up of operators applied to other expressions. We can put parentheses around any expression. This will do the trick (but spoiler alert: we will have to change it later):

Exp



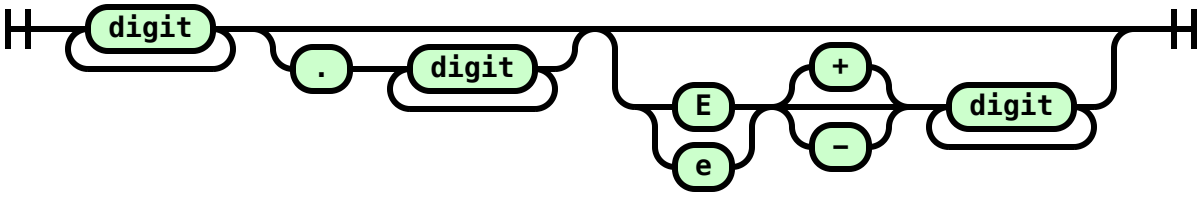
Call



***Exercise**: Explain the difference between expressions and statements to a friend. Make yourself a sacred vow never to mix the terms up. Precise language is a good thing. Sometimes we slip up, but still, let's try not to say things like "plus statement."*

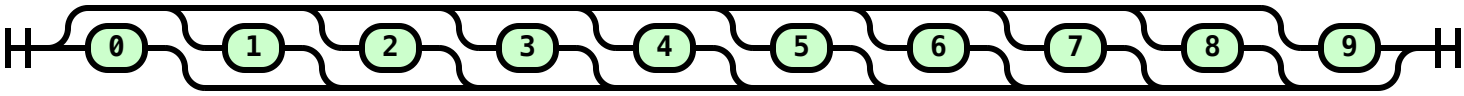
We mentioned numerals here, so let's define these. We'll keep things in decimal only (no worries about hex or binary), and use the times-ten-to-the notation from popular programming languages:

num



What about letters and digits? Well digits are pretty easy:

digit

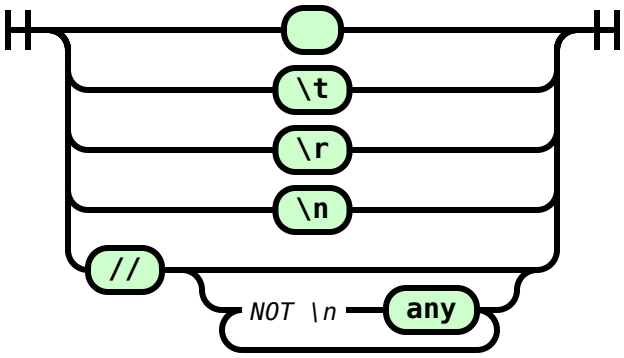


For letters, well, there are tens of thousands of these in Unicode, so we'll just say the rule for `letter` is "built-in."

Lexical vs. Phrase Syntax

Did you notice that we used two kinds of nodes in our syntax diagrams? We used ovals for constructs that internally cannot have spaces in them; we call these **lexical nodes**, or **tokens**. We used rectangles for constructs whose components can have spaces between them; we call these **phrase nodes**, or **phrases**. The special category called `space` defines what can separate tokens within phrases.

space



In the little language Astro that we are designing, the separators are the space character, the tab character, the carriage return character, the linefeed character (the linefeed (U+000A) is pretty much universally accepted as the “newline”), and comments. Comments function as spaces, too! For comments, we chose to start with `//` and go to the end of the current line.

We also followed an accepted convention to start the names of lexical nodes with lowercase letters and capitalize the names of phrase nodes.

Another convention we will follow for programming language syntax is to take the following as predefined:

- `letter`, for any Unicode letter
- `digit`, for `"0".."9"`
- `alnum`, for `letter | digit`
- `upper`, for any Unicode uppercase letter
- `lower`, for any Unicode lowercase letter
- `hexDigit`, for `digit | "a".."f" | "A".."F"`
- `any`, for any Unicode character at all

and also allow these conveniences:

- `\u{` and `}`, delimit a code point in hex and stands for the unicode character with that code point
- `\n`, for the LF character (same as `\u{a}`)
- `\r`, for the CR character (same as `\u{d}`)
- `\t`, for the tab character (same as `\u{9}`)
- `\"`, for the quote character (same as `\u{22}`)
- `\'`, for the apostrophe character (same as `\u{27}`)
- `\\`, for the backslash character itself (same as `\u{5c}`)

Note that `any` can be defined as `\u{0}..\u{10ffff}`.

> ***Remember***
>
> *The term* `space`*, though it looks like a lexical node, is something meta; it is treated specially.*

Why do we separate the lexical and phrase syntax? The lexical syntax, with the exception of the special rule for `space`, show us how to combine characters into words and punctuation into tokens (words) and the phrase syntax show us how to combine words and symbols into phrases (sentences), where the words and symbols can, if you wish, be surrounded by spacing.

> ***Exercise**: Meditate on how profound this is. Have you seen anything like this before? Hint: Are you reading this page in English?*

As a specific example, this program:

```
  print( // 麻🌿
420 );
```

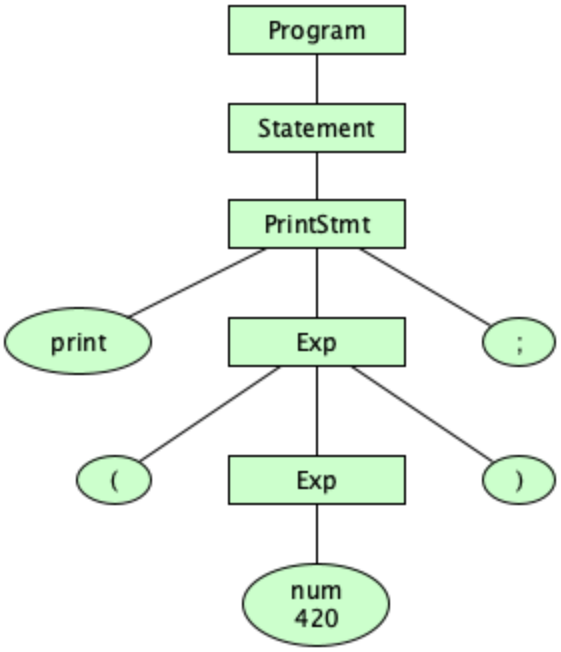
is made up of these characters:

SPACE *SPACE* *LATIN SMALL LETTER P* *LATIN SMALL LETTER R* *LATIN SMALL LETTER I* *LATIN SMALL LETTER N* *LATIN SMALL LETTER T* *LEFT PARENTHESIS* *TAB* *SOLIDUS* *SOLIDUS* *SPACE* *KANGXI RADICAL HEMP* *HERB* *LINEFEED* *DIGIT FOUR* *DIGIT TWO* *DIGIT ZERO* *SPACE* *RIGHT PARENTHESIS* *SEMICOLON*

Following the lexical syntax, skipping the spaces (and comments), we get the **token stream**:

`print` `(` `num(420)` `)` `;`

Following the phrase syntax, we can uncover the underlying **parse tree**:



A very important thing to note: **The frontier of the parse tree is the token stream**.

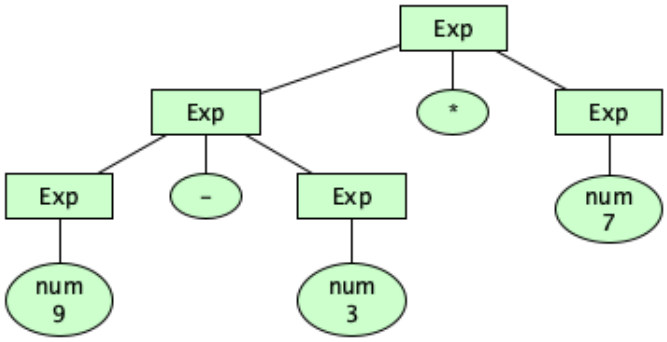
> ***Exercise**: Repeat this phrase to yourself five times: The frontier of the parse tree is the token stream.*

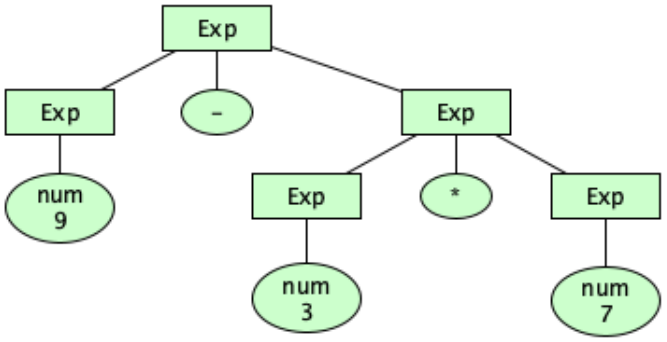
> ***The parse tree ends at tokens, not characters***
>
> *Please take the time to consider how silly it would be if the parse tree expanded all of the lexical rules down to characters. Having the parse tree stop at tokens is a good example of what we would call "breaking a complex problem down into simpler parts."*

Another term for "parse tree" is **concrete syntax tree** (CST).

Ambiguity

How are we doing so far? We are able to distinguish well-structured Astro programs from all other Unicode strings. But there are some things we haven't dealt with yet. For one, we have some strings with multiple structural forms. For example, the phrase `9-3*7` can be **parsed** in two ways:





Having more than one parse tree for a given input string means that our syntax description is **ambiguous**. It's possible to handle this particular kind of ambiguity in the syntax. Here's how.

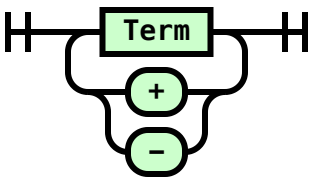
Precedence

We can create rules that force certain operators to be applied before others; that is, the **precedence** of operators can be enforced in our syntax definition. To do so, we define additional syntactic categories. We say:

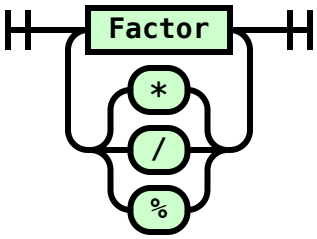
- An expression is a sequence of one or more **terms**, separated by additive operators.
- A term is a sequence of one or more **factors**, separated by multiplicative operators.
- A factor is made up of **primaries**, separated by exponentiation operators, or a factor can just be a negated primary.
- A primary is the most basic expression possible: either a simple identfier, a numeral, a call, or a parenthesized expression. Parentheses allow unlimited nesting.

We can write:

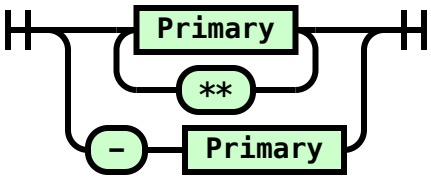
`Exp`



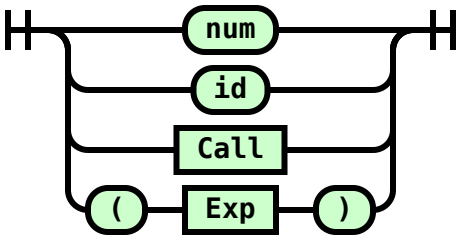
`Term`



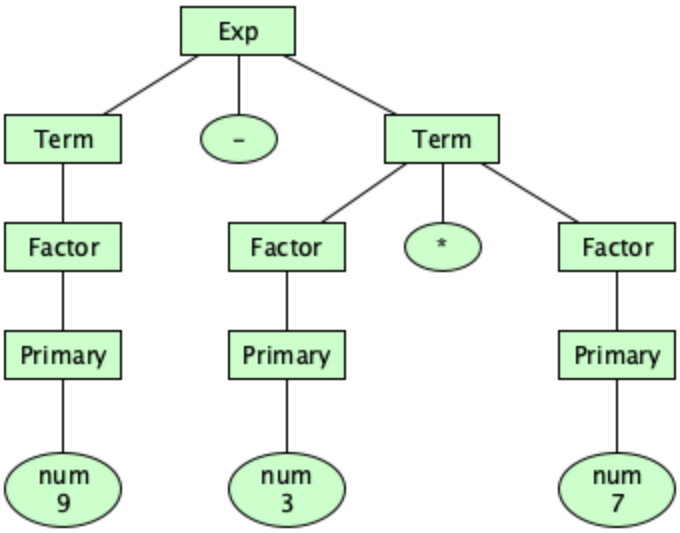
`Factor`



Primary



Great! Now there is one and only one parse tree for that previously problematic expression:



Note that the new syntax has forced the binary operators into a precedence hierarchy!

- `+` and `-` have the lowest precedence.
- `*`, `/`, and `/` have the next higher precedence.
- `**` has the highest precedence.

Of course, you can think of parenthesized expressions as being done before anything else, though we don't usually think of these as operators.

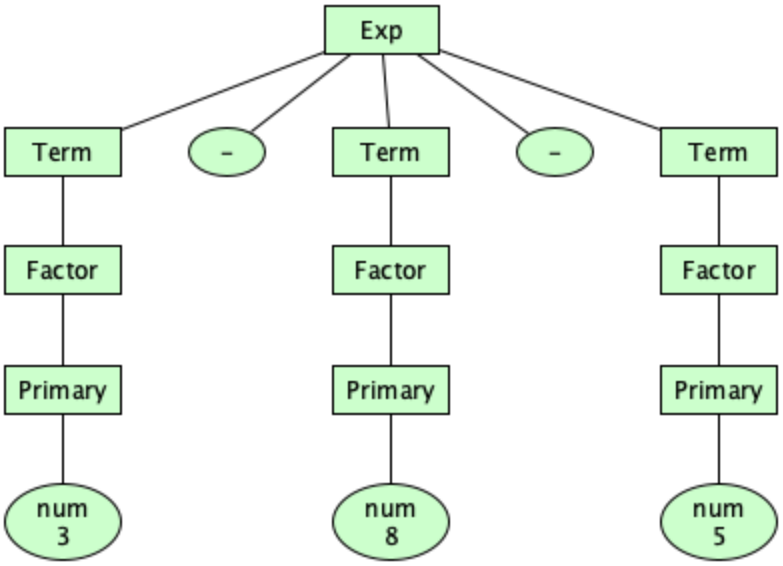
> ***Exercise**: Precedence is usually a concept that applies only to binary operators, but they way in which unary operators mix with binary operators does need to be addressed. For instance, how should* `-3**2` *be parsed? In Python exponentiation precedes negation, like* `-(3**2)`*. In Elm, negation precedes*

exponentiation, like `(-3)**2`. *Astro follows JavaScript and simply does not allow this expression (forcing programmers to use parentheses in this case)! Show how this is done.*

***Exercise**: Our syntax diagram above does not allow* `-3**2` *as an expression, but it does allow* `-3+2` *and* `-3*2`. *Why did we care only enough to ensure negation did not mix with exponentiation, but we were fine with it mixing with addition and multiplication?*

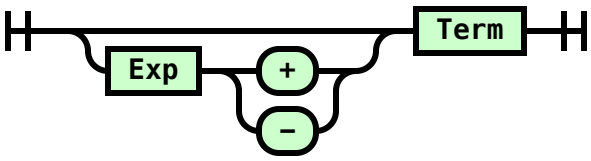
Associativity

Wait, we are not done with structuring our operators just yet. The way things stand now, the parse tree for `3-8-5` looks pretty flat:

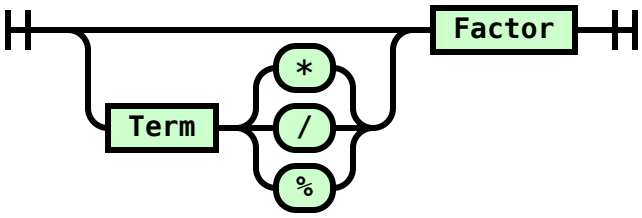


It doesn't suggest whether we mean to compute `(3-8)-5` (which would be -10) or `3-(8-5)` (which would be 0). We can give a syntax that makes this clear. In our design, let's make the additive and multiplicative operators **left-associative** and the exponentiation operator **right-associative**:

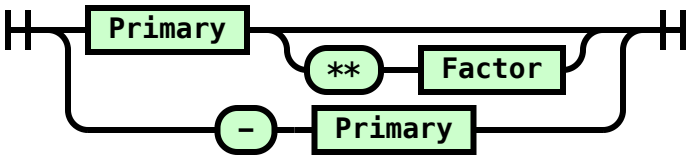
`Exp`



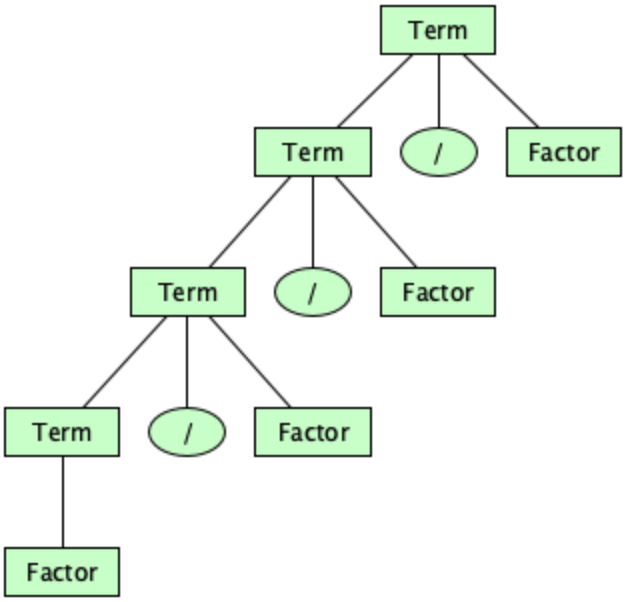
`Term`

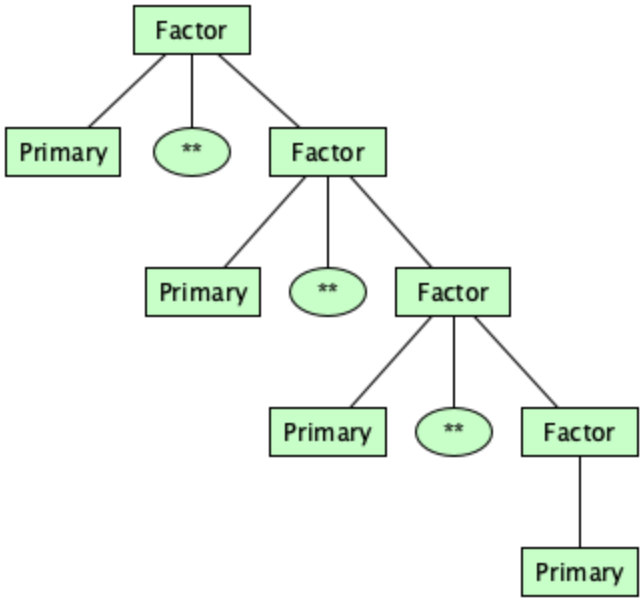


Factor



How the heck does this work? Study these parse trees, and hopefully the insight will come to you! (Hint: remember the syntax is *designed* to force the tree to "come out" a certain way.





> ***Exercise**: Study this until you understand it well.*

Grammars

Syntax Diagrams are fine for giving an overview of program structure, but, like all pictorial notations, they can take up a lot of space and are rather hard to process by a machine. If we could write a denser, text-based description of a syntax, we could write a program that checked a candidate program for conformance with the syntax! This is exactly what compilers, interpreters, and related tools do.

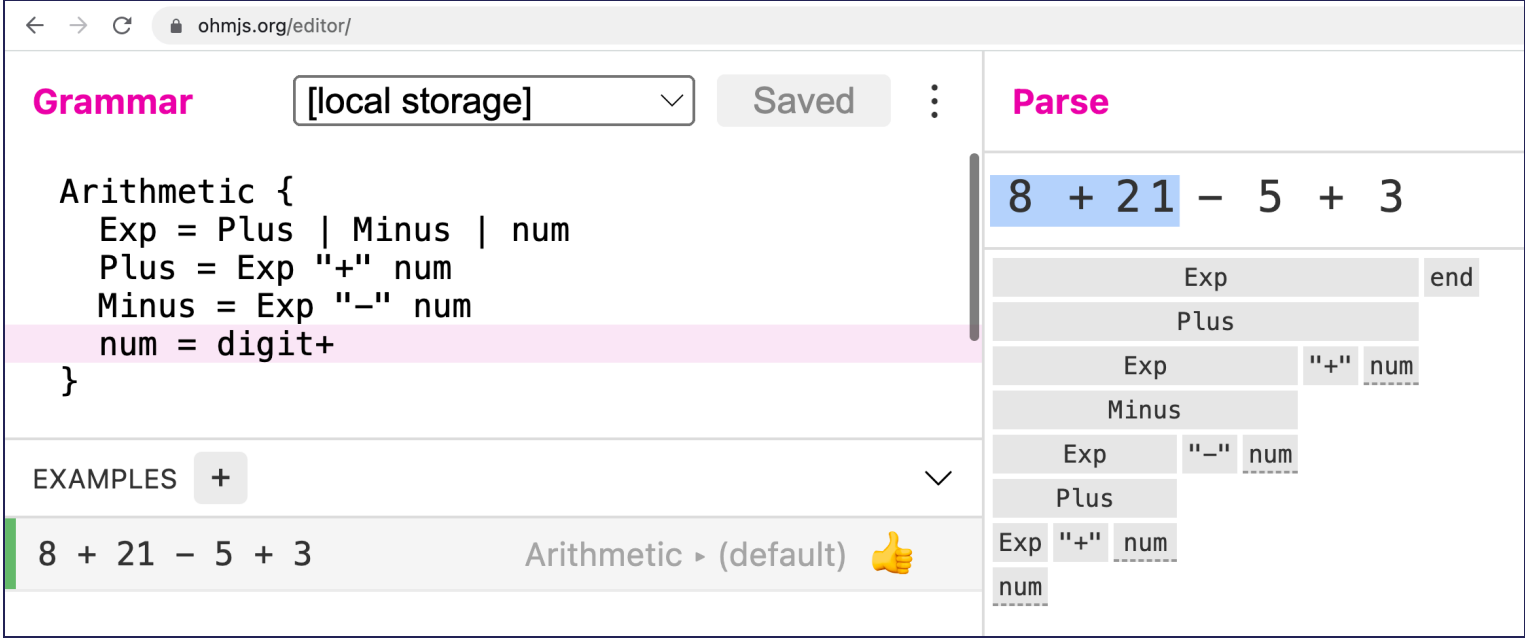
Let's translate the syntax diagrams above into a grammar for Astro. There are many different kinds of grammars that have been created for describing programming languages. We'll use an Ohm grammar for our syntax definition of Astro. We'll cover Ohm in detail later in the course, for now, we just present the grammar. Take time in your self-study to see how it matches up exactly with the syntax diagrams we've shown above.

astro.ohm

```
Astro {
  Program     = Statement+
  Statement   = id "=" Exp ";"                          --assignment
              | print Exp ";"                           --print
  Exp         = Exp ("+" | "-") Term                    --binary
              | Term
  Term        = Term ("*" | "/" | "%") Factor           --binary
              | Factor
  Factor      = Primary "**" Factor                     --binary
              | "-" Primary                             --negation
              | Primary
  Primary     = id "(" ListOf<Exp, ","> ")"             --call
              | numeral                                 --num
              | id                                      --id
              | "(" Exp ")"                             --parens

  numeral     = digit+ ("." digit+)? (("E" | "e") ("+" | "-")? digit+)?
  print       = "print" ~idchar
  idchar      = letter | digit | "_"
  id          = ~print letter idchar*
  space      += "//" (~"\n" any)*                       --comment
}
```

Keep in mind, though, that you should not just write a bunch of test programs, then write your whole grammar in one short. As design is iterative, you should *build up* the grammar from increasingly more complex examples, and test the grammar as you go. Tools will help you here! If you are using Ohm, and you should, take advantage of the use the Ohm Editor. This is an amazing tool for experimenting with programming languages.



In the upper left panel, design your grammar. You can load/save from your browser's local storage, and even publish gists to GitHub. In the lower left panel, enter test cases: both tests you want to succeed (thumbs up) and those you want to fail (thumbs down). The right panel is an interactive concrete syntax tree for the currently selected test case.

This tool will save you a lot of time.

It is an essential component of your language design toolbox.

> ***How essential is it?***
>
> *Unless your language is trivial, tools like the Ohm Editor are very important! Design is an iterative process, and creativity is enabled and enhanced with immediate feedback. So you should design with tools that allow you to experiment and test your ideas.*
>
> *That said, it is true that in practice, many production-level compilers do not use Ohm or related tools like ANTLR, Bison, etc.—they do everything by hand. But developers that go this route will write their grammar tests concurrently with their design.*

CLASSWORK

Let's do a code-along with the Ohm Editor for developing the Astro grammar. During the code-along, note how examples are done first, and note how we will evolve from the basics to more complex features, bringing in notions such as a precedence and associativity where needed.

During the code-along, bits of Ohm will be introduced as needed. In a subsequence course unit, we will cover Ohm in detail.

Defining the Statics

Question: Does the grammar we defined above for Astro capture the following (desirable) rule:

> "You can only use an identifier if it has been previously assigned to."

Answer: It does not.

***Exercise**: Show that it does not. Hint: Is* `print x;` *a legal program according to the grammar?*

***Exercise**: Show how to capture the rule in a grammar. Hint: Do not be frustrated if you cannot.*

Enforcing this rule requires knowledge of **context**. That is, the syntactic rule for producing expressions such as calls and arithmetic operations would need to somehow know which identifiers appeared on the left hand side of some previous assignment statement. This turns out to be so hard that even designers of real programming languages omit enforcement of contextual rules from the grammar! In fact, while the official grammar for Java will not derive this program:

```
class A {2 / == {{;
```

and report a **syntax error**, a Java compiler will say that this program:

```
class A {int x = y;}
```

is structurally well formed according to the official syntax of the Java language! The compilation unit consists of a type declaration that is a class declaration whose body consists of a field declaration with a type, a name, and an initializing expression which is an identifier. But we know this program is not legal, since the identifier `y` has not been declared. It's not only Java that has grammars overspecifying things: pretty much every programming language uses a grammar to define structural rules only, and specifies contextual rules in prose, or in a separate semantic definition.

For Astro, we will "define" the following contextual rules:

1. The following identifiers are built-in:

    - `π`, a number
    - `sqrt`, a function of exactly one argument
    - `sin`, a function of exactly one argument
    - `cos`, a function of exactly one argument
    - `hypot`, a function of exactly two arguments
2. An identifier cannot be used in an expression unless it is one of the built-in identifiers or has been previously assigned to.
3. All function calls must accept the proper number of arguments.
4. The built-in identifiers cannot be assigned to.
5. Identifiers declared as functions can only be called, not used in any other context. Identifiers declared as numbers cannot be called.

For more complex languages, the statics definition (contextual rules) can be quite large. Here are some things that might appear:

- Identifiers have to be declared before they are used.
- Identifiers may not be used in a place where they may have not yet been initialized.
- Identifier declarations must be unique within a scope, unless a language provides overloading.
- All expressions must be used according to their type.
- Identifiers must be used according to (1) their **scope**, (2) their **access modifiers** (private, protected, package, public, whitelist, blacklist, ...), and (3) any other meta-level attributes (enumerable, configurable, writable, deletable, callable, ...).
- Arguments must match up with parameters in terms of number, order, name, mode, and type.
- `break` and `continue` statements may only appear in a loop. `return` statements may only appear in a function it's *possible* to encode these restrictions in the grammar, but it would be ugly).
- All paths through a function must have a return.
- All possible matches in a pattern match expression, or a switch statement, must be covered.
- When inheriting from a base class with abstract methods, or implementing an interface, all abstract methods must be implemented, or the derived class must be declared abstract.
- All declared local variables must be subsequently read, and declared private functions must be called.

Defining the Dynamics

The dynamics for most programming languages are given in prose. If you language is simple enough, a formal semantic definition is possible. For the sample languages in this course, Astro and Bella are

given both informal and formal semantic definitions. We'll not be studying formal semantics at this time, but feel free to study the definitions on your own.

Prototyping

During your language design, you will want to whip up a simple interpreter to at the very least make sure your design is reasonable. You may wish to developer your interpreter in parallel with your language design.

Ohm was designed for prototyping programming languages, so it is a natural choice. The Ohm Editor, as we just saw, helps you design the syntax. To write an actual interpreter, we'll have to go much deeper into Ohm. We'll be doing this in our next unit of study in this course, which is, indeed, a deep study of Ohm.

Examples

Many well-known programming languages have published, formal definitions. You can find them by searching the web.

For this class, we will be studying five little languages crafted especially to help you in your stufy of language design and implementation. We will be studying them in order, building upon previous languages and learning new things as we progress. This will allow us to introduce the huge topic of language processing in a practical setting, writing real compilers for real languages. The languages are Astro, Bella, Carlos, Dax, and Ekko.

Astro



We all begin as a white belt in every new endeavor. We will start, then, with a very simple, almost trivial, language. All it has are numbers, arithmetic operators, variables, and a few pre-defined constants and functions. Here's an example program:

```
// A simple program in Astro
rAd1uS = 55.2 * (-cos(2.8E-20) + 89) % 21;
the_area = π * rAd1uS ** 2;
print(hypot(2.28, 3 - rAd1uS) / the_area);    // woohoo 👻
```

There are only two kinds of statements: assignments and print statements. Expressions include numbers, variables, function calls, arithmetic expressions with `+`, `-`, `*`, `/`, `%`, and `**`, and can be parenthesized. We will cover the official definition of the language, and use the language to motivate a formal study of syntax.

When studying this language, we'll learn about the separation of context-free syntax from contextual rules. Contextual rules include such things as: having to match the number of arguments in a call with the number of defined parameters, rudimentary type checking, and not allowing assignments to read-only variables.

As Astro will be our first language, we will use it as a case study to learn the amazing Ohm Language Library to build an interpreter. The details of how the interpreter is constructed are covered in the course notes on Ohm.

Bella

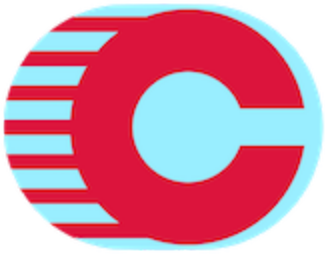


Our second language has a few things Astro does not: a richer set of operators, variable declarations, and user-defined functions. The contextual rules for Bella are much richer than that of Astro, since we now have actual declarations, and scope! Here's an example program:

```
let dozen = 12;
print dozen % 3 ** 1;
function gcd(x, y) = y == 0 ? x : gcd(y, x % y);
while dozen >= 3 || (gcd(1, 10) != 5) {
  dozen = dozen - 2.75E+19 ** 1 ** 3;
}
```

We will first look at the official specification, introducing all sorts of interesting concepts. Then we'll study a real, actual Bella compiler. Here we learn about designing and architecting a compiler, building the components (analyzer, optimizer, and generator), and getting 100% test coverage. The compiler source code is on GitHub.

Carlos



In our third language, we encounter arrays, structs, and optionals: our first language that is basically useful. If you are taking the compiler course for which these notes were written, Carlos is a good example of the minimal language complexity you will need for your term project.

```
const languageName = "Carlos";

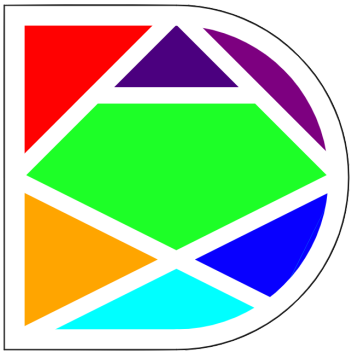
function greeting() {
  return random(["Welcome", "こんにちは", "Bienvenido"]);
}

print("👋👋👋");
repeat 5 {
  print(greeting() + " " + languageName);
}
```

We'll be visiting the language's official specification and a compiler on GitHub. The compiler (of course!) uses the Ohm Language Library.

There's no separate page of notes describing the compiler. After studying the Astro and Bella compilers, you'll be able to find your way around the code on GitHub (there's documentation). And don't worry, it's development and usage will be covered in class, and the teaching staff can help you with any questions you might have.

Dax



Language number four is a functional language, that is, a language with no assignments! The only bindings of names to entities happens when passing arguments to parameters, though there is that famous `let` declaration which nicely sugars a function call: it's nicer to say `let x = 5 in x * y end` than `{x => x * y}(5)`.

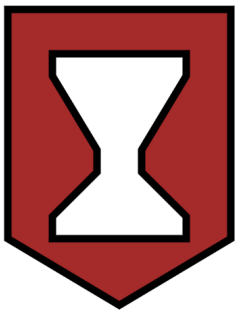
Here's a sample program to get the feel for the language:

```
let
  gcd = {x => {y ==> y == 0 ? x : gcd y (x % y)}};
  z = 5
in
  [1, 3, z] |> filter {x => x > 2} |> map {x => x ** 2} |> print
  then
  "hello" |> substring 2 5 |> print
  then
  print (gcd 33 99)   // This is fine, you don't HAVE to use |>
end
```

If you have not yet seen languages with the awesome `|>` operator, here's your chance to be wowed.

We will discuss the language design and compiler later in the course.

Ekko



Our fifth language, Ekko (starting with E like Erlang and Elixir, which greatly influence it), is a kind of experimental language that deals quite a lot with **time**.

Ekko mixes styles of asynchronous programming from JavaScript and the distributed process-orientation of Erlang and Elixir: Ekko's `future` objects are based on JS promises, and its processes communicate via messages as in Erlang. There's also quite a bit more temporal goodness, including timeout calls, value histories with time travel (influenced by older versions of Elm), and even explicit parallelism.

More Examples

Students in previous iterations of the course have designed and implemented their own languages. Here's a sampling of language over the past decade or so. (Please note that there is a very wide variety of quality in these examples. They are presented here without any evaluative commentary as to whether they are suitable building blocks for one's own project.)

- Blvd
- Sous
- River
- Rat
- Cod
- GLHF
- BuildLang
- Banana

- Pyth-on-Point
- Dino
- Sanscript
- Kis
- YALL*E
- Jazz
- Barista
- LangY
- MODE
- Bang
- Squishi
- SussyScript
- CoffeeMaker
- YeeHaw
- Gullienne
- ElectricScript
- Barista
- Shrek--
- Chinese Baozi
- IcedCoffee
- Chloe
- T.O.A.L.
- Pirates Code
- Leg
- Kobe
- Cringe--
- Bismath
- Plumb
- PokerScript
- Pandemonium
- Easi
- Palet
- VikingScript
- Speare
- Ahtohallan
- VineScript
- Big Mama's Kitchen
- Medley
- Custom
- LemonScript
- Aegis
- Midi-Cholorian
- qcq
- HYPER!
- RealHotGirlScript
- Pivot
- SnekQL
- Respec++
- r0b0p
- uwuScript
- PawvaScript
- Cuttlefish
- Inkling
- Casper
- StoneScript
- FUZZ++
- Goof3
- Scriptofino
- 101Script
- Jen
- Olive
- Nebula
- J4
- Alula
- Pycante
- Whiteboard
- Favascript
- Queen
- MemeScript
- Skrt
- JCaml
- Boozie
- Deeg
- AvaJava
- Yah
- Roo
- Agate
- AboveAverageScript
- TeaScript
- YACBL
- Yoda
- Spitfire
- Derpodile
- GollumScript
- Singularity
- KobraScript
- Min

Recall Practice

Here are some questions useful for your spaced repetition learning. Many of the answers are not found on this page. Some will have popped up in lecture. Others will require you to do your own research.

What are the four phases of programming language design? HIDE ANSWER

(1) Working out the desired context (audience, purpose, scope), (2) sketching example programs, (3) formalizing the syntax and semantics, and (4) prototyping.

Name five programming language paradigms. HIDE ANSWER

A few are: imperative, declarative, structured, object-oriented, functional, applicative, concatenative, logic, protocol-oriented, aspect-oriented, array, event-driven, dataflow, agent-based. There are others

What were the four big ideas in programming languages mentioned in Bret Victor's *Future of Programming* talk? HIDE ANSWER

(1) Direct manipulation of data rather than showing the code, (2) Goals and constraints rather than procedures, (3) Spatial representation of code rather than textual representations, and (4) Concurrent computation rather than sequential

What language did Alan Kay feature in his Programming Languages video? HIDE ANSWER

eToys

What are some things to decide upon while undertaking language design? HIDE ANSWER

The audience, the purpose, the scope, the problem domain it is target to, whether it is reasonable or esoteric, whether it has a simple foundation or favors pragmatism, its concurrency model.

In what essay did Bret Victor lay out several principles that languages and programming enviroments should follow to aid their users' learning? HIDE ANSWER

Learnable Programming

What is abstract syntax? HIDE ANSWER

The structure of a program without regard to the specific syntax used to represent it.

What is the name of the specification of JavaScript's abstract syntax? HIDE ANSWER

ESTree

At what web address can you find and interactive JavaScript AST builder? HIDE ANSWER

https://rtoal.github.io/js-ast/

What are five approaches of showing (concrete) syntactic structure in popular programming languages? HIDE ANSWER

Curly braces, indentation, terminal-ends, parentheses, postfix

What is an example of a programming language for which newlines are significant? HIDE ANSWER

Answers include Python and Ruby

What is syntactic sugar? HIDE ANSWER

Syntax within a programming language that is designed to make certain phrases more clear, concise, or elegant than the basic forms defined in the language.

What is syntactic salt? HIDE ANSWER

Syntax within a programming language that is designed to make certain phrases more confusing, verbose, or awkward than the basic forms defined in the language.

What is a candygrammar? HIDE ANSWER

A syntax that looks sweet and natural language-like that appears good but turns out to be bad for you.

What is a golfing language? HIDE ANSWER

A language designed to be as terse as possible, often for the purpose of helping you win shortest-possible code challenges.

What are some examples of golfing languages? HIDE ANSWER

CJam, Pyth, Stax, GolfScript, 05Ab1E

What are three styles of programming language definition? HIDE ANSWER

Informal, formal, and executable

What is the difference between syntax and semantics? HIDE ANSWER

Syntax deals with program structure. Semantics deals with program meaning.

What is the difference between a language's **statics** and its **dynamics**? HIDE ANSWER

Statics refers to the rules that can be checked at compile time. Dynamics refers to the rules that can only be checked at run-time.

What are syntax definitions generally split into lexical and phrase syntaxes? HIDE ANSWER

Lexical syntax defines how invidividual characters are grouped in to words (tokens), while phrase syntax defines how these tokens are combined to form larger structures.

What is a parse tree? HIDE ANSWER

A tree that describes the syntactic structure of a program as defined by the language's syntax.

The frontier of the parse tree is the program's _____. HIDE ANSWER

Token stream

A parse tree is also called a _____. HIDE ANSWER

Concrete syntax tree

What does it mean for a syntactic specificiation to be ambiguous? HIDE ANSWER

There exists at least on program that has more than one parse tree.

What attributes of operators are generally used to disambiguate a syntactic specification? HIDE ANSWER

Precedence and associativity

Because syntax diagrams may take up a lot of space, what alternaive method of syntactic specification is typically used? HIDE ANSWER

Grammars

What tool is provided by the authors of the Ohm library to help you prototype a new language design? HIDE ANSWER

The Ohm Editor

What was the title of Alex Warth's dissertation? HIDE ANSWER

Experimenting with Programming Languages

What are some examples of legality rules that are considered too hard to be captured in a grammar and are thus typically defined within a program's static semantics? HIDE ANSWER

You can only use an identifier if it has been previously assigned to, all function calls must accept the proper number of arguments, the built-in identifiers cannot be assigned to, identifiers declared as functions can only be called, not used in any other context, identifiers declared as numbers cannot be called, type checking, access checking (private, public), declared identifiers must be initialized or must be read. (There are many more.)

What example programming languages have been designed for this course? HIDE ANSWER

Astro, Bella, Carlos, Dax, Ekko

Summary

We've covered:

- ✅ The cycle of language design phases
- ✅ What to know before undertaking language design
- ✅ Pointers to excellent articles and essays about language design
- ✅ Two videos (one by Alan Kay, one by Bret Victor) on languages and language design
- ✅ How to begin the language design process
- ✅ Questions to ask while designing your language
- ✅ Concepts and features to think about when sketching during design
- ✅ Abstract Syntax
- ✅ Concrete Syntax
- ✅ The use of the Ohm Editor in language prototyping
- ✅ (Formal) Language Definition
- ✅ Brief notes on five example languages

Theories in Computer Science

To develop expertise in any endeavor, you need both practice and theory. Computer Science is a discipline with many practices and many theories. Let's visit a few of the major theories—the science in Computer Science. Why? Don't worry, we'll explain the usefulness of studying theory, too.

CONTENTS

What is a Theory? • Why Study Theory? • Theories of Computation • Language Theory • Automata Theory • Computability Theory • Complexity Theory • Working With These Theories • Theories and Creativity, Again • Recall Practice • Summary

What is a Theory?

We'll begin with this definition from an actual respected dictionary:

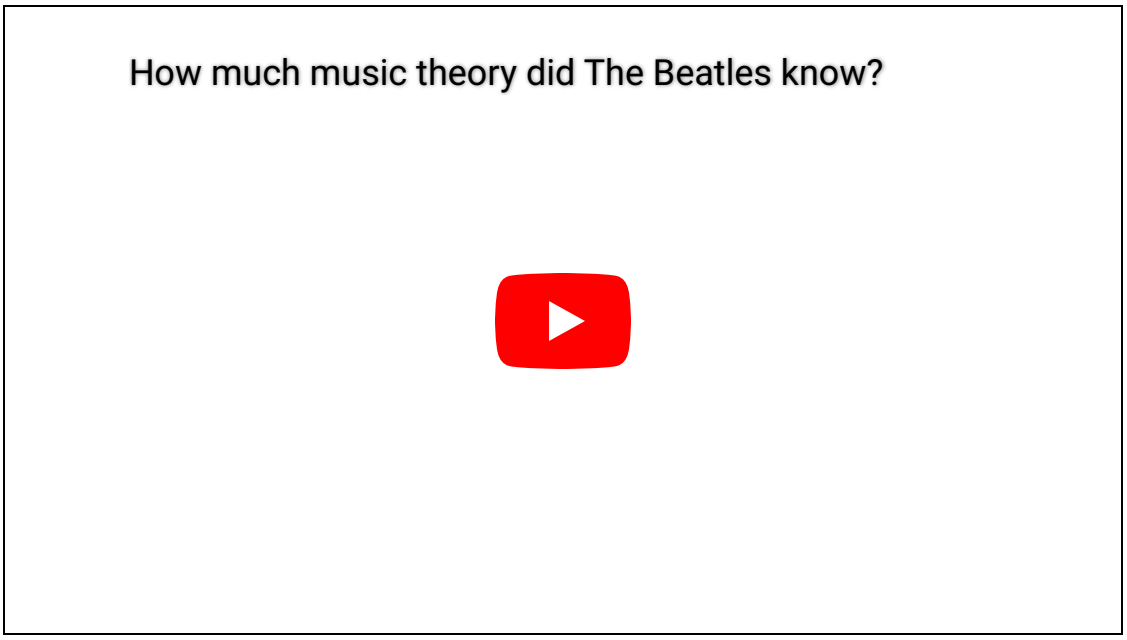
> ***theory (n.)***
>
> 1. *A set of statements or principles devised to explain a group of facts or phenomena, especially one that has been repeatedly tested or is widely accepted and can be used to make predictions about natural phenomena.*
>
> 2. *The branch of a science or art consisting of its explanatory statements, accepted principles, and methods of analysis.*

Sure, sometimes folks use the word "theory" for a hunch or a supposition, but in the arts and sciences, a theory is *an organized body of knowledge with explanatory and predictive powers*.

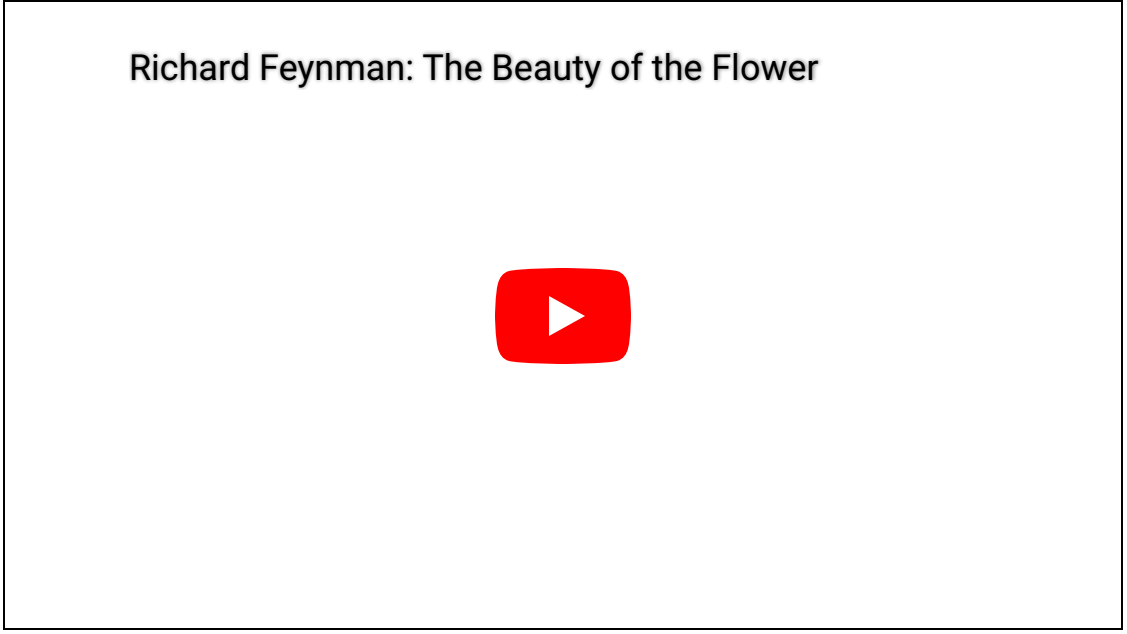
Why Study Theory?

We build a theory to **organize** our knowledge and develop a **vocabulary** that we can use to reason about things, communicate with others, make predictions, and generate new knowledge.

Without a theory, we can still do things, but not as well: there's a lot of improvisation, difficulty in communicating ideas, guess work, and flailing around. Your practice will not be as strong, and your insights may be limited. Let's see how David Bennett expertly makes this point. The key ideas are expressed from 17:21-18:42 (though the whole video is interesting):



Music theory can be helpful in creating music, but can it help us enjoy music more? Certainly! After all, theoretical knowledge *adds* to our awe of that which we study, allowing us to appreciate the beauty of something at more than just one level:



Theories of Computation

The field we now call **Computer Science** has a long and fascinating history. Several theories emerged as the discipline took shape. A **computer scientist** has a working knowledge of these theories and employs them to not only create useful applications and systems, but to advance the state of the art. There are four major theories under the umbrella term "The Theory of Computation." Two are rather fundamental:

Language Theory

How are computations **expressed**?

Automata Theory

How are computations **carried out**?

The third and fourth get into certain specifics but have deep and fascinating philosophical roots:

Computability Theory

Complexity Theory

What are the fundamental **limits** of computation?

What **resources** are required to perform certain computations?

To study and apply these theories, we need some context.

Computation augments and amplifies human thought. Modern computers speed up time to such an extent that our reasoning powers are at a qualitatively different level than what we had before these machines. These computational theories help us qualify and quantify how exactly this augmentation occurs. They enable us to ask and answer questions such as

- What even *is* computation?
- What can and cannot be known through computation?
- What can and cannot even *be* computed?
- What can and cannot be computed under certain time and space constraints?"

In order to be accepted and useful, these theories must be grounded with practical axioms that match our intuition of what computations actually are. We will take as given that a computer is a finite device and a program is a finite set of instructions.

***Exercise**: Why are the assumptions of finiteness not only reasonable but essential?* SHOW ANSWER

Language Theory

Language Theory is concerned with how information and computation are expressed. It begins with the question "What is information?" and approaches the question with the hypothesis that any piece of (human-usable) **information** can be represented as a string of **symbols**. To see why this hypothesis is useful, note these examples:

- `91772.3e-8`
- `Stay behind the yellow line!`
- ᐊᐃᖏᓚᐃ ᓄᓇᕐᔪᐊᖅ
- `/Pecs/Los Angeles/Berlin/Madrid//1 2/3 0/2 2/2 0/3 2///`

- `(* (+ 4 3) (-9 7))`
- `int average(int x, int y) {return (x + y) / 2;}`
- `<dog id="30024"><name>Lisichka</name><age>13</age></dog>`
- `{"type": "dog", "id": "30024", "name": "Lisichka", "age": 13}`
- `∀α β. α⊥β ⇔ (α•β = 0)`
- `1. f3 e5 2. g4 ♕h4++`

Symbols are chosen from some finite **alphabet**, such as $\{0, 1, 2, 3, 4, 5, 6, 7, 8, 9\}$ for the language of natural numbers, or $\{0, 1, 2, 3, 4, 5, 6, 7, 8, 9, \mathrm{e}, \mathrm{E}, +, -\}$ for the language of floating-point numbers. What is a language? A **language** (in Formal Language Theory) is a set of strings over an alphabet; that is, some strings are in-the-language and some are not. For example, in the language-of-floating-point-numbers, `3.55E+21` is in the language, but `+-9-E...2` is not. Beyond formal language theory, we consider a language to also contain an assignment of "meaning" to each string in the language.

Here's an obvious, but important, point worth stating: strings in a language represent not only numbers and text and complex data structures, but also abstractions of computations known as **functions**, which produce output information from input information. Examples:

- `double`, which takes a number and returns twice that number
- `plus`, which takes a pair of numbers and returns their sum
- `nextMove`, which takes a game state and returns the next (best?) move that should be made
- `upperCase`, which takes a piece of text and returns a string like it in every way *except* all letters are in upper case
- `sorted`, which takes a list of things and returns a new list containing the same values in sorted order
- `even`, which takes a number and returns true or false depending on whether the number is even
- `twice`, which takes a function and a value and applies the function to the result of applying the function to the value
- `javac`, which takes a string in the Java programming language and returns an executable class file
- `java`, which takes a class file and returns (the effect of) its execution
- `python`, which takes a string in the Python programming language and returns (the effect of) its execution

Functions are **applied** to **arguments** to produce their results. (The application is sometimes called a **invocation** or a **call**.) For example:

- `double(3.5)` $\Longrightarrow$ `7`
- `plus(4, -3)` $\Longrightarrow$ `1`
- `upperCase("Please don't shout!")` $\Longrightarrow$ `"PLEASE DON'T SHOUT!"`
- `sorted([4, 7, 1])` $\Longrightarrow$ `[1, 4, 7]`
- `even(797)` $\Longrightarrow$ `false`
- `twice(double, 5)` $\Longrightarrow$ `double(double(5))` $\Longrightarrow$ `double(10)` $\Longrightarrow$ `20`
- `python("print(8 * 5 - 2 / 1)")` $\Longrightarrow$ `{stdout: "38"}`

Functions can be expressed in so many ways. Alonzo Church invented the **Lambda Calculus** to show that function definition and application could serve as a foundation for mathematics. Here are example functions:

- `λx. not (equal (mod x 2) 0)`
- `λp. λn. λr. λt. times p (power (plus 1 (divide r n)) (times n t))`

Abbreviations are okay:

- $\lambda x\,.\,x \bmod 2 \neq 0$
- $\lambda(p, n, r, t)\,.\,p \times (1 + \frac{r}{n})^{nt}$

There are other notations, too:

- `(LAMBDA (x) (NOT (EQUAL (MOD x 2) 0)))` (Lisp)
- `lambda x: x % 2 != 0` (Python)
- `x => x % 2 != 0` (JavaScript)
- `(x) -> x % 2 != 0` (Java)
- `->(x) {x % 2 != 0}` (Ruby)
- `#(not= (mod % 2) 0)` (Clojure)
- `{$0 % 2 != 0}` (Swift)
- `|x| { x % 2 != 0 }` (Rust)

In Language Theory, we study how to **express** these functions, as well as the data upon which they operate, and the modules, objects, and control structures which house these functions. We study how **syntax definitions** can be written to define which strings belong to the language, and how **semantic definitions** can give meaning to these strings. We even study how these languages are used by human beings, something we call **pragmatics**. But how can we actually **compute** (or "carry out", or "evaluate") these functions? For that we turn to automata theory.

Automata Theory

In **Automata Theory**, we build and study formal models of computational devices (i.e., computers). We can start to develop a theory by thinking about how it is we expect our functions to be formally defined, and in some sense, how they "work".

Early Computing Machines

Of course, people have entertained the idea of building calculating machines for centuries. Notable examples include:

- Abacus
- Antikythera
- Astrolabe
- Pascal's adder
- Babbage's Analytical Engine
- Jacquard's Loom
- and more

These were all amazing feats of engineering and human ingenuity in their own right, but it wasn't until Turing that we finally had a solid understanding of what computation really is.

> *"Babbage never foresaw the terrible consequences of his invention—a machine that would autocorrect his name to 'cabbage' every single time." —Philomena Cunk*

Turing Machines

When Turing was in his early 20s, he set out to prove Hilbert wrong by solving the Entscheidungsproblem in the negative. To do so, he needed to define, formally, what was meant by computation, so he created a simple machine model, or **automaton**, to capture the essence of computation (or algorithm, or effective procedure). He called machines derived from this model **a-machines**, but we know them today as **Turing Machines**. Turing Machines are by no means the only kind of automaton that can carry out computations, but they are special, for a variety of reasons.



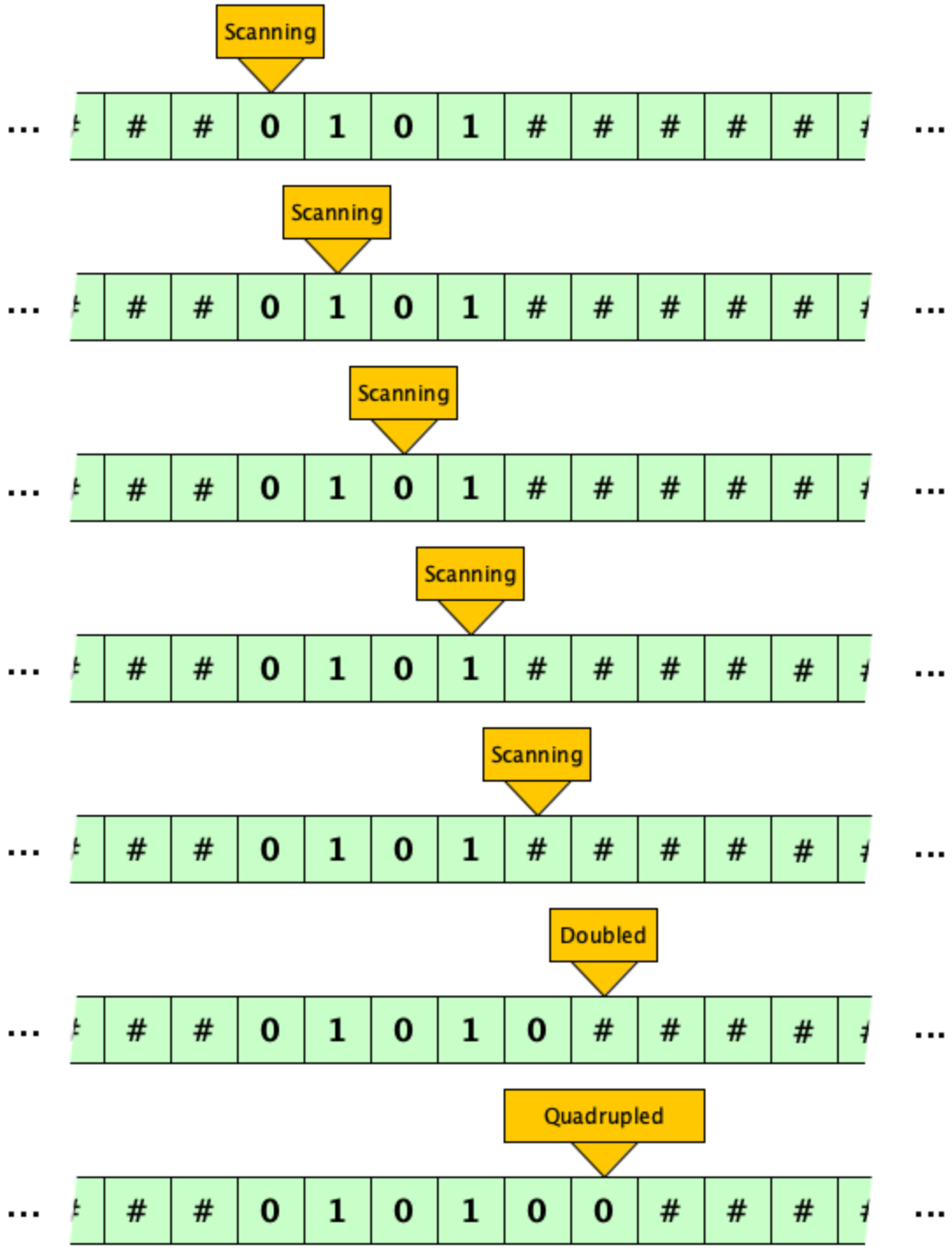
Turing set out to formalize what (human) computers did: they followed a set of instructions in which they examined some portion of their paper worksheet, wrote something down or erased something, then moved somewhere else or finished with their result. The actual instructions were based on the human's **state of mind** and what was already on the paper. He realized the contents of a two-dimensional sheet of paper could be just as easily represented as a one-dimensional sequence of symbols. So he then reduced the idea to a machine operating on an unbounded sequential paper **tape** with a read-write head initially pointing to the leftmost symbol of the input. At each step, the machine reads the current symbol, and depending on its current state, possibly rewrites it with a different symbol, moves one square to the left or right, and possibly enters a new state.

> ***Turing's Story***
>
> *History is important in understanding just about anything. The story of what motivated Turing to work on the foundations of computing and how he arrived at these fascinating machines, and his brilliant insights into the nature of computation are told very well in the book Turing's Vision: The Birth of Computer Science*
>
> *That book is well worth a read.*

Let's build a Turing Machine to multiply a binary number by four. How would we do this? Well, we just append two zeros to the end of the input! But how can we describe these steps more rigorously? What might the machine's instructions look like? We might say:

- Begin in the `Scanning` state, where our job is to look for the end of the input. In this state, we keep moving right until we encounter our first blank, which we replace with a 0 then move right, entering the `Doubled` state.
- In the `Doubled` state, we replace the blank with our second zero, entering the `Quadrupled` state.

Here is how our machine operates with the number five (binary 0101) initially on the tape (And oh, following convention, we use `#` for blank squares, just because it is easier to see!):

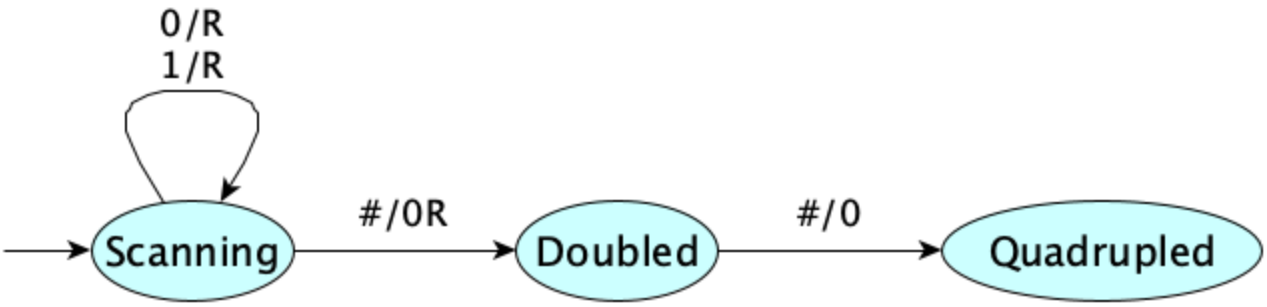


Nice! 5 × 4 is indeed 20 (binary 010100). Now let's capture the the multiply-by-four machine in all its glorious generality:

When in State	Looking At	Perform these actions	And go to State
Scanning	**0**	Move right	Scanning
Scanning	**1**	Move right	Scanning
Scanning	#	Print 0, Move right	Doubled
Doubled	#	Print 0	Quadrupled

This says: starting in the `Scanning` state, while you see a 0 or 1, leave it alone and move right, but if you see a blank, replace with a 0, move right, and go to the `Doubled` state. In this state, if you see a blank, replace with a 0 and go to the `Quadrupled` state. There is no place to go from here, so we're done.

We can give a pictorial representation of the TM program as a **state diagram**:



CLASSWORK

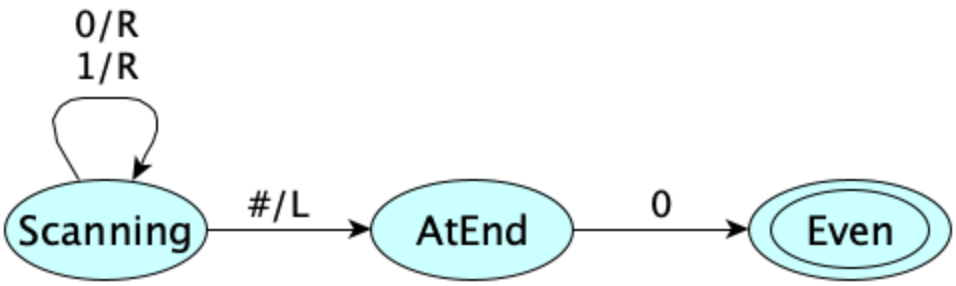
> Create a Turing Machine that **floor-divides** its input by four. This will be a little harder, as you will have to sometimes move left, and handle the case in which the input is very small and you will need to make sure an answer of 0 is properly written out.

> ***Exercise**: Give Turing machines that (a) increment a binary number by 1, (b) compute the two's complement of a binary number, and (c) compute the value of a binary number modulo 2.*

TMs for Decision Problems

If a Turing Machine is created to answer a yes-no question (a **decision problem**), such as "Is this number even?" or "Is this string a palindrome?" it theoretically could erase the whole tape and write a YES or NO symbol as soon as it figures out the answer. But there is a shortcut we almost always take: we can mark certain states as **accept states**. In such a TM, we don't care what remains on the tape—we only care whether the state we end up in when there are no moves to make is an accept state or not.

Example Time! Let's give a TM to determine whether a binary number is even:



Hey, there is a connection to language theory here! The TM just given can both answer the question "Is this number even?" but it also **recognizes the language** $\{w \in \{0,1\} \mid w \text{ is a even binary integer}\}$. That's right, answering yes-no questions is isomorphic to determining membership in a language.

***Exercise**: Reflect on this. Woah, right?*

***Exercise**: Give Turing machines that determine whether (a) a binary integer is odd, (b) a signed binary integer is negative, and (c) a string over $\{a, b, c\}$ is a palindrome.*

Universality

In principle, we can build Turing Machines for computing all kinds of functions, such as capitalizing text, finding words in text, computing interest payments, finding documents on the World Wide Web, etc. But functions themselves can be represented in symbols, whether in mathematical notation, plain text Python, or any formalism of your choice. In fact, the Turing machine we saw to multiply by four could be represented as:

```
S00RS;S11RS;S#0RD;D#0NQ;
```

Given this, perhaps we can build a machine (call it U), that when you give it ANY function f (as text), and some data x, then U will produce $f(x)$? That is:

$$U(f, x) = f(x)$$

As it turns out, for a certain class of functions, you *can* construct such a machine. You might take that for granted today, but it was not shown to be true until the 20th century! Really! (As far as anyone knows, that is.) This profound result is due to, you guessed it, Alan Turing, and it pretty much changed the world. This U is called the **Universal Turing Machine** in his honor. Today we build variations of the original U. We call them **computers**.

Stop.

Think about this.

Woah.

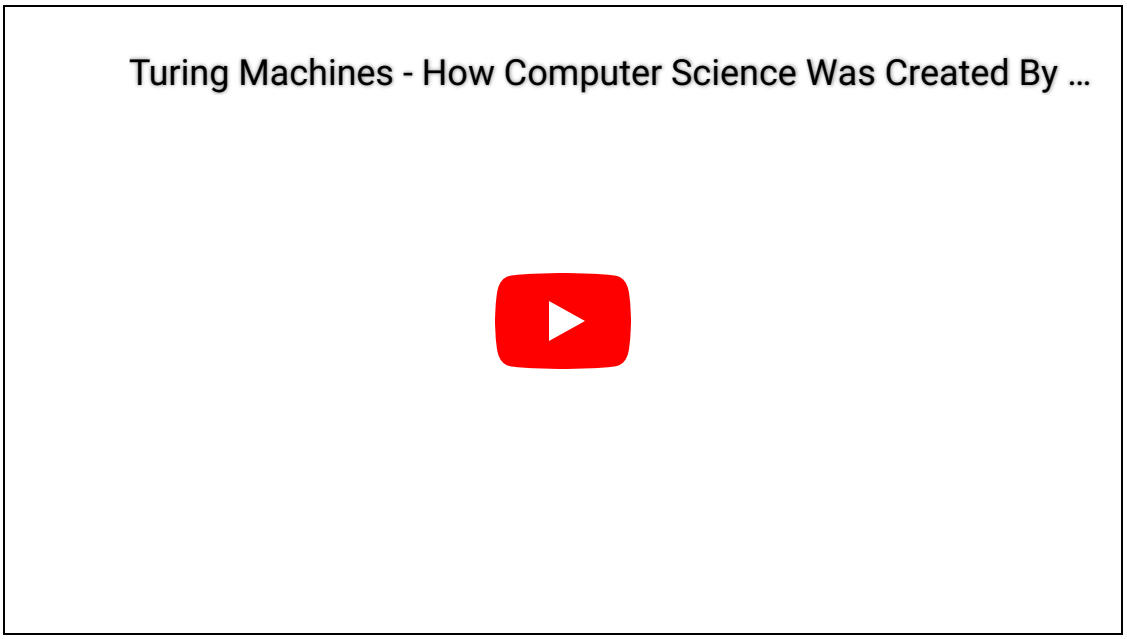
Just woah.

This may the most profound idea anyone has ever had about computing.

Are you starting to see why Alan Turing is revered as the founder of computer science?

Jade explains it better

Here's Jade Tan-Holmes explaining all about Turing Machines and Universality. She explains what motivated Turing to come up with the idea for formalizing computation, how he came up with the model he did, and the impact of universality. This video is so spot on, you need to watch it. Maybe watch it twice:



Other Kinds of Automata

Turing Machines are not the only model of what we think of as computation (though they might be the most famous). Other models that have been created include:

- Register Machines
- The Lambda Calculus

- Post Systems
- Partial Recursive Functions
- Analytic Grammars
- Queue Automata
- Stack Automata
- String Rewriting Systems
- Cellular Automata

Some have equivalent "computability and recognition power" as Turing Machines, while others do not. What do we mean by "computability and recognition power"? That's the domain of our next theory, computability theory.

Computability Theory

Now for our next philosophical question. **Can *every* function be represented as a string of symbols that we can process with a machine? In other words, is there a program for every function?**

It turns out that the answer to this is no.

Here's why. Programs are just strings, so we can list them in increasing alphabetic order. Programs can be thought of as representing functions in $\mathbb{N} \to \mathbb{N}$ (since all data can be encoded in binary). Call the enumeration $f_0, f_1, f_2, \ldots$. But $\lambda n.\ f_n(n) + 1$ isn't in the list! Bottom line:

There exist functions that cannot be computed by a computing machine.

Hey, *this is philosophically interesting*.

***Exercise**: Discuss why the limits to computation might have something to do the possibility of finding a "Theory of Everything". (Hint: See what Stephen Hawking has to say about this.)*

CLASSWORK

That explanation of why uncomputable functions exist was given rather nonchalantly. Did it make sense? It might not have right away because it is an instance of a very powerful proof technique called **diagonalization** which was at first somewhat controversial. Carry out this proof by sketching the hypothetical list of all functions as a matrix: each row is a function and each column is the argument, and each cell is the result of applying the function (row) to the

argument (column). Look at how $\lambda n.\ f_n(n) + 1$ is generated. Do you see why the technique is called diagonalization? Do you see why it works? 🤯

So which functions can be computed? People have, over the years, created many models of computation—the Lambda Calculus, Post Production Systems, Phrase Structure Grammars, Partial Recursive Functions, Random Access Machines, and more. But get this: all have turned out to be **equal in power, in terms of what they can compute, to the Turing machine**! Even enhancing the basic Turing Machine design with **non-determinism** or even **multiple tapes** won't increase its computing power. These facts suggest the class of computable functions is fundamental. The **Church-Turing Thesis** claims that what we intuitively understand as the "computable functions" are precisely those for which a Turing Machine can be given, i.e.,

A function f is **computable** if and only if a there exists a Turing Machine that outputs $f(x)$ for any input x.

***Exercise**: Research why this is called a thesis and not a law or theorem.*

Remember that function U we saw before? The Universal Turing Machine? It is "universal" because it can simulate any Turing Machine. If the Church-Turing Thesis is true, it's *really* universal: it can compute any computable function.

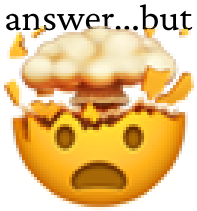
***Exercise**: Discuss why this is profound.*

Computability Theory is concerned with figuring out which functions are and are not computable.

> ***Turing Complete***
>
> *Because Turing Machines capture that which is computable, any computational model that can simulate a Turing Machine is called* **Turing Complete**. *Being Turing-Complete is a badge of honor, as it means your model can compute anything that is computable.*
>
> *Read more at Wikipedia.*

Fun fact: One of the more interesting things to come out of computability theory is that there are some languages that Turing Machines can recognize, but not decide. (A TM recognizes a language L if it enters an accept state and halts whenever given a string in L. It **decides** a language L if it both accepts L *and* for all strings not in L, the machine will always stop in a non-accept state.) So what

does it mean that some languages are recognizable but not decidable? It means there are some languages—and therefore some decision problems— for which we can definitely compute a YES answer...but when the answer is NO we might...never...know.



How about an example? We'll use JavaScript rather than TMs to simplify the explanation. Let's write a function called `halts` that will determine if a function halts on a given input. So we would like:

```
halts(Math.sqrt, 100)                 → true
halts(x => {while (x) {}}, true)      → false
```

The idea is simple. Just analyze the input and see if it will halt. But it turns out to be logically impossible to write this function. Why? Assume you can write it. Then consider:

```js
function fox(f) {
  if halts(f, f) while (true) {}
}
```

Now consider the expression `halts(fox, fox)`. Woah. We have an antinomy. The assumption you could write `halts` leads to a contradiction, so you can not write this function to work for all inputs. Interestingly, you can write:

```js
function halts(f, x) {
  f(x);
  return true;
}
```

So if `f(x)` halts, you'll eventually know it, but if it doesn't, we may never know.

Complexity Theory

Computability theory gives us theoretical limits on what can and cannot be computed, but doesn't tell us what is **practically** computable. (A problem requiring 2^{100} years to complete is not worth

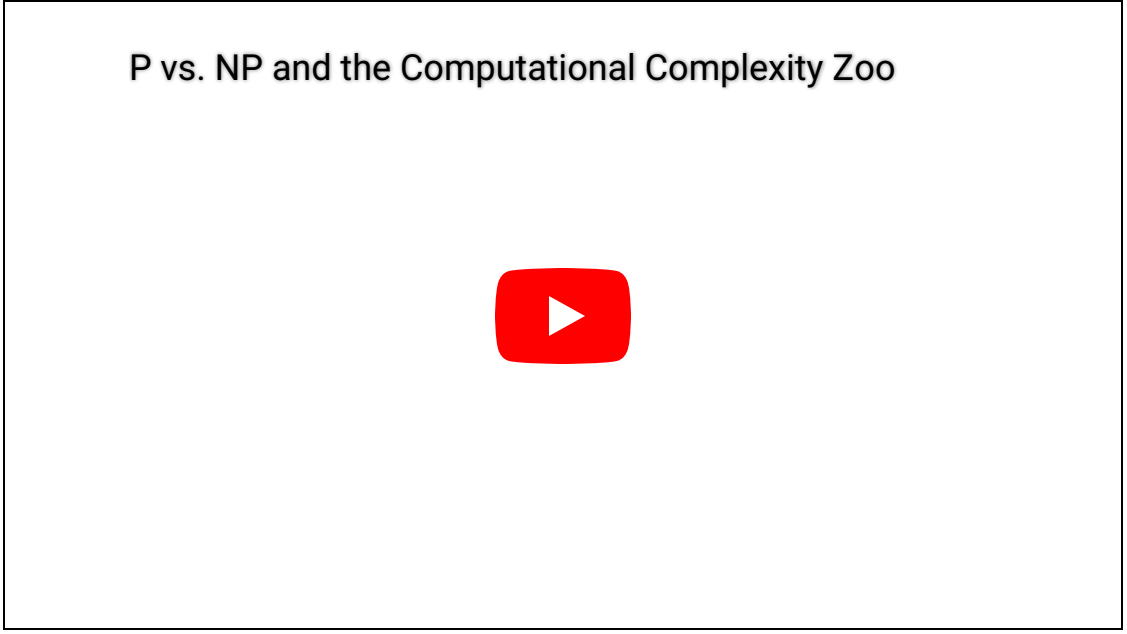
running.) The question “What resources are required to perform certain computations?” or in other words “How much time and memory is needed to solve this problem?” give rise to **Complexity Theory**.

Complexity Theory gives us results such as “This function requires _____ to compute”:

- only a single step
- only a fixed number of steps
- logarithmic time
- linear time
- quadratic time
- polynomial time
- exponential time
- no more than a single cell of scratch memory
- scratch memory in the form of a stack (which is very efficient)
- logarithmic space
- linear space
- polynomial space
- exponential space

It gets (massively) more interesting, because the theory deals with time and space performance on non-deterministic machines, and parallel machines, and interactive machines, and quantum computers, too. It even considers **probabilistic algorithms**. Interested? Check out the amazing Complexity Zoo.

Complexity Theory has philosophical as well as practical use. Make sure you are familiar with one of the most famous open questions in all of complexity theory, and perhaps even all of computer science, namely: Does $P = NP$? (Solve it and you will win a million dollars!)



Oh, wait! Here's Jade again with another great explainer:



There is so, so, much information out there about the P vs. NP question as it really seems to captivate the public. Not unlike Gödel's Incompleteness Theorems.

***Exercise**: Scott Aaronson wrote a blog post in 2006 (worth a read) in which he waxed poetic about a world in which P=NP. He later disavowed some of his more colorful claims, writing:*

> *I, Scott Aaronson, hereby formally disown my statement about how "everyone who could appreciate a symphony would be Mozart" if P=NP.*

In 2014, he wrote a new post on the scientific case for P≠NP. Read the later post. What do you think?

Working With These Theories

We've only briefly introduced the major concerns of four computer science theories without any real depth (or breadth). We'll have a chance to see each theory in more detail later, covering aspects such as:

- Formal, mathematical, definitions of languages (via grammars) and automata
- Restricted grammars and automata that capture specific kinds of computation
- Language hierarchies capturing both computability and complexity concerns
- Models of computation other than the Turing Machine
- More examples of non-computable functions (beyond the "Halting Problem" we saw above)
- How to construct a Universal Turing Machine
- Examples of languages in many of the common complexity classes
- How to determine the complexity of a problem
- How to create grammars for realistic programming languages
- Why non-trivial programming languages require both a static and dynamic semantics
- Extending these ideas into the much larger Programming Language Theory (PLT)
- How to run a program given its source code (interpret)
- How to translate one programming language into another (compile)

Theories and Creativity, Again

In much the same way that Bennett pointed out that music theory can expand your abilities in music composition, and Feynman spoke of theoretical knowledge in science only expanding and enhancing the feelings of awe and beauty when contemplating nature, might computer science theory enhance one's imagination for creating new worlds that Booch talks about here?



I am a computer scientist; I approach the cosmos though a very different path: I take the simplest of principles - information theory, logic, design - and from them create new worlds that are bound only by

my imagination.

— Grady Booch (@Grady_Booch) February 12, 2020

Recall Practice

Here are some questions useful for your spaced repetition learning. Many of the answers are not found on this page. Some will have popped up in lecture. Others will require you to do your own research.

What exactly is a **theory**, in the context of art and science? HIDE ANSWER

An organized body of knowledge with explanatory and predictive powers.

Why are theories important? HIDE ANSWER

They provide us with a vocabulary to communicate ideas, test hypotheses, and generate new knowledge.

What are the four major computation theories and what are they concerned with? HIDE ANSWER

1. Language Theory: expressing computations
2. Automata Theory: carrying out computations
3. Computability Theory: What can and cannot be computed?
4. Complexity Theory: Resources required for certain computations

What is a language, in formal language theory? HIDE ANSWER

A set of strings over some finite alphabet.

What is the lambda calculus seemingly most concerned with representing and manipulating? HIDE ANSWER

Functions

Who came up with the idea of formalizing computing via the Lambda Calculus? HIDE ANSWER

Alonzo Church

Who came up with the idea of formalizing computing via the Turing Machine? HIDE ANSWER

Alan Turing

What was Turing trying to do when he realized he would have to do no less than formally define the very notion of an algorithm (or computational process)? HIDE ANSWER

He was trying to show Hilbert's Entscheidungsproblem did not have a solution

On what did Turing more-or-less base his fundamental idea of how a formal (abstract mechanical) computing device should work? HIDE ANSWER

(Human) computers, calculating on paper by moving through mental states looking at symbols, adding new symbols, and erasing existing ones.

What are the main components of a Turing Machine? HIDE ANSWER

1. A control unit made up of states and a transition relation, and
2. An infinite one-dimensional tape with cells that can each hold a single symbol or be blank.

Usually, a Turing machine just rewrites its input into an output. But sometimes, all we want to compute is the answer to a YES-NO question. In this case, how does the Turing Machine announce its answer? HIDE ANSWER

It says YES by entering an ACCEPT state with no outgoing transitions for the current symbol, and answers NO by getting into a non-ACCEPT state with no outgoing transition for the current symbol. It is possible for the machine to

loop forever on a given input, in which case it is said to give no answer.

Answering a YES-NO question on a Turing Machine is isomorphic to _____ a language.

HIDE ANSWER

Recognizing

Why is Turing's discovery/invention of the Universal Turing Machine so profound?

HIDE ANSWER

It means that truly general-purpose, programmable computing machines exist (i.e., we don't need specialized machines for all computations).

How can we prove there are functions that cannot be computed? HIDE ANSWER

By diagonalizing over an assumed enumeration of all functions.

What does the Church-Turing Thesis say? HIDE ANSWER

That the Turing Machine lines up exactly with our notion of what is intuitively computable.

What is the difference between *deciding* and *recognizing*? HIDE ANSWER

A TM decides a language if it always halts with a YES or NO answer. A TM recognizes a language if it always correctly answers YES for strings in the language, but it may or may not halt with a NO for non-members.

What are the language classes P and NP? HIDE ANSWER

P is the set of languages decidable in polynomial time on a deterministic TM; NP is the set of languages decidable in polynomial time on a nondeterministic TM.

Does P = NP?

Did Scott Aaronson really write “everyone who could appreciate a symphony would be Mozart”? How does he feel about writing that these days? HIDE ANSWER

He did indeed write that but he regrets writing it now.

What are some variations we can make to Turing Machines that possibly affect complexity (if not computability)? HIDE ANSWER

Randomized machines, probabilistic machines, quantum machines.

How does Grady Booch distinguish the work in material domains (physics, chemistry, biology, etc.) from that of computer science? HIDE ANSWER

Scientists in material domains observe the cosmos and reduce it to simple principles; computer scientists start with simple principles and from them create new worlds bound only by the imagination.

Summary

We’ve covered:

- ✅ What a theory is
- ✅ Why study theory?
- ✅ Theories within computer science
- ✅ What Language Theory is
- ✅ What Automata Theory is
- ✅ What Computability Theory is
- ✅ What Complexity Theory is

✅ Where we can go from here

Language Theory



To understand how to express information and computation, it is useful to have a formal theory of what a language is.

CONTENTS

Why? • Preliminaries • Symbols • Alphabets • Strings • Formal Languages • Language Generation • Language Recognition • Formal Language Classification • Semantics • Programming Language Theory (PLT) • Recall Practice • Summary

Why?

Why would we ever even want a theory of languages?

Languages are all about how we represent and communicate information and computations. A formal theory of language allows us to rigorously define what language is, and from that we can show exactly what various languages can represent and say.

Preliminaries

Theories of computation, including language theory, start from the idea that information can be represented as a string of symbols.

Information is that which informs. In information theory, it is the resolution of uncertainty. The more you know, the less uncertain you are.

Information theory and language theory both use a fair amount of logical and mathematical notation. If you need to brush up, see these logic notes and these math notes.

Symbols

A **symbol** is a primitive unit of data representation. Information can be represented with **strings** of symbols. For example:

- `91772.3e-8`
- `Stay behind the yellow line!`
- `ᐊᐃᖏᓗᐃ ᓄᓇᕐᕎᐊᖅ`
- `/Pecs/Los Angeles/Berlin/Madrid//1 2/3 0/2 2/2 0/3 2///`
- `(* (+ 4 3) (-9 7))`
- `int average(int x, int y) {return (x + y) / 2;}`
- `<dog id="30024"><name>Lisichka</name><age>13</age></dog>`
- `{"type": "dog", "id": "30024", "name": "Lisichka", "age": 13}`
- `∀α β. α⊥β ⇔ (α•β = 0)`
- `1. f3 e5 2. g4 ♕h4++`

> ***Exercise**: Do you buy it? Can you think of any information that cannot be represented with a string of symbols? What about images? Sounds? Thoughts? Ideas? It's okay to get philosophical and debate!*

Alphabets

Symbols are chosen from an **alphabet**, such as $\{0, 1, 2, 3, 4, 5, 6, 7, 8, 9\}$ for the language of natural numbers, or $\{0, 1, 2, 3, 4, 5, 6, 7, 8, 9, ., \mathrm{e}, \mathrm{E}, +, -\}$ for the language of floating-point numbers.

Here are some examples of alphabets:

- $\mathsf{BinaryDigits} = \{0, 1\}$
- $\mathsf{DecimalDigits} = \{0, 1, 2, 3, 4, 5, 6, 7, 8, 9\}$
- $\mathsf{HexDigits} = \mathsf{DecimalDigits} \cup \{a, b, c, d, e, f, A, B, C, D, E, F\}$
- $\mathsf{Suits} = \{\spadesuit, \heartsuit, \diamondsuit, \clubsuit\}$

- PropositionalLogicSymbols $= \{p, q, r, \prime, (,), \neg, \wedge, \vee, \supset, \equiv\}$
- Unicode $= \{\sigma \mid \sigma \text{ is a Unicode character}\}$

Strings

A **string** is a finite sequence of symbols over some alphabet.

> **Examples**: For the alphabet $\{a, b, c\}$, three example strings are: $abbcbabb$, b, aaa.

When talking about strings, we might use variables to represent them. In this case the variable names should be, necessarily, outside the alphabet:

> **Examples**: $w = abbcbabb$, $x = b$, $y = aaa$.

The **length** of a string is the number of *symbol occurrences* in the string.

> **Example**: if $w = abbac$, then $|w| = 5$.

The **empty string**, denoted ε, is the (unique) string of length 0.

The **concatenation** of two strings x and y is the string obtained by appending y to the end of x. We have a notation for repeated concatenation: $w^0 = \varepsilon$, $w^1 = w$, $w^2 = ww$, and $w^{k+1} = ww^k$.

> **Example**: if $x = abb$ and $y = ca$ then $xy = abbca$.
>
> **Example**: $w\varepsilon = \varepsilon w = w$.
>
> **Example**: $(abc)^4 = abcabcabcabc$.

> ***Tricky Notation?***
>
> *String concatenation looks like numeric multiplication, and string repetition looks like numeric exponentiation, but behaviorally, concatenation is like numeric addition and repetition is like numeric multiplication. If you've programmed in Python, you'll see that is indeed true.*
>
> ```
> $ python
> >>> "abc" + "def"
> 'abcdef'
> >>> "abc" * 3
> 'abcabcabc'
> ```

Guido got it right.

The **reversal** w^R of a string w is the string formed by reversing w; formally $\varepsilon^R = \varepsilon$ and $(\sigma w)^R = w^R \sigma$ for some symbol σ.

x is a **substring** of y iff $\exists v.\ \exists w.\ y = vxw$.

Because we are defining languages with an eye toward modeling computation, we almost always require alphabets to be non-empty and finite, and require all strings to have a finite (but unbounded) length. Even with a finite alphabet and finite-length strings, you can still have an infinite number of strings!

> **Example**: The finite alphabet $\{0, 1\}$ gives rise to the infinite number of strings 0, 1, 00, 01, 10, 11, 000, 001, etc.

Formal Languages

A **formal language** is a set of strings over some alphabet. That's it. That's the definition. If a string w is a member of a language L we say w is an **utterance** of L.

Examples

Examples first! Here are some example languages:

- $\{w \in \{a, b\}^* \mid w \text{ has the same number of } a\text{'s as } b\text{'s}\}$
- $\{w \in \{a, b\}^* \mid w \text{ has more } a\text{'s than } b\text{'s}\}$
- $\{w \in \{a, b\}^* \mid \text{the first and last symbol of } w \text{ are the same}\}$
- $\{a^n b^n \mid n \geq 0\}$
- $\{a^i b^j \mid j = 2^i\}$
- $\{a^i b^j c^k \mid k = i + j\}$
- $\{ww \mid w \in \{a, b\}^*\}$
- $\{ww^R \mid w \in \{a, b\}^*\}$
- $\{1^n \mid n \text{ is prime}\}$
- $\{w \in \{0, 1\}^* \mid w \text{ is a binary numeral greater than } 55\}$
- $\{w \in \{0, 1\}^* \mid w \text{ is a binary numeral divisible by } 5\}$
- $\{w \in \text{DecimalDigit}^* \mid w \text{ is a valid MasterCard number}\}$

- $\{w \in \text{Unicode}^* \mid w \text{ contains the substring } dog\}$
- $\{w \in \text{Unicode}^* \mid w \text{ is a valid Python program}\}$
- $\{w \in \text{Unicode}^* \mid w \text{ is a valid JavaScript program}\}$
- $\{w \in \text{Unicode}^* \mid w \text{ is a valid Swift program}\}$
- $\{w \in \text{Unicode}^* \mid w \text{ is a valid Swift program that prints the integers from 1 to 10}\}$

***Exercise**: For each of the example languages above, give a few strings in (utterances of) the language.*

> ***Why are languages sets?***
>
> *A language as a set of strings is actually quite useful. It distinguishes strings "in" the language from those not in the language. A language is a set of (legal) utterances.*
>
> *Now it's true that this definition does not say anything about the "meaning" of each utterance. That has to be handled separately. But we are off to a good start.*

Language Operations

Because languages are sets, $\varnothing$ is a language, and the **union** $(L_1 \cup L_2)$ and **intersection** $(L_1 \cap L_2)$ of languages make perfect sense. So does the **cardinality** of a language, $|L|$, the number of strings in the language. In addition:

- The **concatenation** of two languages L_1 and L_2 is $L_1L_2 = \{xy \mid x \in L_1 \wedge y \in L_2\}$. Also, we define $L^0 = \{\varepsilon\}$, $L^1 = L$, $L^2 = LL$, ..., $L^{k+1} = LL^k$.
- The **Kleene Closure** of L is $L^* = L^0 \cup L^1 \cup L^2 \cup \ldots = \{w_1w_2 \ldots w_k \mid w_i \in L \wedge k \geq 0\}$.
- The **Positive Closure** of L is $L^+ = L^1 \cup L^2 \cup L^3 \cup \ldots = \{w_1w_2 \ldots w_k \mid w_i \in L \wedge k > 0\}$.

Therefore, given an alphabet Σ, the following are languages over Σ:

- $\varnothing$, the empty language, the language of no strings
- $\{\varepsilon\}$, the language containing the empty string and nothing else
- Σ, the language of all possible one-symbol strings
- Σ^2, the language of all possible two-symbol strings
- Σ^3, the language of all possible three-symbol strings
- Σ^*, the language of all possible strings over the alphabet Σ

> ***The Infinite Library***
>
> *The language Σ^* (the language of all possible strings over an alphabet) is fascinating. It's basically the infinite library, which contains every book that has ever been written and that could possibly ever be written, i.e., every single possible arrangement of symbols. It contains solutions to climate change and world hunger, the cures for all cancers, and all human histories and futures (yours included). It also contains an infinite amount of misinformation. The problem is, you can't distinguish what is right from wrong.*
>
> *When you "know" everything, you know nothing.*

For any language L over the alphabet Σ, we have $L \subseteq \Sigma^*$. Since we *must* specify an alphabet for each language, the **complement** of a language makes sense. It is defined as follows: $\overline{L} = \Sigma^* - L$

Let's see the language operations in use. Suppose we are given the alphabet $\Sigma = \{a, b\}$, and two languages over Σ, which we'll define as $L_1 = \{ba, b\}$ and $L_2 = \{abb, bb\}$.

Then:

- $\Sigma^* = \{\varepsilon, a, b, aa, ab, ba, bb, aaa, aab, aba, abb, baa, bab, \dots\}$
- ${L_1}^* = \{\varepsilon, ba, b, baba, bab, bba, bb, \dots\}$
- ${L_1}^+ = \{ba, b, baba, bab, bba, bb, \dots\}$
- $L_1 L_2 = \{baabb, babb, bbb\}$
- $L_1 \cup L_2 = \{ba, b, abb, bb\}$
- $L_1 \cap L_2 = \varnothing$

Random Language Facts

Here are some interesting theorems about languages. Ready? Let Σ be an alphabet and A, B, C, and D be languages over Σ (i.e., subsets of Σ^*). Then:

- $A\varnothing = \varnothing A = \varnothing$
- $A\{\varepsilon\} = \{\varepsilon\}A = A$
- $(AB)C = A(BC)$
- $(A \subseteq B) \wedge (C \subseteq D) \Rightarrow AC \subseteq BD$
- $A(B \cup C) = AB \cup AC$
- $(B \cup C)A = BA \cup CA$
- $A(B \cap C) \subseteq AB \cap AC$

- $(B \cap C)A \subseteq BA \cap CA$

- $A^* = A^+ \cup \{\varepsilon\}$
- $n \geq 0 \Rightarrow A^n \subseteq A^*$
- $n \geq 1 \Rightarrow A^n \subseteq A^+$
- $A \subseteq AB^*$
- $A \subseteq B^*A$
- $A \subseteq B \Rightarrow A^* \subseteq B^*$
- $A \subseteq B \Rightarrow A^+ \subseteq B^+$
- $AA^* = A^*A = A^+$
- $\{\varepsilon\} \in A$ iff $A^+ = A^*$
- $(A^*)^* = A^*A^* = A^*$
- $(A^*)^+ = (A^+)^* = A^*$
- $A^*A^+ = A^+A^* = A^*$
- $(A^*B^*)^* = (A \cup B)^* = (A^* \cup B^*)^*$

- $(C = AC \cup B) \Rightarrow (C = A^*B)$

> ***Exercise**: For each of the above theorems, try to understand what they are saying and why they are true. If you are motivated, try to write proofs for each. They aren't super hard to prove, provided you know some basic Set Theory.*

Language Definition

Formal languages were developed to help us study the foundations of mathematics and computation. It's true that in this context, languages are just sets, so we can communicate them in set notation, for example:

- $\{w \mid w \textbf{ is a legal floating point numeral}\}$

But wait, even though that definition used mathematical symbols in part, there was a big informal piece between the curly braces. This can be a problem. Informal definitions lack precision. Like what even are floating point numerals? We can list as many examples as possible to get the idea. Maybe we can list them all, say something like { 0, 8, 3.5, 13.55, 3E8, 3.2E89, 5.111E+2, 21.34E-2, 1e-8, 2.55e+233, ...}. This is also not precise. Let's try harder, with a definition:

- The set of all strings that begin with a sequence of one or more decimal digits,
  - optionally followed by a dot then one or more decimal digits
  - optionally followed by:

- an `E` or an `e`
- then, optionally, a `+` or `-`
- then a sequence of one or more decimal digits

Also not very satisfying: it is very verbose, kind of confusing, and hard to tell if it is even right. It is "precise" *but it is still informal*! We need something better. Something formal. As it happens, we can do this! There are two major approaches:

Definition by Generation

We can give a formalized set of rules that systematically generate all and only those strings in the language. The language is then the set of all string **generated** by the language.

Definition by Recognition

We can give a formal definition of a process that determines, for a given string, whether or not it is in the language. The language is then the set of all strings **recognized** by the process.

Language Generation

The primary mechanism to define a language via generation is the generative grammar.

Generative Grammars

A **generative grammar** is a list of **rules** that generates all and only those strings in a particular language. The rules have **variables** that get replaced in order to generate the actual strings of the language. Here's a grammar for our floating point numeral language. The alphabet for this language is $\{0, 1, 2, 3, 4, 5, 6, 7, 8, 9, ., E, e, +, -\}$. Our grammar will use four variables, $\{\textit{numeral}, \textit{fractionalPart}, \textit{exponentPart}, \textit{digit}\}$:

numeral	$\longrightarrow$	*digit*$^+$ *fractionalPart*? *exponentPart*?
fractionalPart	$\longrightarrow$	`"."` *digit*$^+$
exponentPart	$\longrightarrow$	(`"E"`|`"e"`) (`"+"`|`"-"`)? *digit*$^+$
digit	$\longrightarrow$	`"0".."9"`

There are multiple grammar notations out there; the one used here features the following notation:

- → means “can be replaced with”
- `+` means “one or more”
- `*` means “zero or more”
- `?` means “optional”
- `|` means “or”
- Double quotes delimit the actual symbols in the language you are defining, while variables are unquoted
- Parentheses are for grouping
- `..` makes a range over individual symbols (yeah, it’s kind of shorthand)

If G is a grammar, then $L(G)$ is the language defined by the grammar G. A string is in $L(G)$ if it is **derivable** from G’s initial rule, repeatedly replacing instances of a rule’s left hand pattern with its right hand side, until you get completely rid of the variables and are left only with strings in the language’s alphabet. Here is an example **derivation**:

$$
\begin{aligned}
\textit{numeral} &\Longrightarrow \textit{digit}^+\ \textit{fractionalPart}?\ \textit{exponentPart}? \\
&\Longrightarrow \texttt{338}\ \textit{fractionalPart}?\ \textit{exponentPart}? \\
&\Longrightarrow \texttt{338.}\ \textit{digit}^+\ \textit{exponentPart}? \\
&\Longrightarrow \texttt{338.253}\ \textit{exponentPart}? \\
&\Longrightarrow \texttt{338.253}\ (\texttt{"E"}|\texttt{"e"})\ (\texttt{"+"}|\texttt{"-"})?\ \textit{digit}^+ \\
&\Longrightarrow \texttt{338.253e}\ (\texttt{"+"}|\texttt{"-"})?\ \textit{digit}^+ \\
&\Longrightarrow \texttt{338.253e}\ \textit{digit}^+ \\
&\Longrightarrow \texttt{338.253e18}
\end{aligned}
$$

***Exercise**: Show that the following strings are in the language defined by the grammar above:*

- *3*
- *8.5501e-88*
- *5E+11*

and that the following are not in the language:

- *-2.3E1*
- *55.e+2*
- *^^Dog!*

Also, generate three of your own strings in the language and three strings not in the language.

***Exercise**: Make sure you understand the difference between $\longrightarrow$ and $\Longrightarrow$.*

By the way, note that there can be many grammars that specify the same language. For example, here's another grammar for the floating-point numeral language:

numeral	⟶	*digit*$^+$ ("." *digit*$^+$)? (("E"|"e") ("+"|"−")? *digit*$^+$)?
digit	⟶	"0".."9"

If the first grammar above was called G_1 and this grammar was called G_2, we would have $L(G_1) = L(G_2)$.

Practice with Grammars

If you think about it, coming up with a grammar for a language you have in mind is an endeavor similar to programming: you know what you want to end up with, but you need to create the very formal, precise steps to get you there. How do you get good at coming up with grammars? Certainly it helps to have a mentor and learn tricks of the trade over time with practice. Whether you have a mentor or not, it does help to start with a whole bunch of examples. So with that in mind, here are a handful of grammars for you to study:

The language of integer binary numerals divisible by 2:

even	⟶	"−"? ("0" | "1")* "0"

The language of integer decimal numerals divisible by 5:

divisibleByFive	⟶	"−"? *digit** ("0" | "5")
digit	⟶	"0".."9"

The language $\{\varepsilon, 0, 1, 01, 10, 010, 101, 0101, 1010, \ldots\}$ of alternating zeros and ones:

alternating	⟶	*startsWithZero* | *startsWithOne*
startsWithZero	⟶	ε | "0" *startsWithOne*
startsWithOne	⟶	ε | "1" *startsWithZero*

The language $\{0^i 1^j \mid i \text{ is even} \wedge j \text{ is odd}\}$:

evenZerosThenOddOnes	⟶	("00")* "1" ("11")*

The language $\{a^n b^n \mid n \geq 0\} = \{\varepsilon, ab, aabb, aaabbb, aaaabbbb, \ldots\}$:

anbn	⟶	ε | "a" *anbn* "b"

The language of palindromes over $\{a, b\}$:

pal	⟶	ε | `"a"` | `"b"` | `"a"` *pal* `"a"` | `"b"` *pal* `"b"`

The language of parentheses, curly braces, and square brackets, in which everything is properly balanced and nested, for example `()[{()}[[]]()]`:

balanced	⟶	(`"("` *balanced* `")"` | `"["` *balanced* `"]"` | `"{"` *balanced* `"}"`)*

The language $\{a^n b^n c^n \mid n \geq 0\}$:

s	⟶	ε | `"a"` *s*? *x* `"c"`	
`"c"` *x*	⟶	*x* `"c"`	—move *c*'s to the right of the *x*'s
`"a"` *x*	⟶	`"ab"`	—only make a *b* after an *a* ...
`"b"` *x*	⟶	`"bb"`	—... or after a *b*

The language $\{a^{2^n} \mid n \geq 0\}$:

aToTwoToN	⟶	*double** `"a"` *right*	—mark the end
double `"a"`	⟶	`"aa"` *double*	—move right, doubling *a*'s as you go
double right	⟶	*right*	—got to end, no more to double
right	⟶	ε	

The language $\{ww \mid w \in \{a, b\}^*\}$:

s	⟶	`"a"` *s x* | `"b"` *s y* | *middle*	
middle x	⟶	*middle* `"a"`	—an *x* generates an *a*
middle y	⟶	*middle* `"b"`	—a *y* generates a *b*
`"a"` *x*	⟶	*x* `"a"`	—move *a*'s to end
`"a"` *y*	⟶	*y* `"a"`	
`"b"` *x*	⟶	*x* `"b"`	—move *b*'s to end
`"b"` *y*	⟶	*y* `"b"`	
middle	⟶	ε	—drop midmarker when no longer needed

Exercise: *Give a grammar for the language of unary primes:* $\{1^p \mid p \text{ is prime}\}$

Why are we studying such silly languages?

Obviously we need to get to the point where we can design grammars for real programming languages. But studying the fundamentals of language and grammars with bite-size pieces

helps us uncover patterns that we can use in designing our large grammars. It also gives insights into what is easy and what is hard. For example: some grammars above looked very straightforward and some required bizarre "marker" symbols that were passed around to..."compute" something it seems. The scientist in you should wonder if distinctions like that are fundamental in some sense, and if so, how can we capture this state of affairs formally.

CLASSWORK

We are going to do lots of derivations for many of the grammars above.

More Example Grammars

You can't have too many examples, so let's see more. To help us out a bit, we're going to take the following variables a being previously defined:

- ***digit***, for `"0".."9"`
- ***hexDigit***, for ***digit*** | `"A".."F"` | `"a".."f"`
- ***letter***, for any Unicode letter
- ***alnum***, for ***letter*** | ***digit***

Here is the language of words starting with a letter and containing letters, digits, and underscores only:

id	⟶	***letter*** (***alnum*** | `"_"`)*

The language of simple hex colors in HTML: Hash followed by 3 or 6 hex digits:

simpleHexColor	⟶	`"#"` ***threeHexDigits threeHexDigits?***
threeHexDigits	⟶	***hexDigit hexDigit hexDigit***

Now let's create a language for integer expressions with addition, subtraction, multiplication, division, exponentiation, negation, and parentheses. When languages start to get complex, it helps to see examples and non-examples:

Examples	Non-Examples
432 24*(31/899+3-0)/(54/-2+(5+2)*3)	43 2 24*(31////)/(5+---+))

(2) 8*(((3-6)))	[fwe]23re31124efr$#%^@ --2--

Here is a first pass. We say: "An expression is either (1) an integer, (2) a negated expression, (3) two expressions with a binary operator between them, or (4) a parenthesized expression." This gives us:

$$
\begin{array}{lcl}
\mathit{exp} & \longrightarrow & \mathit{digit}^{+} \\
& | & \texttt{"-"}\ \mathit{exp} \\
& | & \mathit{exp}\ (\texttt{"+"}\ |\ \texttt{"-"}\ |\ \texttt{"*"}\ |\ \texttt{"/"}\ |\ \texttt{"\%"}\ |\ \texttt{"**"})\ \mathit{exp} \\
& | & \texttt{"("}\ \mathit{exp}\ \texttt{")"}
\end{array}
$$

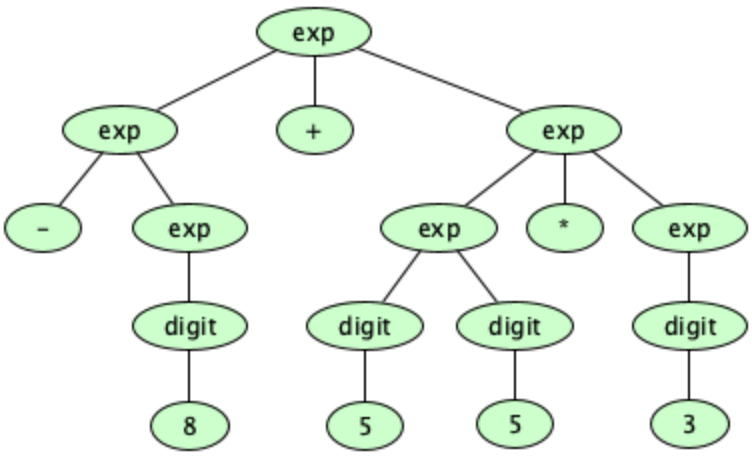
How did we do? Let's see if we can derive something interesting:

$$
\begin{array}{lcl}
\mathit{exp} & \Longrightarrow & \mathit{exp}\ \texttt{+}\ \mathit{exp} \\
& \Longrightarrow & \texttt{-}\ \mathit{exp}\ \texttt{+}\ \mathit{exp} \\
& \Longrightarrow & \texttt{-}\ \mathit{digit}\ \texttt{+}\ \mathit{exp} \\
& \Longrightarrow & \texttt{-8+}\ \mathit{exp} \\
& \Longrightarrow & \texttt{-8+}\ \mathit{exp}\ \texttt{*}\ \mathit{exp} \\
& \Longrightarrow & \texttt{-8+}\ \mathit{digit}\ \mathit{digit}\ \texttt{*}\ \mathit{exp} \\
& \Longrightarrow & \texttt{-8+5}\ \mathit{digit}\ \texttt{*}\ \mathit{exp} \\
& \Longrightarrow & \texttt{-8+55*}\ \mathit{exp} \\
& \Longrightarrow & \texttt{-8+55*}\ \mathit{digit} \\
& \Longrightarrow & \texttt{-8+55*3}
\end{array}
$$

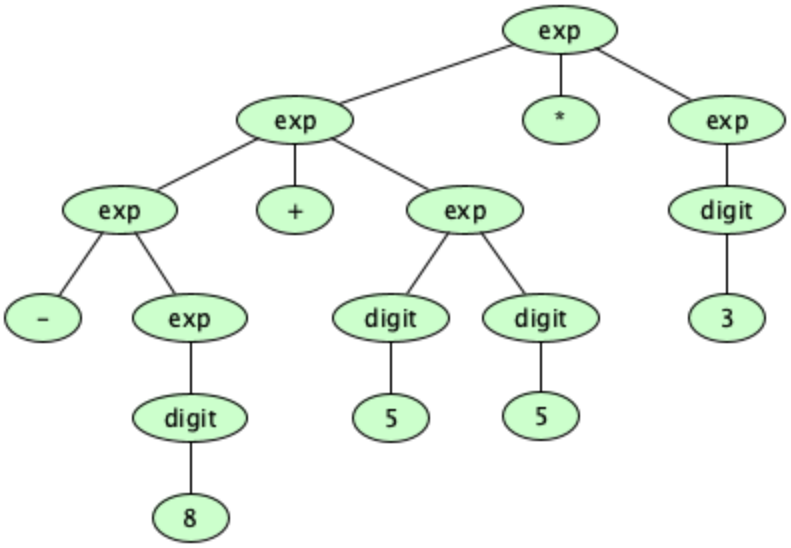
Derivations can get pretty long. Is there a better way to see how our strings are made?

Derivation Trees

Here's a better way to "show" the derivation we just performed:



But wait! There can be more than one way to derive a string, and sometimes (**but not always**) the resulting **derivation tree** is different for different derivations:



> ***Exercise***: *Show the (long-form) derivation of the string corresponding to the second tree above.*

> ***Exercise***: *Give at least two derivation trees for* `-5*(34/-55+(21-8)+233)`. *How many distinct trees are there for this string?*

Is having multiple derivation trees for a string a big deal?

Ambiguity

A grammar is **ambiguous** if there exists more than one derivation tree for a given string. Natural language grammars are often ambiguous.

> ***Exercise***: *Show that the following sentences, possibly derived from a grammar for English, are ambiguous.*
>
> - *Lawyers give poor free advice.*
> - *Teacher strikes idle kids.*
> - *Seven foot doctors sue hospital.*
> - *Stolen painting found by tree.*
> - *Enraged cow injures farmer with ax.*
> - *Two sisters reunited after 18 years at checkout counter.*
> - *The train left the station crowded and dirty.*

- *Landmine claims dog arms company.*
- *Hershey bars protest.*
- *They cooked for the first time on the stove.*

Try your best to show reasonable English grammar "trees."

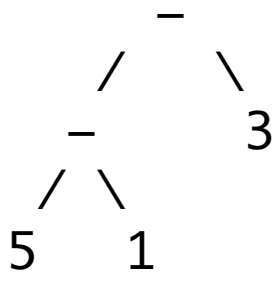
But for a programming language, we don't really like ambiguity, since the derivation tree tells us the structure of a program (its **syntax**) from which we wish to assign a meaning (**semantics**). Under the widely accepted meanings of decimal integers and arithmetic operations, those two trees we saw earlier suggest meanings of 157 (multiply-then-add) and 141 (add-then-multiply). Yikes!

One way to get rid of ambiguity in our language is to force operator **precedence** and **associativity** in the grammar. The following convention has arisen:

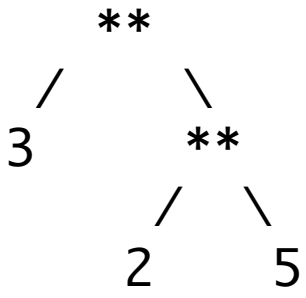
- Expressions are terms separated by left-associative additive operators (`+`, `-`)
- Terms are factors separated by left-associative multiplicative operators (`*`, `/`, `%`)
- Factors are primaries separated by right-associative exponentiation operators (`**`)
- Primaries are numerals, negated primaries, or parenthesized expressions.

Because expressions are made up of terms and terms are made up of factors, the utterance `1+3*2` is made up of two terms, `1` and `3*2`. Therefore we say multiplication has **higher precedence** than addition. This is the only way to make the derivation tree! There is no ambiguity here. Now what about `(1+3)*2`? This expression has a single term, which has two factors: `(1+3)` and `2`. Note how the parentheses matter!

But what about the utterance `5-1-3`? Should this be structured to do the leftmost subtraction first (yielding 1) or the rightmost first (yielding 7)? This is where associativity comes in. Tradition says subtraction is **left-associative**, so we want a structure with:



Exponentiation is traditionally **right-associative**, so `3**2**5` would desire a derivation structure as:



Do you know what else might be ambiguous? Expressions like `-2**2`. In Python, exponentiation is done before negation, so this expression produces `-4`. In Elm, negation happens first, so the expression evaluates to 4. JavaScript uses a grammar that prevents the expression from being derived by the grammar! This is a good thing. Let's do it.

We are now ready to give the **non-ambiguous** grammar for our integer arithmetic expression language:

exp	⟶	*exp* ("+" | "−") *term*
	|	*term*
term	⟶	*term* ("*" | "/" | "%") *factor*
	|	*factor*
factor	⟶	*primary* ("**" *factor*)?
	|	"−" *primary*
primary	⟶	*numeral*
	|	"(" *exp* ")"
numeral	⟶	$digit^+$

CLASSWORK

We are going to make a long-ish expression and show that it has one and only one derivation tree.

***Exercise**: Show that `-3**2` cannot be derived from this grammar.*

The Formal Definition of a Grammar

We've seen that we use grammars to formally describe mathematical objects we call languages. But we were kind of hand-wavy in defining what a grammar is and how derivations work. Grammars are mathematical objects themselves and so can be described formally. Here is how:

> A **grammar** is a quadruple (V, Σ, R, S) where:

- Σ is the alphabet of the language being described
- V is a set of **variables**, such that $\Sigma \cap V = \varnothing$
- $R \subseteq (V \cup \Sigma)^* V (V \cup \Sigma)^* \times (V \cup \Sigma)^*$ is a set of rules
- $S \in V$ is the start variable

The formal definition of R looks complex but it really just says "the left hand side must contain at least one variable." That is all. Super reasonable too! It's there so once you get rid of all the variables, you don't keep going.

Example. The grammar shown earlier for the language

$$\{0^n 1^n \mid n \geq 0\}$$

is *formally* denoted

$$(\{s\}, \{0, 1\}, \{(s, \varepsilon), (s, 0s1)\}, s)$$

Please work through this to make sure you "see" it and understand it.

***Exercise**: Give the formal representation of the grammars from the previous section.*

Derivations can be formalized too:

> Given any grammar rule set R, there exists a corresponding **derivation relation** $\Longrightarrow_R$ defined as $\{(xwy, xzy) \mid wRz\}$. That is, $xwy \Longrightarrow_R xzy$ **iff** wRz.

As can the language defined by a grammar:

> The **language defined by** grammar (V, Σ, R, S) is $\{\sigma \in \Sigma^* \mid S \Rightarrow_R^* \sigma\}$

We also use the notation $L(G)$ for the language defined by grammar G.

CLASSWORK

You might be wondering about the `|` and `?` and `*` and `+` symbols and why they do not appear in our formal definition. As it turns out, they are not needed: they are just shorthands ("syntactic sugar") for proper rules. We will work out in class exactly how they can be replaced with regular rules.

Terminology

The page you are reading calls V the set of **variables** *and Σ the* **alphabet**. *You may encounter folks who call V the set of* **non-terminals** *and Σ the set of* **terminals**. *No matter what the names, $V \cap \Sigma = \varnothing$.*

Well...actually...there are still other folks that call V a **vocabulary** *and make it the set of all variables plus the alphabet symbols and consequently tell you now that $\Sigma \subset V$.*

I know, I know. The moral here is that there ain't just one way to label all the ideas behind grammars. Different authors describe this stuff differently.

Can grammars be simplified?

The development of formal languages originated with early work by Emil Post, Axel Thue, and others working in the early 20th century on questions related to the foundations of mathematics and computation. Post created a system for transforming strings, called a Post Canonical System. We now know a much simpler formulation of Post's work, namely a semi-Thue system, a.k.a. string rewriting system. An SRS can rewriting any string into any other; it is nothing more than a tuple (Σ, R) where Σ is the alphabet of symbols and $R \subseteq \Sigma^* \times \Sigma^*$ is the rewriting relation.

That's it!

However, despite their simplicity, they are rather clunky to use to define languages. Chomsky's genius move was to add variables, making a special kind of a rewriting system known as a **grammar**. *Brilliant! This is why all computer scientists encounter grammars and only very few have heard of string rewriting systems.*

Restricted Grammars

In practice, it is often super-convenient to talk about grammars with restricted rule sets, as these may yield grammars which are much simpler to understand and work with. Of course, you lose the ability to derive as many languages as an unrestricted grammar, but the simplicity might be worth it. There are many dozens of known restricted grammar types, but there are three that you should know: Context-Free, Right-Linear, and Essentially Noncontracting.

Context-Free Grammar

A **CFG** is a grammar where $R \subseteq V \times (V \cup \Sigma)^*$, that is, the left hand side is **just a single variable**. So simple and clean, sensible and intuitive. Because of this, they've been massively successively successful and the foundation for the vast majority of structural definitions for real programming languages. Interestingly, they aren't actually powerful enough to capture all legal programs in most programming languages! We'll see later how to handle this problem in practice.

> ***Why are they called context free?***
>
> *Context-free grammars are so-named because variables can be rewritten regardless of the context in which they appear. (For example, context free grammars would allow say, an expression in a programming language, to be any identifier whatsoever at any time, regardless of whether one is in the context in which the identifier was previously declared or not.)*

Right-Linear Grammar

An **RLG** is a grammar where $R \subseteq V \times (\Sigma^* \cup \Sigma^* V)$, that is, the left hand side is just a single variable and the right hand side is a string of zero or more symbols followed by at most one variable. Pretty interesting: their derivations line up with the idea of a linear computation without memory. More on this later.

Essentially Noncontracting Grammar

In every rule in a **ENCG**, the right hand side never has fewer symbols and variables than the left hand side—with only one possible exception: if the language contains the empty string, we can have one rule that maps the start symbol to the empty string. The conceptual idea for a noncontracting grammar is that the derivation string never gets shorter; the **essentially** noncontracting grammars are those that have that one exception to allow for the empty string being in the language. These correspond to computations with bounded memory.

Language Recognition

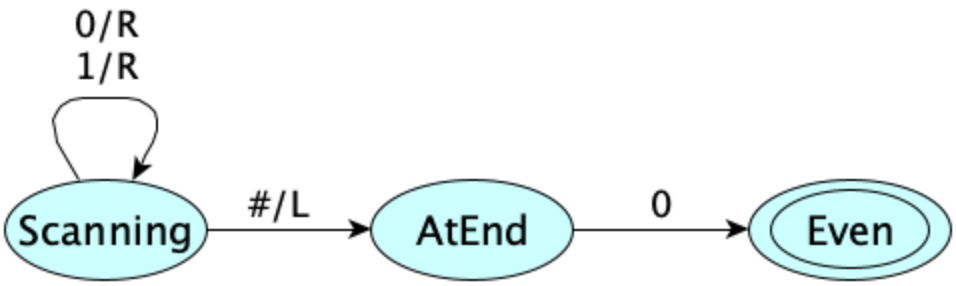
A grammar lets us formally specify a language by **generating** all and only those strings in the language. But we can define a language another way: we can give a process that, when given a string,

determines whether it belongs to a language?

Hey this is actually very cool! Such a process can be used in a compiler! Let's learn.

Automata

One way to define a language through recognition is to use automata with **accept states**. In earlier notes we saw a Turing Machine which accepted all and only those strings in the language of even binary numerals:



Both the grammar `even → ("0"|"1")*"0"` and the Turing Machine above "define" the same language, one in a generative fashion and the other in a recognitive fashion. Cool.

But there's more! Just like we showed several ways to ~~dumb-down~~ restrict grammars for the purpose of generating certain interesting classes of languages, Turing Machines can be restricted in several ways, too, each of which correspond to certain kinds of grammars. (This is either creepy, a massive coincidence, or something fundamental about information.) Languages and Automata are intimately related! For example, we can:

- Ditch the infinite tape and bound the tape to the size of the input plus one cell on either end, containing a left-end-marker-symbol on the left and a right-end-marker-symbol on the right. These cells cannot be changed nor can the TM head move off the ends. This is called a **linear bounded automaton**, (LBA). The languages they recognize are equivalent to those generated by a Type-1 Grammar.

- Restrict the machine to be read-only (never write anything) and only move right. This is called, and note this is a misleading name, a **finite automata** (FA). It is a poor name because it is not the only kind of machine with a fine number of states (all TMs have a finite number of states) and it is not the only kind with a finite memory (LBAs are bounded also). But FAs are very important: they define computations that *process one symbol at a time and don't require any scratch memory*. The recognize the same class of languages as are derived by RLGs.

- Add a second tape (called the "stack") and keep the input tape read-only and allow right-moves only. The next step is chosen based on both the current input symbol and the symbol(s) on the top of the stack, At each step you can pop symbols and push symbols. This contraption is called a **pushdown automata** (PDA). PDAs recognize exactly the context-free languages. PDAs can be restricted even further so that there is only one possible move to make at each step: this yields a "deterministic" PDA, known as a DPDA.

> ***Determinism***
>
> *Automata theory is a huge topic, and there is so much more to study. We'll get to it later. But there's one interesting aspect worth noting now: Machines that allow more than one possible operation from a given configuration are called* **nondeterministic**, *and those that allow only one are called* **deterministic**. *Okay, fine, but here's the philosophical question: is nondeterminism more "powerful" than determinism? That is, can we recognize more languages with nondeterministic machines of a given class than deterministic ones? It turns out:*
>
> - *For FAs and TMs, there is NO difference in recognition power between deterministic and nondeterministic versions.*
> - *For PDAs, there IS a difference in recognition power between deterministic and nondeterministic versions.*
> - *For LBAs, we don't know!*
>
> *Who knew right?* 🤯

Analytic Grammars

An **analytic grammar** is crafted with the idea of making recognition—going from candidate string to start symbol—much more natural. You don't think about generating all possible utterances from the start symbol; instead, you think of **matching** rules to a candidate string. This idea is pretty nice when you need to write compilers or interpreters.

The biggest difference between generative and analytic grammars is in the handling of alternatives. In a generative grammar G, $L(G)$ is contains all strings that *can* be generated by applying rules whenever possible. For example, the rule $x \rightarrow y \mid z$ means you can replace an x with an y or a z, so you can generate both. So the grammar:

$s \longrightarrow$ `"a"` | `"ab"`

describes the language $\{a, ab\}$.

In an analytic grammar, the rules are deterministic; you don't get to pick one or the other. So $A \,/\, B$ means: "Try to match A. If you can, the rule matches and you are done. Otherwise (and only otherwise), try to match B". So:

```
s ⟵ "a" / "ab"
```

recognizes the language $\{a\}$, but:

```
s ⟵ "ab" / "a"
```

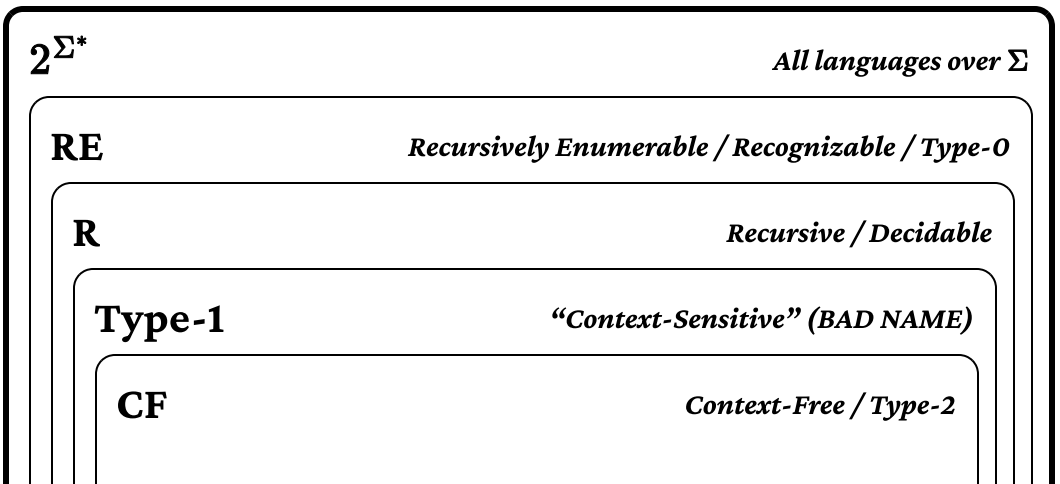
recognizes the language $\{a, ab\}$.

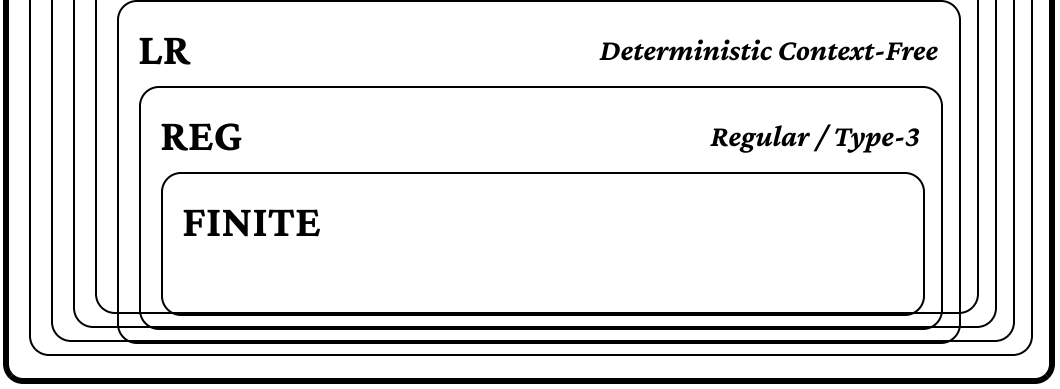
Unlike generative grammars, analytic grammars feature special **lookahead operators**, including the **negative lookahead operator** (~) and the **positive lookahead operator** (&). This enables the definition of non-context-free languages, despite having only one symbol on the left-hand side! Here's an analytic grammar for the non-context-free language $\{a^n b^n c^n \mid n \geq 0\}$:

```
s ⟵ & (x "c") "a"+ y
x ⟵ "a" x? "b"
y ⟵ "b" y? "c"
```

Formal Language Classification

Time for a little side exploration into how we can categorize languages. There are a gazillion ways to do so. Perhaps the first and certainly the most historically significant classification scheme is due to Chomsky, and is now known as the **Chomsky Hierarchy**:





The main language types in this hierarchy are listed below.

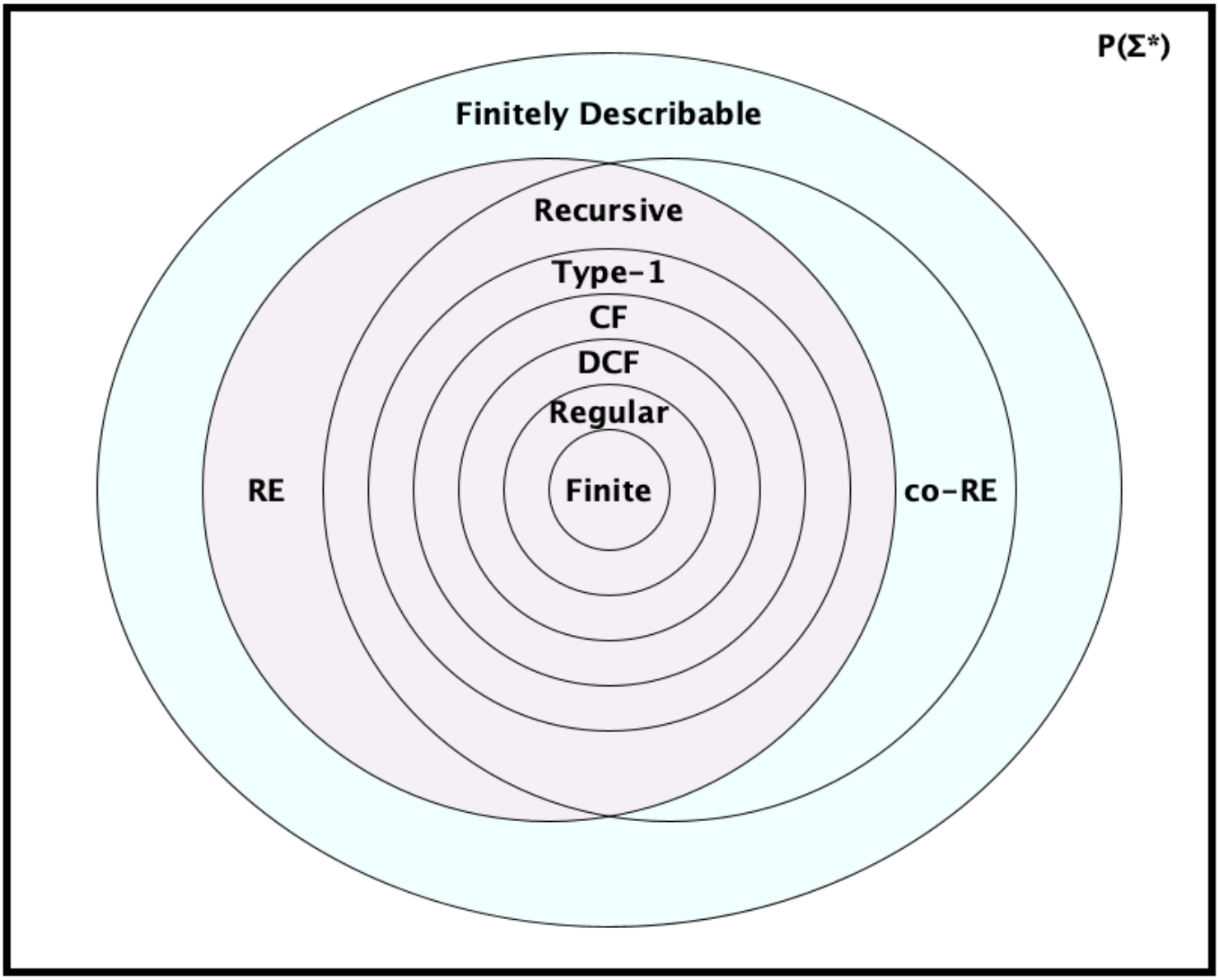
Language Type	Description	Grammar	Recognizer
Finite	A language with only a finite number of strings.	$S \to w_1 \vert \ldots \vert w_n$	Loop-free FA
Regular (Type 3)	A language in which strings can be recognized by processing them one symbol at a time, without backtracking, and **without using additional memory**.	RLG	FA
Deterministic Context Free (LR)	A language recognizable on a deterministic PDA. (Stacks are nice! $\Theta(1)$ access! Determinism is nice too! Much more efficient.)	DCFG	DPDA
Context Free (Type 2)	A language recognizable on a nondeterministic PDA. (Stacks are nice! $\Theta(1)$ access! But with nondeterminism, recognition is slower.)	CFG	PDA
Type 1	A language recognizable on an LBA. (Type 1 languages are often called "context-sensitive," but that is not a very accurate name because all languages "beyond" Type 1 in the hierarchy are sensitive to context).	ENCG	LBA
Recursive (R)	A language in which string membership is **decidable**. There are no predetermined memory bounds, and no bounds on the number of steps.	?	TM that always halts

Recursively Enumerable (Type 0 / RE)	A language in which string membership is **partially decidable**: if a string is in the language, you can determine that for sure; if a string is not, you might not be able to determine this.	(Unrestricted) Grammar	TM

We sometimes add:

- **co-RE**: A language whose complement is RE
- **Finitely Describable**: A language that has a finite description

Fun fact: $RE \cap co\text{-}RE = R$.



The language class RE lines up with what we understand to be **computable**.

The Chomsky Hierarchy is just a start. There are hundreds (no kidding) of other classes of languages. Perhaps the two most famous are:

- P: The set of all languages that can be recognized by a deterministic TM in a number of steps bounded by a fixed polynomial in the size of the input.
- NP: The set of all languages that can be recognized by a TM in a number of steps bounded by a fixed polynomial in the size of the input.

Note, by definition, $P \subseteq NP$, but no one has been able to prove whether or not $P = NP$. There is a cash prize waiting for *you* if you can do it.

There's so much more, including languages tied to probabilistic and quantum computing. All of the language families described above, as well as hundreds more, can be found in the Complexity Zoo.

> ***Exercise**: Research and provide definitions for these other classes of languages: LL, $LOGSPACE$, $PSPACE$, $EXPSPACE$, $EXPTIME$, $LALR$, SLR, BPP, RP, and $LINEARSIZE$.*

Semantics

Up to now, our only definition of "language" has been a set of strings over some alphabet, and we've looked at many ways to describe a set of strings. A description of the set of strings in the language is called a **syntax**. But you probably have a sense that languages exist to convey meaning. The way meaning is attached to utterances of a language is known as **semantics**.

For instance, we saw the language of simple arithmetic expressions above. We gave the grammar:

$$
\begin{array}{lcl}
exp & \longrightarrow & digit^{+} \\
 & | & \texttt{"-"}\ exp \\
 & | & exp\ (\texttt{"+"}\ |\ \texttt{"-"}\ |\ \texttt{"*"}\ |\ \texttt{"/"}\ |\ \texttt{"\%"}\ |\ \texttt{"**"})\ exp \\
 & | & \texttt{"("}\ exp\ \texttt{")"}
\end{array}
$$

and said the language **WAS** the set of strings generated by this grammar. To give the utterances meaning, we could define a function mapping every utterance to its meaning, which in this case would be a number.

> ***Exercise**: Look up the difference between a numeral and a number and make sure you understand it*

deeply.

It's customary to define the semantics of a language by a set of rules that show how to evaluate syntactic forms of the language. Here's a pretty straightforward definition:

$$\frac{}{[\![n]\!] \Downarrow n} \qquad \frac{e \Downarrow x}{[\![-e]\!] \Downarrow -x} \qquad \frac{e_1 \Downarrow x \quad e_2 \Downarrow y}{[\![e_1 + e_2]\!] \Downarrow x + y} \qquad \frac{e_1 \Downarrow x \quad e_2 \Downarrow y}{[\![e_1 - e_2]\!] \Downarrow x - y}$$

$$\frac{e_1 \Downarrow x \quad e_2 \Downarrow y}{[\![e_1 * e_2]\!] \Downarrow x \times y} \qquad \frac{e_1 \Downarrow x \quad e_2 \Downarrow y}{[\![e_1 \,/\, e_2]\!] \Downarrow x \div y} \qquad \frac{e_1 \Downarrow x \quad e_2 \Downarrow y}{[\![e_1 \,\%\, e_2]\!] \Downarrow x \bmod y}$$

$$\frac{e_1 \Downarrow x \quad e_2 \Downarrow y}{[\![e_1 ** e_2]\!] \Downarrow x^y} \qquad \frac{e \Downarrow n}{[\![(e)]\!] \Downarrow n}$$

How do we deal with semantics in formal language theory? Interestingly, semantics tends to live in a different theory—not in "formal" language theory, but rather in a theory affectionately known as PLT.

Programming Language Theory (PLT)

Formal Language Theory, it seems, is only concerned with defining or determining which strings are in a language and which are not. It is not concerned with assigning meaning to those strings. We have to go beyond what we've learned so far to deal with questions of meaning. Linguists study **semiotics**. Computer Scientists study **programming language theory**, otherwise known as PLT.

In PLT, **programming languages** are, yes, fundamentally sets of strings, but where each of the members of the language is a **program**, representing a computation. The concern of the syntactic aspect of the formal (contrived) languages we've seen so far is really just to provide a foundation for studying computability and complexity, while the concern of programming languages is for humans to express powerful computations. Programming languages feature extremely rich syntaxes and complex semantics, and introduce us to two worlds: the static and the dynamic.

PLT is a significant and very active branch of computer science that deals with everything about programming languages, including their design, classification, implementation, interpretation,

translation, analysis, features, expressiveness, limitations, syntax, semantics, and so on. It is concerned with:

Syntax

How do we formally specify the **structure** of programs?

Semantics

How do we formally specify the **meaning** of programs?

Type Theory

How do we classify constructs and entities in our programs (by their structure and behavior)?

Static Analysis

What can we say about a program just by looking at it (without running it)?

Translation

How do we construct **compilers** (translators) and **interpreters** (executors)?

Runtime Systems

What do programs look like at runtime, and what do we need to execute them?

Verification

Metapro-gramming

How do we mechanically prove program properties, such as correctness and security?

How do we write programs that operate on, and generate, other programs?

Classification

On what bases can we rigorously classify PLs in meaningful ways?

Basically, anything in the study of Programming Languages seems to fall under the topic of PLT. But in the popular culture, the PLT community seems to be associated with the most theoretical, the most mathematical, the most foundational. PLT people write papers whose formulas have a lot of Greek letters. Logic and category theory are everywhere. And when it comes to actual programming, the PLT community is all over the most cutting-edge stuff, like dependent types. And they appear to favor functional programming languages. Haskell, Scheme, and the ML Family of Languages show up a lot in current research. So do proof assistants, like Agda, Coq, and Lean.

This is good, though.

PLT is a huge field but there is good news for students: Some one has already built, and is currently maintaining, a most excellent curated portal for PLT. So many great links to books, papers, articles, tutorials, and even to other portals! Browse and lose yourself in the good stuff.

You can also begin a journey at Wikipedia's PLT article.

Recall Practice

Here are some questions useful for your spaced repetition learning. Many of the answers are not found on this page. Some will have popped up in lecture. Others will require you to do your own research.

Why do we need a theory of language? HIDE ANSWER

A theory of language allows us to rigorously define exactly how we may express and transmit information.

What is information and how do we conventionally represent it? HIDE ANSWER

Information is that which informs. Technically, it can be defined as a resolution of uncertainty. We represent it with a string of symbols.

What is an alphabet, in language theory? HIDE ANSWER

A finite set of symbols.

What is a string, in language theory? HIDE ANSWER

A finite sequence of symbols over a given alphabet.

How do we denote the empty string (the string of zero symbols)? HIDE ANSWER

ε

What is $|abbabcb|$? HIDE ANSWER

7

How do we denote the concatenation of two strings x and y? HIDE ANSWER

xy

What is ho^3? What is $(ho)^3$ HIDE ANSWER

hooo

hohoho

Why is the traditional representation of concatenation and repetition so annoying?

HIDE ANSWER

String concatentation is written like multiplication, but it is really a kind of addition. String repetition is written like exponentiation, but it is really a kind of multiplication.

What are the substrings of abc? HIDE ANSWER

$\varepsilon, a, b, c, ab, bc, abc$.

What is a language, in formal language theory? HIDE ANSWER

A set of strings over some finite alphabet.

What do we call a member of a language? HIDE ANSWER

An utterance.

What is the difference between $\varnothing$ and $\{\varepsilon\}$? HIDE ANSWER

The former is the language of zero strings; the latter is a language with one string, namely the empty string.

How do the Kleene Closure and the Positive Closure of a language differ? HIDE ANSWER

The Kleene Closure includes the empty string; the Positive Closure does not.

How do we denote the concatenation of two languages, L_1 and L_2? HIDE ANSWER

L_1L_2

What is $\{a, b, c\}^2$? HIDE ANSWER

The set of all strings of length 2 over the alphabet $\{a, b, c\}$, namely $\{aa, ab, ac, ba, bb, bc, ca, cb, cc\}$.

For language L, what are $L\varnothing$ and $L\{\varepsilon\}$? HIDE ANSWER

$\varnothing$ and L, respectively.

What are the two means by which we can formally, but "computationally," define a language, other than listing all the strings or providing a description with a set comprehension? HIDE ANSWER

By generation or recognition.

What is the primary mechanism for which languages are described by generation? HIDE ANSWER

The generative grammar.

What are **variables** in a generative grammar? HIDE ANSWER

Symbols that are distinct from those symbols in the language that the grammar is defining, that are intended to be replaced by (strings of) symbols in the process of generating strings in the language.

What do the metacharacters $|$, $*$, $+$, and $?$ mean in a generative grammar? HIDE ANSWER

Alternation (OR), repetition (Zero-or-more), positive repetition (One-or-more), and optional (Zero-or-one), respectively.

How do we denote the language defined by grammar G? HIDE ANSWER

$L(G)$

When taking about grammars, we sometimes see the metacharacters $\longrightarrow$ and $\Longrightarrow$. What do they mean? HIDE ANSWER

The former is used to denote a rule in a grammar; the latter is used to denote a derivation step.

When does a derivation in a generative grammar "stop"? HIDE ANSWER

When there are no more variables.

What is the single rule in the generative grammar defining the language of positive odd binary numerals? HIDE ANSWER

$odd \rightarrow$ `("0" | "1")* "1"`

What language is defined by the following generative grammar?

$s \longrightarrow$ `"a"` s? `"b"`

HIDE ANSWER

$\{a^n b^n \mid n \geq 1\}$

How do you write a grammar for the language of nested and balanced parentheses?

HIDE ANSWER

$s \longrightarrow$ `("("` s? `")")*`

Why was writing the grammar for $\{a^n b^n c^n \mid n \geq 0\}$ conceptually harder than writing the grammar for $\{a^n b^n \mid n \geq 0\}$? HIDE ANSWER

Because the former requires either (1) left hand sides with multiple symbols or (2) the use of lookahead operators.

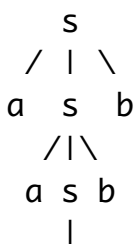
What are some predefined variables that are often used in grammars? HIDE ANSWER

digit, *hexDigit*, *letter*, *alnum*

For the grammar

$s \longrightarrow$ **"a"** $s?$ **"b"**

what does the derivation tree for the string ***aabb*** look like? HIDE ANSWER



The grammar

$s \longrightarrow x \mid y$
$a \longrightarrow x$
$y \longrightarrow b$

defines a language with exactly one utterance, namely ***ab***. But there are two possible derivations. Show them. HIDE ANSWER

- $s \Longrightarrow xy \Longrightarrow ay \Longrightarrow ab$
- $s \Longrightarrow xy \Longrightarrow xb \Longrightarrow ab$

What does it mean for a grammar G to be ambiguous? HIDE ANSWER

There exists a string in $L(G)$ with more than one derivation tree for G.

What are the four components of the mathematical structure representing a formal, generative grammar? HIDE ANSWER

The structure is (V, Σ, R, S) where V is the set of variables, Σ is the alphabet over which the language is defined, R is the set of rules, and S is the start symbol.

In a generative grammar, (V, Σ, R, S), we require $R \subseteq (V \cup \Sigma)^* V (V \cup \Sigma)^* \times (V \cup \Sigma)^*$. Explain this requirement. HIDE ANSWER

Rules map over strings of variables and language symbols (that is, strings from $(V \cup \Sigma)^*$), but the left hand side MUST have at least ONE variable.

Why does the formal definition of a grammar not include ?, $*$, $+$, or $|$? HIDE ANSWER

Because when doing super theoretical meta-level work, brevity and simplicity are key. We want the smallest possible number of building blocks to simplify discussions about what grammars are. We can define these constructs in terms of the basic constructs.

How is the derivation relation $\Longrightarrow_R$ in a generative grammar formally defined? HIDE ANSWER

$\{(xwy, xzy) \mid wRz\}$

What are alternate terms used by different authors for “variable” and language symbol”? HIDE ANSWER

“Nonterminal” and “terminal”

What do we call the union of variables and language symbols? HIDE ANSWER

The vocabulary

What is a context-free grammar (CFG)? HIDE ANSWER

A restricted form of a generative grammar in which the left hand side of every rule is a single variable.

What is a right-linear grammar (RLG)? HIDE ANSWER

A restricted form of a generative grammar in which the right hand side of every rule is a string of zero or more symbols followed by at most one variable.

What is an essentially noncontracting grammar (ENCG)? HIDE ANSWER

A restricted form of a generative grammar in which the right hand side of every rule never has fewer symbols and variables than the left hand side, with one exception: if the language contains the empty string, we can have one rule that maps the start symbol to the empty string.

What are two mechanisms to recognize languages? HIDE ANSWER

Automata and analytic grammars.

What is the most well-known and influential kind of automaton? HIDE ANSWER

The Turing Machine.

What restrictions of Turing Machines correspond to RLGs, CFGs, and ENCGs? HIDE ANSWER

Finite Automata, Pushdown Automata, and Linear Bounded Automata, respectively.

What is the difference between a deterministic and a nondeterministic automaton? HIDE ANSWER

Deterministic automata have only one possible transition to take when in a given configuration; nondeterministic automata have more than one.

What is the difference between a generative and an analytic grammar? HIDE ANSWER

Generative grammars generate all and only the strings in a language. Analytic grammars define a process to determing whether a candidate string is in the language or not.

What is the language defined by the analytic grammar $s \longleftarrow$ `"a"` / `"ab"`? HIDE ANSWER

The language $\{a\}$. (Importantly ab is not in the language because when presented with that string, the grammar matches the a part and will not backtrack to try the second alternative.)

Although analytic grammars have only a single variable on the left hand side of each rule, how can they define non-context-free languages? HIDE ANSWER

By using lookahead operators.

The classic arrangement of language classes in terms of computation power is known as the _____. HIDE ANSWER

The Chomsky Hierarchy

What is a finite language and what kinds of grammars generate them and what kinds of automata recognize them? HIDE ANSWER

A language with only a finite number of strings. They are generated by a grammar in which every rule maps the start symbol to a string in the language. They are recognized by a loop-free finite automaton.

What is a regular language and what kinds of grammars generate them and what kinds of automata recognize them? HIDE ANSWER

A language in which strings can be recognized by processing them one symbol at a time, without backtracking, and without using additional memory. They are generated by a right-linear grammar. They are recognized by a finite automaton.

What is a context-free language and what kinds of grammars generate them and what kinds of automata recognize them? HIDE ANSWER

A language in which strings can be recognized by processing them one symbol at a time, without backtracking, and by using only a simple stack for working memory. They are generated by a CFG. They are recognized by a PDA.

What is a Type 1 language and what kinds of grammars generate them and what kinds of automata recognize them? HIDE ANSWER

A language in which strings can be recognized with fixed apriori bounds on working memory. They are generated by an ENCG. They are recognized by an LBA.

What is the difference between a recursive and a recursively enumerable language? HIDE ANSWER

Recursive languages are decidable, that is, recognizable on a always-halting Turing Machine; recursively enumerable languages are partially decidable, meaning they can be recognized by a Turning Machine that always halts for YES answers but may loop forever on strings not in the language.

Arrange R, FINITE, RE, DCFL, CFL, and REG in subset order. HIDE ANSWER

FINITE $\subset$ REG $\subset$ DCFL $\subset$ CFL $\subset$ R $\subset$ RE

What is $RE \cap$ co-RE? HIDE ANSWER

R

What is a “computable language”? HIDE ANSWER

A language that is recursively enumerable.

What is the language class P? HIDE ANSWER

The set of languages that can be recognized by a deterministic TM in polynomial time.

What is the language class NP? HIDE ANSWER

The set of languages that can be recognized by a TM in polynomial time.

What is semantics concerned with? HIDE ANSWER

The meaning of utterances in a language.

What is the difference between a number and a numeral? HIDE ANSWER

A number is an abstract concept; a numeral is a representation of a number.

What are some of the concerns of PLT? Try to name at least five. HIDE ANSWER

Syntax, Semantics, Type Theory, Static Analysis, Translation, Runtime Systems, Verification, Metaprogramming, Classification.

Who is the curator of the GitHub portal for PLT? HIDE ANSWER

Steven Shaw

Summary

We've covered:

- ✅ Symbols, alphabets, and strings
- ✅ Formal Languages

- ✅ Grammars
- ✅ Parse Trees
- ✅ Ambiguity
- ✅ The formal definition of a grammar
- ✅ Context-Free, Right-Linear, and Type-1 Grammars
- ✅ Language Recognition
- ✅ The Chomsky Hierarchy
- ✅ A bit of history
- ✅ What Programming Language Theory is concerned with